Enterprise Investment Account and Transaction Management Platform: Architecture and Workflows

Project architecture and end-to-end workflow guide (synthetic educational implementation)

Synthetic demonstration project

Independently designed for engineering-interview preparation. It is not a real client system and uses no real customer data, credentials, proprietary APIs or confidential processes.

Document: PDF 1 of 2 - Project Architecture and End-to-End Workflow Guide

Stack: Java 21, Spring Boot 3.5, Spring Security (OAuth2 resource server), JPA/Hibernate, Flyway, PostgreSQL, Angular 20, Docker, Kubernetes, GitHub Actions, Prometheus, Grafana, OpenTelemetry

Synthetic data only. Educational implementation - not a production financial system.

Companion: SSNC_Senior_Lead_Java_Fullstack_Interview_Guide.pdf

Status wording used throughout: executed = run and passing in the build environment; written, not executed = present but not run here; conceptual = design discussion only.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Table of contents

Business problem and project objectives . . . **7**

The business problem . . . 7
Objectives . . . 7
Assumptions . . . 8
What this is not . . . 8

Functional and non-functional requirements . . . **9**

Roles . . . 9
Functional requirements . . . 9
Non-functional requirements . . . 10
Out of scope (explicitly) . . . 11

System context diagram . . . **12**

System context . . . 12

Component architecture . . . **14**

Decision: modular monolith, not microservices . . . 14
Component architecture . . . 15
Data ownership . . . 16
Communication and failure semantics . . . 16
Synchronous database transactions versus distributed workflows . . . 16

Angular frontend architecture . . . **18**

Stack and conventions . . . 18
Routing, guards, role-aware navigation . . . 19
HTTP layer . . . 19
State handling . . . 19
Forms and accessibility . . . 19
Money on the client . . . 20
Secure frontend practices . . . 20
Testing . . . 20

React frontend edition . . . **21**

What is deliberately identical . . . 21
Mapping of concepts . . . 21
Design notes . . . 22
Running it . . . 22
CORS and same-origin proxying (a finding from the first browser run) . . . 23
Verification (executed on the final code) . . . 23

Java and Spring Boot backend architecture . . . **24**

Technology choices (pinned) . . . 24
Package structure (modular monolith) . . . 24
Patterns used (and why) . . . 25

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Transaction boundaries in code . . . 25
Configuration . . . 26
Graceful shutdown . . . 26
Database ER diagram . . . 27
Choice: PostgreSQL (ADR-002) . . . 27
Schema . . . 28
Flyway migrations and safe schema evolution . . . 29
Indexes and why . . . 30
Query plans (real, captured on PostgreSQL) . . . 30
Connection pool, transactions, timeouts . . . 30
Useful SQL . . . 31
Data retention considerations . . . 31
Oracle in an enterprise deployment (conceptual - Oracle is not run here) . . . 31
MongoDB: where it would (and would not) fit . . . 32
Customer and account lifecycle . . . 33
Domain model in one paragraph . . . 33
Customer and account lifecycle . . . 33
Transaction lifecycle and state machine . . . 36
Transaction lifecycle and state machine . . . 36
Money rules . . . 37
Account transfer sequence diagram . . . 38
Account transfer: end-to-end sequence . . . 38
Idempotency and concurrency-control flow . . . 40
Idempotency and concurrency control . . . 41
Authentication and authorization . . . 44
Authentication and authorization flow . . . 44
Deny by default and hardening . . . 45
OWASP mapping . . . 46
Educational implementation versus a production financial system . . . 46
REST API examples . . . 47
Resource model and endpoints . . . 47
HTTP semantics and status codes . . . 47
The error envelope . . . 48
Pagination, filtering, sorting . . . 48
Idempotency, correlation, limits . . . 49
Versioning and backward compatibility . . . 49
Example requests and responses . . . 49
Captured examples (real output of the running application) . . . 49

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Database transaction boundaries . . . **64**
Database transaction boundaries (processing) . . . 64
Error-handling and recovery flows . . . **67**
Error handling and recovery for the external call . . . 68
CI/CD pipeline . . . **70**
CI/CD pipeline . . . 70
Local infrastructure . . . 71
Cloud deployment architecture . . . **72**
Kubernetes . . . 72
AWS reference architecture . . . 73
Observability architecture . . . **75**
Logs . . . 75
Metrics . . . 76
Traces . . . 76
Health . . . 76
From signal to action . . . 76
Security threat model . . . **77**
Trust boundaries and actors . . . 77
Threat register . . . 77
Residual risks and next steps (production) . . . 79
OWASP mapping . . . 80
Test strategy . . . **82**
Principles . . . 84
Layers . . . 84
Defects found by the test suite during development (kept as regression tests) . . . 85
Test data . . . 86
Coverage . . . 86
How to run . . . 86
Complete TDD example: daily withdrawal limit . . . 86
BDD scenarios . . . 88
AI-assisted development workflow . . . **90**
Layout . . . 90
Context engineering: small always-on file, modular on-demand files . . . 90
The workflow (specification-driven, human in the loop) . . . 90
Guardrails when using an assistant . . . 91
What assistants are good and bad at here . . . 91
Diagram: the specification-driven workflow . . . 92
Workflow example: implementing the account transfer with an assistant . . . 93

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Reviewing AI-generated code: five failure patterns with before/after . . . 93
Architecture mind map . . . 96
Troubleshooting scenarios . . . 97
Performance and reliability investigation guide . . . 98
Production runbooks . . . 100
Architecture decisions and trade-offs . . . 103
ADR-001: Modular monolith instead of microservices . . . 103
ADR-002: PostgreSQL as the primary relational database . . . 103
ADR-003: Explicit transaction boundaries; the external call is never inside a database transaction . . . 104
ADR-004: Idempotency keys enforced by a database unique constraint . . . 105
ADR-005: Pessimistic row locks (ordered) for balances, optimistic versioning elsewhere . . . 105
ADR-006: Transactional outbox with an in-process dispatcher; no message broker, no Redis (yet) . . . 106
ADR-007: URI versioning, strict input, committed OpenAPI contract . . . 107
ADR-008: Stateless JWT resource server with two-layer authorization . . . 108
ADR-009: GitHub Actions pipeline with security gates; container images promoted unchanged; Kube... . . . 108
ADR-010: Money as BigDecimal / NUMERIC(19,4) / JSON string, rejected (not rounded) excess precisi... . . . 109
ADR-011: Reserve funds first; classify external outcomes; reconcile the unknown . . . 109
ADR-012: Access token held in memory only in the SPA . . . 110
ADR-013: Two frontends, one API contract . . . 110
Known limitations and future enhancements . . . 112
What was executed, what was not (summary; details in the validation report) . . . 112
Functional limitations . . . 112
Technical limitations . . . 113
Security limitations . . . 113
Future enhancements (prioritised) . . . 113
Appendix A: Captured PostgreSQL query plans . . . 115
Account history for a SPARSE account (planner switches to the per-account indexes) . . . 115
Account history for a BUSY account (5,000 rows): newest-first walk of ix_tx_created with a filter . . . 115
Operations attention queue (partial index ix_tx_attention) . . . 115
Idempotency lookup (unique index uq_idempotency_principal_key) . . . 116
Outbox polling (ix_outbox_status_available) . . . 116
Daily withdrawal limit sum (ix_tx_source_created) . . . 116
Row lock taken when applying a transfer (primary key) . . . 116
Open reconciliation records (partial unique index uq_recon_open_per_tx) . . . 116
Appendix B: Validation summary . . . 117
1. Executed and passing . . . 117
2. Written but NOT executed . . . 119
3. Honest caveats . . . 119

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Appendix C: Operations console screenshots **120**
Appendix D: React console screenshots **131**

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Business problem and project objectives

The business problem

A financial-services organisation runs several internal applications (advisor desktops, a customer portal, operations tooling, reporting). Each of them needs the same things from a shared back end:

- a **single source of truth** for customers, investment accounts and balances;
- a **safe way to submit money movements** - deposits, withdrawals and transfers - that cannot overspend an account, cannot be applied twice when a client retries, and cannot leave a transfer half-applied;
- **explicit business rules** (account status, ownership, currency, funds, limits, review thresholds) enforced in one place;
- **auditability**: who did what, to which record, when, and from which request;
- **operational visibility**: is the platform healthy, what is stuck, what failed, what needs a human decision?

The platform in this repository - *Enterprise Investment Account and Transaction Management Platform* - is a deliberately realistic but **synthetic** answer to that problem. It was designed to demonstrate the engineering expected of a senior or lead Java full-stack engineer at a financial-technology company, **without** claiming to reproduce any real vendor's proprietary system. All customers, account numbers and balances are fictional; no real money is moved, no real brokerage is contacted.

Objectives

#	Objective	Where it is demonstrated
1	Correct money handling: `BigDecimal` with a written rounding policy, DB `CHECK` constraints, string money in the API	`MoneyPolicy`, `V1__core_schema.sql`, `MoneyPolicyTest`
2	Transactional integrity under concurrency: row locks in a fixed order, idempotency keys, outbox, ledger	`LedgerService`, `TransactionService`, `ConcurrencyScenarios`, ADR-003/004/005
3	Safe integration with an external system that cannot join our DB transaction	`TransactionProcessor`, `RestClearingClient`, `ReconciliationService`, ADR-006
4	Defence in depth: JWT, roles, object-level authorization, audit, rate/size limits, masking	`SecurityConfig`, `AccessPolicy`, `SecurityIT`, threat model
5	A professional Angular client: role-aware routes, typed API layer, interceptors, accessible forms, honest state handling	`frontend/`
6	Quality engineering: layered tests, TDD example, BDD scenarios, contract test, E2E	`docs/testing`, `tests/`, `*IT.java`
7	Delivery and operations: CI/CD with security gates, containers, Kubernetes, observability, runbooks	`.github`, `infrastructure`, `observability`, `docs/operations`
8	Technical leadership artefacts: ADRs, HLD/LLD, risk register, review checklists, mentoring material	`docs/decisions`, `docs/leadership`
9	Responsible AI-assisted development: instructions, spec-driven slice, review recipes	`ai-engineering/`, `specs/`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Assumptions

1. **Synthetic everything.** Customers, taxpayer fragments, balances and the external "clearing network" are simulated. Dummy credentials exist only in the `local` profile and are supplied through environment variables, never committed.
2. **Currencies** are USD, EUR, GBP and INR (two minor-unit digits); amounts are stored as `NUMERIC(19,4)` for headroom.
3. **Withdrawals leave the platform** (a payout through the external clearing network); deposits and internal transfers do not. That asymmetry is what makes the failure-handling design interesting and honest.
4. **One customer owns an account; transfers stay inside one customer.** (Cross-customer transfers would add payee management and sanctions screening that are out of scope.)
5. **Identity provider is external.** Production-style validation uses an OIDC issuer; a local issuer exists only so the demo runs without infrastructure.
6. **PostgreSQL** is the system of record (ADR-002). H2 in PostgreSQL mode is a zero-install convenience for local runs and fast tests; the PostgreSQL behaviour is verified separately against a real PostgreSQL server.
7. **Single deployable (modular monolith)** plus one small simulator service (ADR-001).

What this is not

This is an **educational, interview-oriented implementation**. A production financial-services system would additionally need, at minimum: a security and compliance review (SOC 2 / PCI DSS scope assessment, data-protection impact assessment), real KYC/AML and sanctions screening, regulatory reporting and record-retention rules, a formally reviewed key-management and secrets design, penetration testing, disaster-recovery rehearsals with agreed RPO/RTO, capacity testing on production-like infrastructure, segregation of duties in the delivery pipeline, and a governance process around the AI tooling. The "known limitations" chapter lists the gaps this repository knows about.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Functional and non-functional requirements

Roles

Role	Can	Cannot
`CUSTOMER`	see own customer record, accounts, balances, history; submit deposit/withdrawal/transfer on **own** accounts; cancel own pending transactions	see anyone else's data; use staff pages; approve anything
`SUPPORT_AGENT`	look up customers (PII masked), accounts, transactions, attempts, audit trail, dashboard	change anything; submit money movements
`OPERATIONS_ANALYST`	everything a support agent can, plus: create customers, open/activate/suspend accounts, submit on a customer's behalf, approve/reject transactions **submitted by someone else**, resolve reconciliation records	close accounts, suspend customers, platform administration
`ADMIN`	everything operations can, plus: suspend/reactivate customers, close accounts, drain/requeue outbox events, view dead letters	approve a transaction they submitted (maker-checker applies to everyone)

Functional requirements

ID	Requirement	Realised by
F1	Manage customer profiles (create, update, suspend, reactivate, search)	`CustomerService`, `CustomerController`
F2	Multiple accounts per customer; types `BROKERAGE`, `RETIREMENT`, `MONEY_MARKET`; currency per account	`Account`, `AccountType`
F3	Account lifecycle: `PENDING_ACTIVATION -> ACTIVE <-> SUSPENDED -> CLOSED`; activation requires an active customer; closing requires zero balance and nothing in flight	`AccountStatus`, `AccountService`
F4	Paginated, filterable, sortable account and transaction searches	`AccountService.search`, `TransactionService.search`, `PageableSupport`
F5	View balances (ledger, minimum, available) and transaction history	`AccountController`, `TransactionController`
F6	Submit deposits, withdrawals and internal transfers	`TransactionService.submit`
F7	Validate: account status, ownership/authorization, amount, currency, available balance, daily limit, duplicates, permitted state transitions	`TransactionValidator`, `AccessPolicy`, `TransactionStatus`, idempotency table
F8	Nine-state lifecycle `RECEIVED, VALIDATED, PROCESSING, COMPLETED, REJECTED, FAILED, PENDING_REVIEW, RECONCILIATION_REQUIRED, CANCELLED` with explicit transitions	`TransactionStatus`, `FinancialTransaction.transitionTo`
F9	Large transactions (>= 50,000) require an independent approver (maker-checker)	`TransactionService.approve`, `AccessPolicy.canReview`
F10	Cancel a transaction only before processing starts	`TransactionStatus.isCancellable`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

ID	Requirement	Realised by
F11	Atomic balance updates; a transfer is all-or-nothing; no overdraft	`LedgerService`, `ck_accounts_balance_nonneg`
F12	Idempotent submission with `Idempotency-Key`; duplicate detection; replay	`IdempotencyRecord`, `TransactionService`
F13	Uncertain external outcomes are held and reconciled (automatically, then manually)	`ReconciliationService`, `TransactionMaintenance`
F14	Audit every important change and every access denial	`AuditService`, `audit_events`
F15	Operations dashboard: summaries, volumes, statuses, pending/failed/reconciliation queues, latency, error rates, health, audit	`OpsService`, Angular `DashboardComponent`
F16	Administrative operations: outbox dead letters, requeue, ledger integrity check, drain	`OpsController`

Non-functional requirements

Area	Requirement	How it is met / verified
Correctness	No floating point for money; two-decimal currencies; HALF_EVEN rounding; overdraft impossible even through direct SQL	`MoneyPolicyTest`, `SchemaConstraintsIT`
Consistency	Debit and credit of a transfer commit atomically; ledger always equals balances	`ConcurrencyScenarios`, `/ops/ledger-integrity` after every scenario
Concurrency	Concurrent requests on one account serialise; opposing transfers cannot deadlock	ordered row locks, `ConcurrencyIT` (H2) and `PostgresConcurrencyIT` (PostgreSQL)
Reliability	Processing survives crashes and duplicate delivery; unknown outcomes never lose or double-spend money	outbox, idempotent processor, stuck-transaction recovery, reconciliation
Resilience	Timeouts, retry with exponential backoff + jitter (idempotent calls only), circuit breaker, bulkhead by design (async worker)	`RestClearingClient`, `RetryAndRateLimitTest`, chaos e2e
Security	OAuth2/OIDC-compatible JWT validation, RBAC, object-level authorization, rate limit, request-size limit, CORS allow-list, secure headers, no sensitive data in logs or errors	`SecurityIT`, threat model, OWASP mapping
Performance (targets, illustrative)	p95 read < 300 ms, p95 submit < 500 ms, error rate < 1 %	`performance/k6/lib.js` thresholds; **measured** local baseline in `docs/validation`
Availability	Stateless service instances (>= 2), graceful shutdown, rolling updates without downtime, readiness/liveness probes	`infrastructure/k8s`, probes, `server.shutdown=graceful`
Observability	Structured JSON logs with correlation id, RED metrics + business metrics, health, distributed tracing	`PlatformMetrics`, `observability/`
Maintainability	Modular monolith with one-directional dependencies, pure domain rules, small classes, ADRs	`docs/decisions`
Portability	Runs with zero infrastructure (H2) and on PostgreSQL; containers; Kubernetes manifests	`application*.yml`, `infrastructure/`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Area	Requirement	How it is met / verified
Accessibility	Labelled form controls, keyboard operable, status not conveyed by colour alone, live regions for async state	`frontend` templates, lint rules `templateAccessibility`
Data retention	Idempotency records purged after 7 days; audit trail append-only; documented retention for the rest	`TransactionMaintenance`, `docs/architecture/06-database.md`

Out of scope (explicitly)

Real brokerage connectivity, order execution, market data, tax lots, corporate actions, multi-currency conversion, fees and interest accrual, statements, sanctions/AML screening, multi-tenancy across legal entities (see the multi-tenant discussion in the interview guide), and regulatory reporting.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

System context diagram

System context

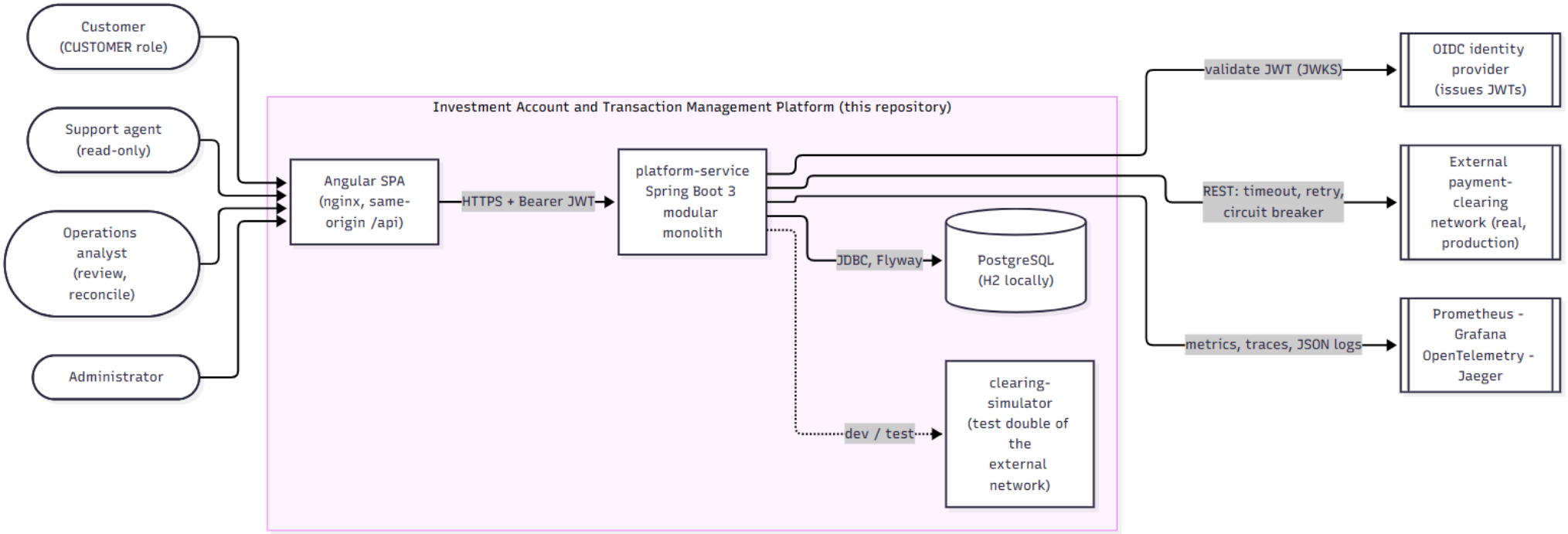


Figure 1. System context

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

The platform exposes one REST API (`/api/v1`) consumed by an Angular single-page application. Staff and customers
provider; the API validates the JWT. Money movements that leave the platform (withdrawals) call an external **clearin**
and tests that network is replaced by the `clearing-simulator`, a small Spring Boot service with deterministic and cha
configured at all, an in-process approve-everything substitute is used so the application runs stand-alone.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Component architecture

Decision: modular monolith, not microservices

The assignment asks for a modular architecture and says to introduce separate deployable services *only when the separation provides meaningful benefits*. The decision (full text: ADR-001):

Chosen: one Spring Boot deployable (`platform-service`) with clear internal modules - `customer`, `account`, `transaction` (ledger, processing, reconciliation, outbox), `audit`, `ops`, `security`, `clearing`, `common` - plus one small second service (`clearing-simulator`) that exists because it stands in for a genuinely separate system.

Why:

Concern	Modular monolith	Microservices (customer / account / transaction / audit)
Transfer atomicity	one local ACID transaction debits and credits	needs a saga or 2PC across services; partial failure states appear and must be compensated
Idempotency, ledger = balance invariant	enforced by constraints and row locks in one database	spread over several databases; invariants become eventually consistent
Operational cost	one pipeline, one database, one set of dashboards	N pipelines, service discovery, distributed tracing mandatory, N databases, contract tests between services
Team fit	one team	pays off with several teams that must release independently
Failure isolation	a bad deploy affects everything (mitigated by rolling updates and the async worker)	blast radius per service
Scaling	scale the whole unit; the DB is the bottleneck anyway	scale hot spots independently

The *money invariants* (a balance can never differ from its ledger; a transfer is all-or-nothing) are the reason **not** to split `account` from `transaction`: they need one transaction boundary. Splitting them would create exactly the distributed transaction the brief warns against creating "merely to demonstrate microservices".

Alternatives considered: (a) microservices per module - rejected for the reasons above; (b) a modular monolith with a separate worker deployment for the outbox dispatcher - a cheap first extraction, noted in the migration path; (c) event sourcing - rejected as unnecessary complexity for this scope.

Migration path (when the pressure appears): the module boundaries are already enforced by package-level dependencies (`account` never imports `transaction`; the `AccountClosureGuard` port inverts that dependency). Extraction order:

1. run the outbox dispatcher/processor as a separate deployment of the same artefact (`app.processing.dispatcher-enabled`);
2. move `audit` + `ops` read models behind events (they are read-mostly) into a reporting service with its own store;
3. only then consider splitting `customer` (low write rate, no money invariants);
4. keep `account` + `ledger` + `transaction` together until a measured bottleneck proves otherwise.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Component architecture

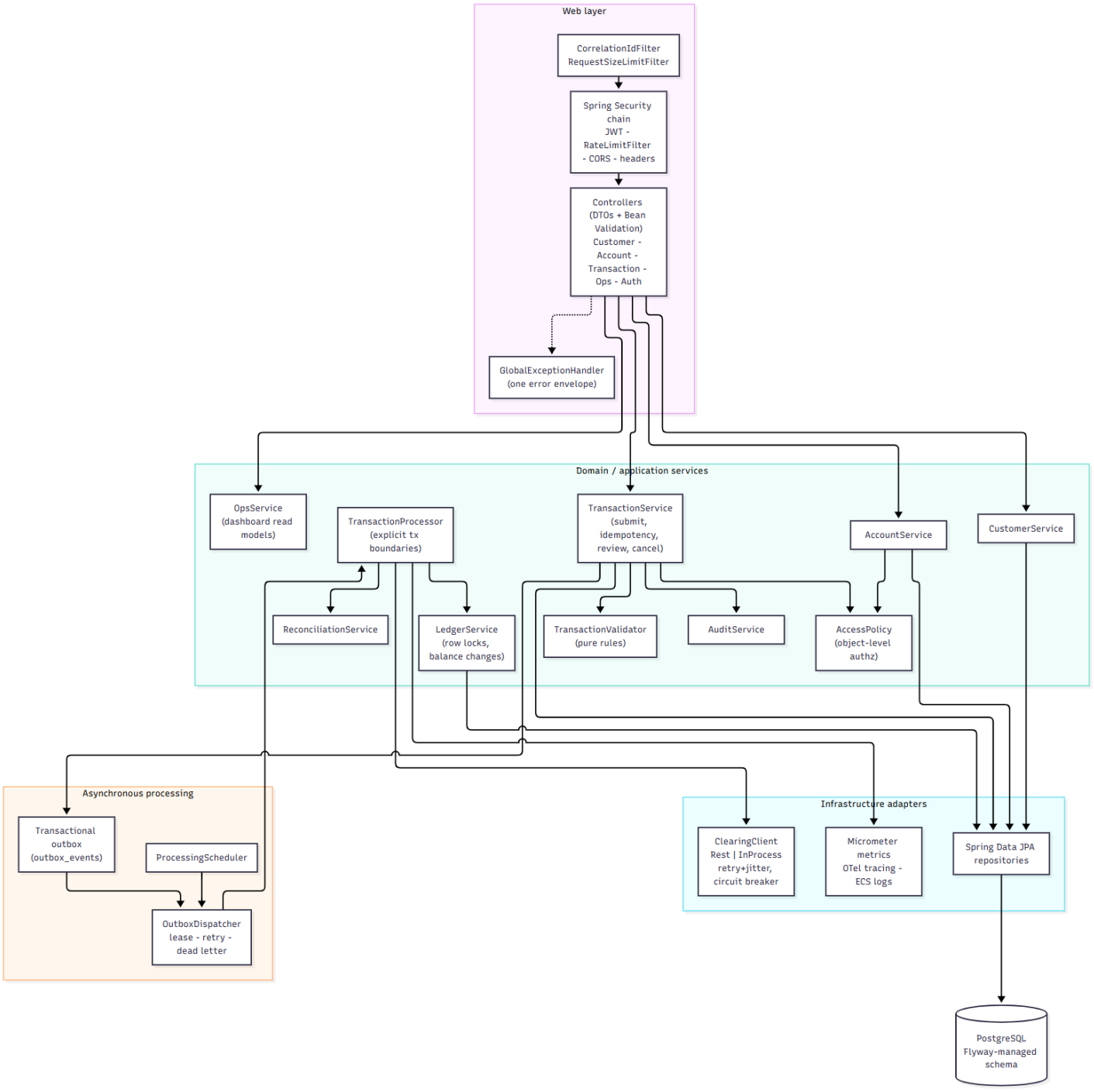


Figure 2. Component architecture

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **Web layer** - servlet filters (correlation id, request-size limit), the Spring Security chain (JWT validation, rate limit, CORS, headers), thin controllers that only translate HTTP to service calls and DTOs, and one `GlobalExceptionHandler` that produces the standard error envelope.
- **Domain/application services** - business rules live here, not in controllers or entities' persistence annotations. `TransactionValidator` and `AccessPolicy` are **pure** (no I/O) so each rule is a one-line unit test.
- **Asynchronous processing** - submission validates synchronously and records an outbox event in the same database transaction; `OutboxDispatcher` delivers it at-least-once to `TransactionProcessor`.
- **Infrastructure adapters** - Spring Data repositories, the `ClearingClient` port (REST adapter with retry/jitter and a circuit breaker, plus an in-process substitute), Micrometer/OpenTelemetry.

Data ownership

There is one database and one schema; every table is owned by exactly one module's service layer (`customers` by `CustomerService`, `accounts` by `AccountService` and balance changes **only** through `LedgerService`, `transactions`/`ledger_entries`/ `idempotency_records`/`transaction_attempts`/ `reconciliation_records`/`outbox_events` by the transaction module, `audit_events` append-only by `AuditService`). Controllers never touch repositories directly (the one exception, the read-only audit listing, is documented in the code).

Communication and failure semantics

Interaction	Style	Timeout / retry	Failure isolation
SPA -> API	HTTPS REST, JSON	browser/nginx timeouts (30 s)	error envelope with correlation id
API -> clearing network	REST, **idempotent on** `reference`	connect 500 ms, read 2 s, 3 attempts, exponential backoff with full jitter	circuit breaker (50 % of 10 calls) -> fast "not sent" failure + refund; unknown -> reconciliation
API -> DB	JDBC	Hikari: 3 s to get a connection, 5 s lock wait on account rows	bounded waits turn pile-ups into fast retryable `503 LOCK_TIMEOUT`
Submit -> processing	transactional outbox, in-process	claim by lease, backoff `2s * 2^n` (max 5 min), 5 attempts	dead-letter state + alert; consumer is idempotent
Correlation	`X-Correlation-Id` in/ out, MDC, trace id	-	every log line and audit row carries it

Synchronous database transactions versus distributed workflows

A **database transaction** gives atomicity and isolation *inside one database*. It cannot make an HTTP call to another company roll back. The platform therefore draws a hard line (see the transaction-boundaries diagram in the integrity chapter):

- everything that is purely about **our** data - validating, reserving funds, posting a transfer, recording a state change, writing the audit row and the outbox event - happens inside local ACID transactions;
- everything that involves an **external system** - the clearing authorization - happens *outside* any database transaction and is modelled as a small **workflow with explicit states** (`PROCESSING -> COMPLETED | FAILED | RECONCILIATION_REQUIRED`), idempotent calls, and a reconciliation path for the "did it happen?" case.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

The platform never claims distributed exactly-once. It provides **at-least-once delivery plus idempotent consumers plus reconciliation**, which together produce *effectively-once* effects on our own data and a *detectable, repairable* state for the one thing we cannot control.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Angular frontend architecture

> A React edition of the same console lives in `frontend-react/` - see 12-react-frontend.md and ADR-013.

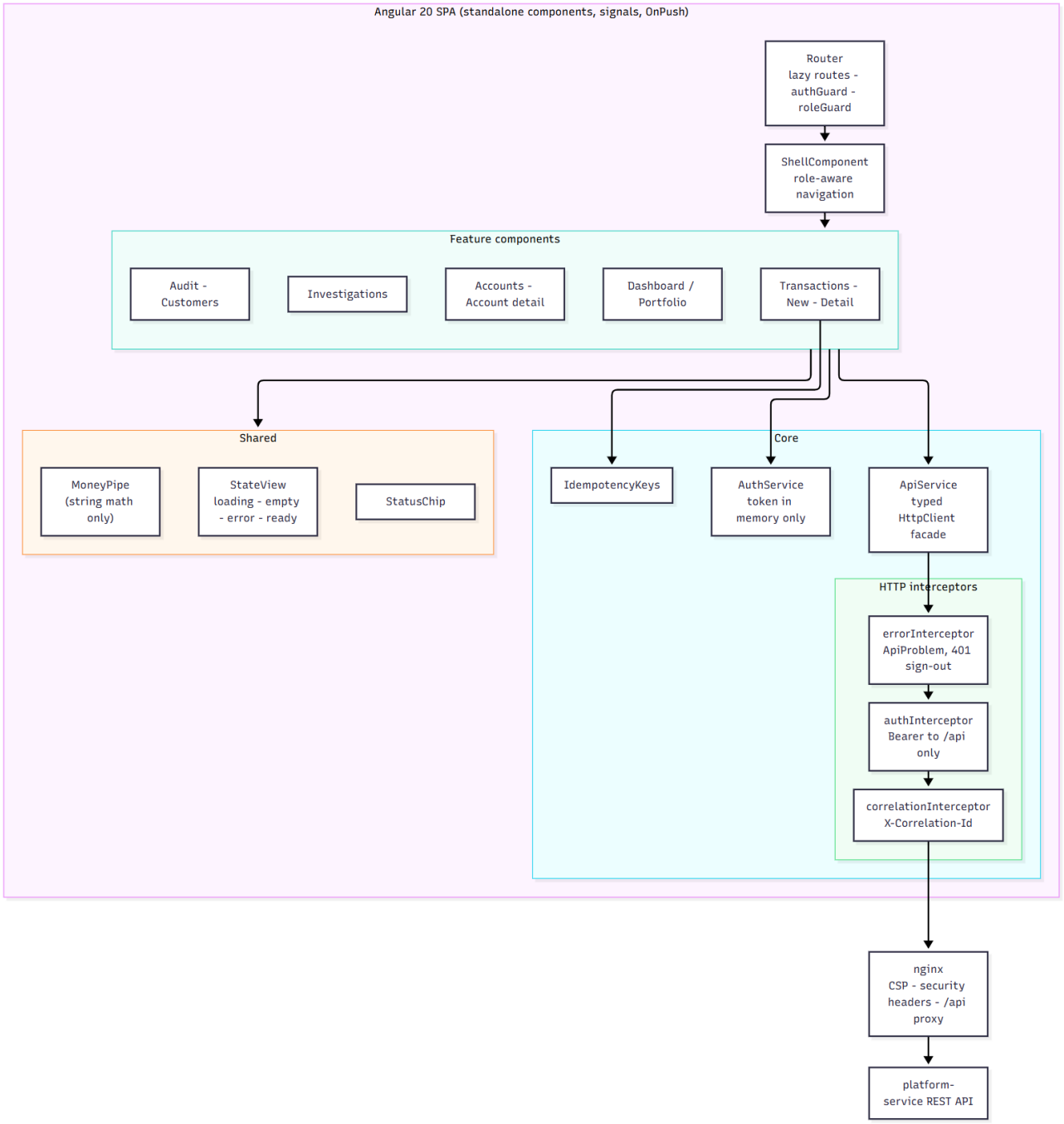


Figure 3. Frontend architecture

Stack and conventions

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Angular 20 (standalone components, no NgModules), TypeScript strict mode, Angular Material 20 for accessible building blocks, RxJS for HTTP streams and **signals** for component state (`signal`, `computed`, `input()`), `OnPush` change detection everywhere, lazy-loaded routes, Reactive Forms, `HttpClient` with functional interceptors, Karma + Jasmine for unit/component tests, ESLint with `angular-eslint` including the template-accessibility rules. No React/Vue/JSX anywhere.

```
src/app/
  core/       auth.service (token in memory), api.service (typed facade), interceptors, guards,
idempotency keys, models
  shared/     money.pipe, status-chip, state-view (loading/empty/error/ready), transaction-table
  features/   login, shell (role-aware nav), home (dashboard | portfolio), accounts, customers,
transactions, investigations, audit
```

Routing, guards, role-aware navigation

- Every route except `/login` is behind `authGuard` (redirects to `/login?returnUrl=...`; the login component only follows in-app relative return URLs, defeating open redirects).
- Staff-only pages add `roleGuard` with `data.roles`; the navigation list in `ShellComponent` is filtered by the same roles. **This is UX, not security** - the API enforces everything (a customer who forces `/audit` gets the "not permitted" page and, if they call the API directly, a 403).
- Routes are lazy (`loadComponent`): the initial bundle is ~340 kB raw / ~97 kB transferred; each feature loads on demand.

HTTP layer

Piece	Responsibility
`ApiService`	one typed method per endpoint; builds params with `toParams` (drops empty filters, repeats array params); **no business rules**
`authInterceptor`	adds `Authorization: Bearer` to `/api/` calls only, never to the login call or third-party URLs
`correlationInterceptor`	adds `X-Correlation-Id` so a click can be found in server logs
`errorInterceptor`	normalises every failure into an `ApiProblem {status, code, message, violations, details, correlationId}`; a `401` on a protected call signs the user out; network errors get a friendly message
`IdempotencyKeys`	reuses the same key while the payload is unchanged (safe retry, double click), issues a new key when the user edits, resets after a definitive outcome

State handling

Every data view uses the same four states through `StateViewComponent`: **loading** (spinner in a polite live region), **error** (alert role + retry button), **empty** (explanatory text), **ready**. The dashboard keeps the last good data and shows a non-blocking warning when an automatic refresh fails (it refreshes every 15 s while the tab is visible).

Forms and accessibility

Reactive Forms with typed controls; every control has a visible label; errors are announced (`mat-error`, `role="alert"`); the submit button disables while a request is in flight (a second submit is ignored); the new-transaction form applies the same shape rules as the server (deposit needs a target, withdrawal a source, transfer two different accounts) and the same amount pattern (positive, <= 2 decimals) - **client validation is a convenience; the server is authoritative**. Status is never conveyed by colour alone (chips carry text), the sidenav marks the current page with `aria-current`, and a CSS-only responsive layout (checked at 390 px by the browser E2E test) collapses the sidenav to an overlay.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Money on the client

The API returns money as **strings**; the SPA never converts them to numbers. `MoneyPipe` groups digits by string manipulation (tested with a value beyond 2^53 to prove no float rounding), and the form submits the typed string as-is.

Secure frontend practices

- Token in memory only; no `innerHTML`/`bypassSecurityTrust*` anywhere (Angular's template sanitisation is left on);
- strict CSP delivered by nginx (`default-src 'self'`; scripts same-origin only);
- same-origin deployment (`/api` proxied by nginx or the dev server) so no CORS and no third-party origins;
- returnUrl validation, no secrets in the bundle, `npm audit` in CI.

Testing

58 unit/component tests (Karma + Jasmine in headless Edge/Chrome): money pipe, status chip, state view, auth service (memory-only token, expiry), all three interceptors, guards, `ApiService` URL/param building, idempotency key policy, the new-transaction form (validation, double-submit protection, 422 handling, safe retry with the same key, new key on edit, replay notice), the dashboard (loading, ready, error + retry, stale-data warning, empty states), login (validation, 401/429 messages, open-redirect guard), role-aware navigation, maker-checker button visibility on the transaction page, investigations resolve flow. A browser E2E script (`tests/e2e/ui-e2e.mjs`) drives the production bundle through the customer, operations and support journeys against the live backend.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

React frontend edition

The platform ships **two interchangeable user interfaces** for the same REST API: the Angular 20 console in `frontend/` and a React 19 console in `frontend-react/`. Neither needs any backend change: both speak the contract in `specs/openapi.json` (the TypeScript types in `core/models.ts` are identical, character for character, in both projects).

	Angular edition	React edition
Folder	`frontend/`	`frontend-react/`
Stack	Angular 20.3, Angular Material 20.2, RxJS, signals, standalone components	React 19.3, React Router 7.18, Vite 8, TypeScript 5.9, plain CSS
State	signals + `computed`	`useState` / `useRef` / `useMemo`, one `useAsync` hook for loading
HTTP	`HttpClient` + three interceptors	`fetch` behind `createHttp()` (same three behaviours)
Forms	reactive forms	controlled inputs and pure validation functions
Route protection	`authGuard`, `roleGuard`	`<RequireAuth>`, `<RequireRole>`
Unit tests	Karma + Jasmine, 58 specs	Vitest + Testing Library, 90 tests
Default port	4280	4281
Initial JavaScript	about 341 kB	about 279 kB (89 kB gzip), plus lazy route chunks of 3 to 9 kB

What is deliberately identical

- **Contract and money handling.** Amounts are strings end to end; `formatMoney` is string manipulation only, never a floating-point number.
- **Security behaviour.** Access token held in memory only (never in web storage), bearer token on `/api/` calls except the login call, a fresh `X-Correlation-Id` on every API call, 401 on a protected call ends the session and returns to the login page with a validated `returnUrl` (no open redirect), roles shape navigation only (the server decides).
- **Idempotency.** `IdempotencyKeys` is the same class: one key per logical payload, reused on retry after an unknown outcome, reset after a definitive outcome (accepted or business-rejected).
- **Screens and wording.** The same ten screens with the same texts, so one browser end-to-end suite drives both (`tests/e2e/ui-e2e.mjs --flavor angular|react`; only the control selectors differ).
- **Error model.** Every failure becomes an `ApiProblem` (`status`, `code`, `message`, `violations`, `details`, `correlationId`); components never see raw HTTP errors.

Mapping of concepts

Angular	React edition
`AuthService` (signals)	`AuthProvider` + `useAuth()` in `core/auth.tsx`
`ApiService`	`createApi(http)` in `core/api.ts`, reached through `useApi()`
`authInterceptor`, `correlationInterceptor`, `errorInterceptor`	`createHttp()` in `core/http.ts`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Angular	React edition
`authGuard`, `roleGuard`	`RequireAuth`, `RequireRole`
`app-state-view`	`StateView` (loading, error with retry, empty, content)
`app-status-chip`	`StatusChip`
`MoneyPipe`	`formatMoney()` in `shared/format.ts`
`mat-table`, `matSort`, `mat-paginator`	semantic `<table>`, `SortHeader`, `Pager` (asc, desc, none; server-side paging)
`mat-select`, `mat-form-field`	native `<select>`, `Field` (label, hint and error linked for assistive technology)
`forkJoin`	`Promise.all` inside `useAsync`
`interval()` with `takeUntilDestroyed`	`setInterval` cleaned up in the effect (dashboard refresh every 15 s, skipped while the tab is hidden)
route `title`	`useTitle()`
`withComponentInputBinding` (query parameters to inputs)	`useSearchParams()`

Design notes

- **`useAsync(loader, deps)`** runs the loader when `deps` change and ignores any result that arrives after the inputs changed or the component unmounted, so a slow earlier response can never overwrite a newer one. It keeps the previous data on screen during a reload and exposes `error`, `errorStatus` and `reload`.
- **No state library.** Server state is page-local and short-lived; the only shared state is the session, held by one context provider. React Query would add caching and deduplication, which this console does not need yet.
- **Double submit.** Disabling the button is not enough, because `setBusy(true)` is asynchronous: two clicks in the same tick both see `busy === false`. The form also guards with a ref (`inFlight`) and relies on the server-side idempotency key, exactly like the Angular form.
- **Native form controls.** The React edition uses native `<select>` and inputs styled with plain CSS. They are accessible by default, need no component library and keep the dependency tree small (3 runtime dependencies).
- **Code splitting.** Each page is a `lazy()` route chunk behind one `Suspense` boundary; the shell and the sign-in page stay in the main bundle.
- **`noValidate` on every form.** Validation messages come from the app (linked with `aria-describedby`), not from the browser's native bubbles, so both editions show identical messages.

Running it

```
cd frontend-react
npm ci
npm run dev              # http://localhost:4281, /api proxied to http://localhost:18480
npm test                 # Vitest + Testing Library
npm run lint && npm run build
```

Against the production bundle (static files plus the same `/api` proxy the Angular edition uses):

```
node scripts/serve-frontend.mjs --port 4281 --dir frontend-react/dist
node tests/e2e/ui-e2e.mjs --flavor react
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

CORS and same-origin proxying (a finding from the first browser run)

The first browser run of the React edition on port 4281 was answered with **403 Forbidden** on the sign-in call. Cause: the development proxy (`scripts/serve-frontend.mjs`) rewrote the `Host` header to the API's own host, so the API saw `Origin: http://localhost:4281` against `Host: localhost:18480` : a cross-origin request from an origin that is not on the allow-list (`CORS_ALLOWED_ORIGINS`, default `http://localhost:4280`, no wildcard).

Options, safest first:

1. **Preserve the `Host` header in the proxy (chosen).** The request is then same-origin for the API, exactly as behind the nginx image (`proxy_set_header Host $host`). No security setting changes. Applied to `scripts/serve-frontend.mjs` and to the Vite dev proxy (`changeOrigin: false`).
2. Add the origin explicitly: `CORS_ALLOWED_ORIGINS="http://localhost:4280, http://localhost:4281"` . Safe, but widens the allow-list for something that is not really cross-origin.
3. A wildcard origin. Rejected: it removes a security control (threat T13).

After the fix both editions sign in and pass the browser suite with the **default, unchanged** CORS configuration. Behind nginx (Docker Compose, Kubernetes) the same-origin proxy already works for either image, so no CORS entry is needed there either.

Verification (executed on the final code)

Check	Result
TypeScript (`tsc --noEmit`, strict)	clean
ESLint (typescript-eslint, react-hooks, jsx-a11y)	clean
Unit and component tests	90 of 90 passed
Production build	OK, 279 kB main chunk (89 kB gzip)
Browser end-to-end against the real backend (headless Edge)	19 of 19 passed (`docs/validation/ui-e2e-results-react.json`)
Same suite against the Angular edition, same backend, same run	19 of 19 passed
API end-to-end (backend only)	50 of 50

Not done for the React edition: Docker image build and Compose run (Docker is not installed here), cross-browser testing (Edge only), automated visual regression and accessibility audit beyond static lint rules.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Java and Spring Boot backend architecture

Technology choices (pinned)

Concern	Choice	Notes
Language/runtime	Java 21 (LTS)	records for DTOs and results, sealed interface for `ClearingResult`, pattern-matching `switch`, text blocks
Framework	Spring Boot 3.5.16	Spring MVC (servlet), Spring Security 6, Spring Data JPA/Hibernate 6, Bean Validation, Actuator
Persistence	PostgreSQL (prod), H2 PostgreSQL-mode (local/tests), Flyway 11	`ddl-auto: validate`
Resilience	hand-written `RetryExecutor` (backoff + full jitter), Resilience4j `CircuitBreaker`	one small library instead of the full starter: fewer moving parts
API docs	springdoc-openapi 2.8	contract committed and tested
Observability	Micrometer + Prometheus registry, Micrometer Tracing (OpenTelemetry bridge), ECS structured logs	
Tests	JUnit 5, Mockito, AssertJ, Spring Boot Test, Testcontainers, embedded PostgreSQL (test scope)	
Build	Maven 3.9, surefire (`*Test`) + failsafe (`*IT`), JaCoCo	

Spring Cloud is **not** used: with one deployable there is no config server, discovery or gateway to justify it (ADR-001). Redis, Kafka and RabbitMQ are **not** used either - the in-database outbox gives reliable asynchronous processing with one fewer system to run, and ADR-006 records when a broker (and a Redis-backed rate limiter) would be introduced. Fewer dependencies is a design decision here.

Package structure (modular monolith)

```
com.fintechdemo.platform
  common/        ApiException, ApiError, GlobalExceptionHandler, MoneyPolicy, filters (correlation id,
size limit), PageableSupport, Masking
  security/      SecurityConfig, AccessPolicy (pure), AuthenticatedUser, RateLimitFilter, local issuer
(profile-gated)
  customer/      Customer (entity), CustomerService, CustomerController, DTOs
  account/       Account (entity with debit/credit invariants), AccountService, controller,
AccountClosureGuard (port)
  transaction/   FinancialTransaction + TransactionStatus (state machine), TransactionValidator
(pure), DailyLimitPolicy (pure),
                 TransactionService (submit/review/cancel/query), LedgerService, TransactionProcessor,
OutboxService/Dispatcher,
                 ReconciliationService, TransactionMaintenance, ProcessingScheduler, repositories
  clearing/      ClearingClient (port + sealed result), RestClearingClient, InProcessClearingClient,
RetryExecutor, ClearingConfig
  audit/         AuditEvent (immutable), AuditService
  ops/           OpsService (read models), OpsController, PlatformMetrics
  config/        AppConfig (Clock, TransactionTemplates, OpenAPI definition)
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Dependency direction: `web -> services -> repositories`; `transaction -> account -> customer`; `account` reaches `transaction` only through the `AccountClosureGuard` port (dependency inversion), so there is no package cycle. Controllers translate and delegate; entities carry invariants (`Account.debit` refuses to go below the minimum balance; `FinancialTransaction.transitionTo` refuses illegal moves) but are never serialised.

Patterns used (and why)

Pattern	Where	Why
State machine	`TransactionStatus`, `AccountStatus`	legal moves in one place, tested exhaustively
Port and adapter	`ClearingClient` -> `RestClearingClient` / `InProcessClearingClient` / test double	the domain does not know HTTP; swap for tests and local runs
Strategy/policy objects (pure functions)	`TransactionValidator`, `AccessPolicy`, `DailyLimitPolicy`, `MoneyPolicy`	rules without I/O are trivially unit-testable
Transactional outbox	`OutboxService` / `OutboxDispatcher`	atomic "state change + announcement" without distributed transactions
Idempotent receiver	`TransactionProcessor`, idempotency keys	at-least-once delivery without duplicate effects
Compensation	`LedgerService.reverseWithdrawal`	undo reserved funds when an external step fails
Template method via `TransactionTemplate`	`TransactionProcessor`, `TransactionService.submit`	explicit, visible transaction boundaries (and no `@Transactional` self-invocation pitfalls)
Specification	`CustomerService/AccountService/ TransactionService.search`, audit search	composable, parameterised filters (no string-built SQL)
Facade / DTO	controllers + `*Dtos`	stable public contract independent of persistence
Dependency inversion	`AccountClosureGuard`	keeps module dependencies one-directional
Circuit breaker / retry with jitter	`RestClearingClient`	failure isolation

SOLID in practice: single-responsibility (the `LedgerService` is the *only* code that mutates balances; the validator only decides; the processor only orchestrates), open/closed (a new transaction type touches the validator, ledger and processor switch - and the compiler's exhaustive `switch` flags every place), Liskov (all `ClearingClient` implementations honour the three-way result contract), interface segregation (small ports), dependency inversion (above).

Transaction boundaries in code

- `@Transactional(propagation = MANDATORY)` on `LedgerService`, `AuditService.record`, `OutboxService.enqueue`: these *cannot* be called without an enclosing transaction - a missing boundary fails loudly in tests instead of silently committing a half-change.
- `AuditService.recordIndependent` (`REQUIRES_NEW`) for access denials, because the request is about to throw and roll back.
- Programmatic `TransactionTemplate`s (`transactionTemplate`, `readOnlyTransactionTemplate`) for the processor and submit, where the boundary must be *around some statements but not others* (the external call).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- A detail that bit during development and is documented in the code: both templates are injected **by parameter name**; marking one `@Primary` would win over name matching and silently hand the read-write template to the read-only injection points.
- Another real one: entities use a **wrapper `Long` version** because Spring Data decides "new vs. existing" from a null version; a primitive (or a getter that hides null) made `save()` merge a brand-new entity, which Hibernate reported as an optimistic-locking failure.

Configuration

`application.yml` holds safe defaults and every tunable as an environment variable (`DB_*`, `CLEARING_URL`, `RATE_LIMIT_*`, `OIDC_ISSUER_URI`, `MANAGEMENT_SERVER_PORT`, `TRACING_SAMPLING`, ...). Profiles: `local` (seed data, local issuer, public API docs), `postgres` (PostgreSQL driver/dialect), and none for production-style runs (OIDC issuer required: the application **fails to start** without any `JwtDecoder`, i.e. it fails closed). `@ConfigurationProperties` records (`ClearingProperties`, `ProcessingProperties`, `TransactionProperties`, `SecurityProperties`) make settings typed and testable.

Graceful shutdown

`server.shutdown=graceful`, `spring.lifecycle.timeout-per-shutdown-phase=20s`, scheduler `await-termination`: on `SIGTERM` the server stops accepting new requests, finishes in-flight work and background batches, then exits. The processor is crash-safe anyway (outbox redelivery + recovery), so a hard kill loses no money - graceful shutdown just avoids needless retries.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Database ER diagram

Choice: PostgreSQL (ADR-002)

PostgreSQL is the system of record. Reasons: strong ACID semantics with MVCC (readers never block writers), mature row-level locking (`FOR UPDATE`, `SKIP LOCKED`), partial and expression indexes, `CHECK` constraints, transactional DDL (migrations are atomic), excellent tooling (`EXPLAIN ANALYZE`, `pg_stat_statements`), open source and available as a managed service on every cloud (RDS/Aurora PostgreSQL). MySQL/InnoDB would also work; PostgreSQL's partial indexes and transactional DDL were decisive. An in-memory **H2 in PostgreSQL mode** is used for zero-install local runs and fast tests; the PostgreSQL-specific migration and behaviour are verified on a real PostgreSQL server (embedded PostgreSQL 14 in the test suite, `postgres:16.4` in Docker Compose and in the optional Testcontainers test).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Schema

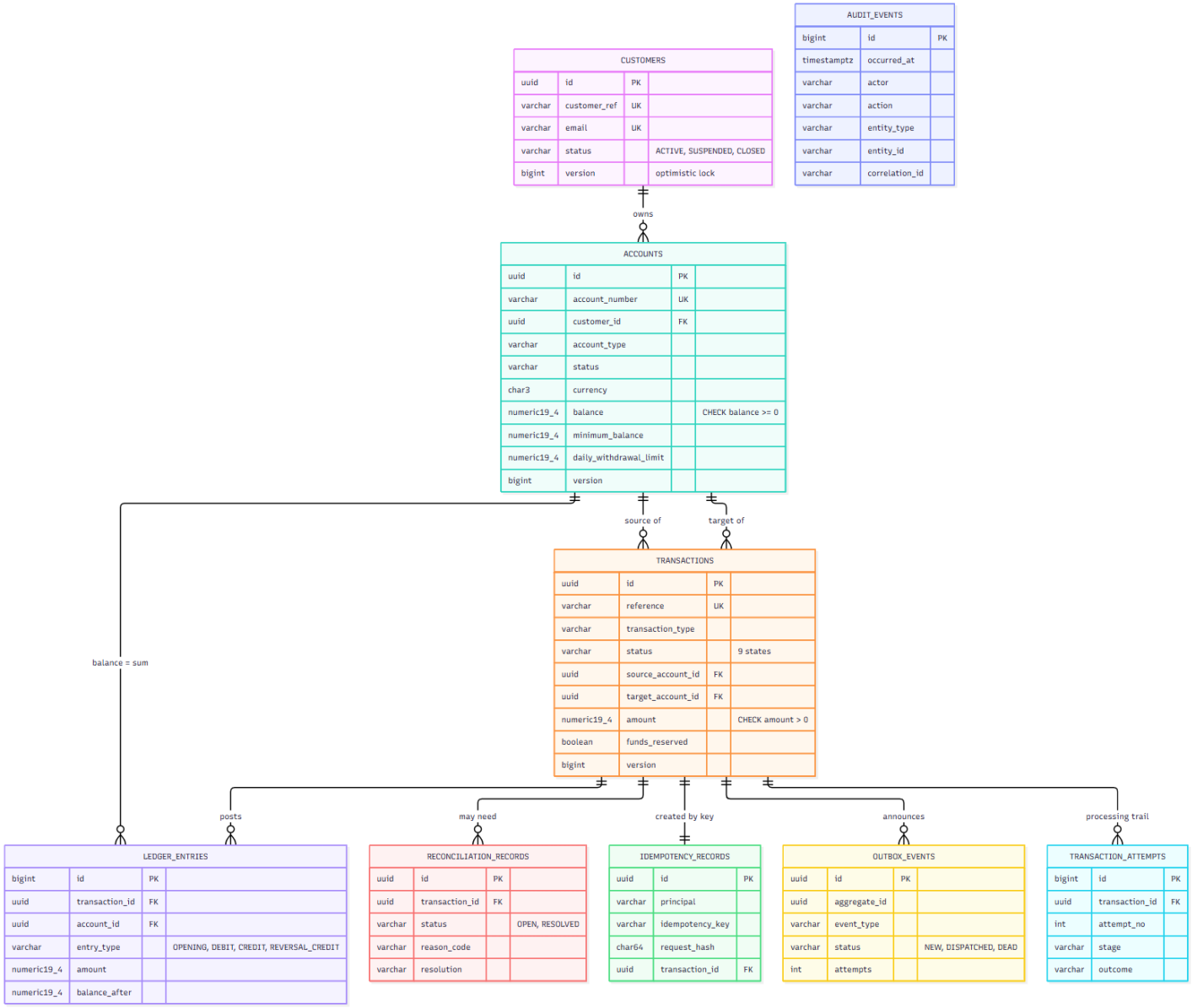


Figure 4. Entity-relationship diagram

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Normalised to third normal form; the denormalisations are deliberate and cheap: `balance` on `accounts` (a materialised sum of `ledger_entries`, kept honest by `/ops/ledger-integrity`), `balance_after` on ledger entries (point-in-time statement without a window query), `funds_reserved` on transactions (so reversal is idempotent).

Table	Purpose	Key constraints
`customers`	profile	`UNIQUE(customer_ref)`, `UNIQUE(email)`, status `CHECK`
`accounts`	balances	`UNIQUE(account_number)`, FK customer, **`CHECK (balance >= 0)`**, minimum/limit non-negative, `@Version`
`transactions`	requests and lifecycle	`UNIQUE(reference)`, FKs to both accounts, **`CHECK (amount > 0)`**, **`CHECK` that source/target match the type** (deposit: target only; withdrawal: source only; transfer: both and different), status `CHECK`
`transaction_attempts`	processing trail	`UNIQUE(transaction_id, attempt_no)`
`idempotency_records`	duplicate protection	**`UNIQUE(principal, idempotency_key)`**, FK transaction
`ledger_entries`	immutable postings	**`UNIQUE(transaction_id, account_id, entry_type)`** (blocks double posting), `amount >= 0`
`reconciliation_records`	uncertain outcomes	status `CHECK`; PostgreSQL: **partial unique index** `WHERE status = 'OPEN'` (one open record per transaction)
`outbox_events`	reliable async hand-off	status `CHECK` (`NEW`, `DISPATCHED`, `DEAD`)
`audit_events`	audit trail	indexes on `(entity_type, entity_id, id)`, `(occurred_at)`, `(action, occurred_at)`; PostgreSQL: **trigger forbids UPDATE/DELETE** (append-only)

Money columns are `NUMERIC(19,4)`. UUID primary keys are generated by the application (random v4). Trade-off: random UUIDs scatter inserts across the B-tree (worse locality than a sequence or UUIDv7); a time-ordered UUID (v7) is the documented upgrade if insert rate ever makes index bloat measurable.

Flyway migrations and safe schema evolution

```
db/migration/common/V1__core_schema.sql         portable schema (PostgreSQL + H2)
db/migration/postgresql/V2__postgresql_hardening.sql   partial indexes, append-only trigger
(PostgreSQL only)
db/migration/h2/V2__h2_equivalents.sql          plain-index equivalents for H2
db/seed/V1000__synthetic_seed_data.sql          synthetic demo data (local profile / tests only)
```

`spring.flyway.locations: classpath:db/migration/common, classpath:db/migration/{vendor}` lets each database get its own V2. The application runs with `ddl-auto: validate`, so Hibernate only checks the mapping against the migrated schema. (A real defect was caught this way: a `VARCHAR(20)` column too short for `CONFIRMED_NOT_EXECUTED`, found by the integration tests before any release; V1 was edited because nothing was deployed - after the first deployment **migrations are never edited**.)

Expand/contract for zero-downtime changes: (1) *expand* - add the new nullable column/table/index (`CREATE INDEX CONCURRENTLY` on PostgreSQL), deploy code that writes both; (2) *backfill* in small batches; (3) deploy code that reads the new shape; (4) *contract* - drop the old column in a later release, after the rollback window. Never rename or drop in the same release as the code change. Large-table index builds and `NOT NULL` additions are done in steps to avoid long locks.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Indexes and why

Index	Serves	Notes
`ix_tx_source_created (source_account_id, created_at DESC)`, `ix_tx_target_created`	per-account history, daily-limit sum	newest-first order matches the default sort
`ix_tx_status_created (status, created_at DESC)`	status filters, dashboards	
`ix_tx_attention` *(PostgreSQL partial: `WHERE status IN ('PENDING_REVIEW', 'RECONCILIATION_REQUIRED', 'FAILED')`)*	operations queues	tiny index over a tiny fraction of rows
`uq_idempotency_principal_key`	duplicate detection **and** arbitration	the one index that is a correctness mechanism
`ix_outbox_status_available (status, available_at)`	dispatcher polling	
`ix_ledger_account (account_id, id)`	statement/ledger reads, integrity check	
`uq_recon_open_per_tx` *(partial unique)*	at most one OPEN record per transaction	H2 cannot express it; `ReconciliationService.open` guards in code

Query plans (real, captured on PostgreSQL)

The plans below are produced by `PostgresFeaturesIT.queryPlans` with `EXPLAIN (ANALYZE, COSTS OFF)` on a real PostgreSQL server loaded with 100,000 synthetic transactions over 20 accounts. They are regenerated on every full build (`backend/platform-service/target/query-plans.txt`) and the captured run is included in the PDF as an appendix.

Reading them: the *shape* matters (which index, sort or filter), not the timings (omitted on purpose: shared developer machine).

- **Account history for a busy account** - PostgreSQL walks `ix_tx_created` newest-first and filters; for an account holding 5 % of all rows it finds 20 matches after ~400 index entries, which is cheaper than two bitmap scans plus a sort.
- **Account history for a sparse account** - the planner switches to the per-account indexes (bitmap OR of `ix_tx_source_created`/`ix_tx_target_created`), because walking the global index would discard almost everything.
- **Honest caveat:** the `source = :id OR target = :id` predicate is the weak spot. It works because each side has its own index and PostgreSQL can combine them, but at very large scale the clean answer is to split it into two index-friendly queries (`UNION ALL ... ORDER BY created_at DESC LIMIT n`) or to add a `transaction_participants (account_id, created_at, transaction_id)` table, and to move deep pagination to keyset pagination.
- Idempotency lookup, outbox polling and the row lock all use their intended indexes (primary/unique/composite).

Connection pool, transactions, timeouts

HikariCP: `maximum-pool-size 20`, `minimum-idle 5`, `connection-timeout 3 s` (fail fast), `max-lifetime 25 min` (below typical network idle cut-offs), `leak-detection-threshold 30 s`. Sizing rule of thumb: pool size is bounded by what the **database** can run concurrently (cores x 2-4), *not* by the number of request threads; more connections beyond that only adds contention. Each instance's pool times the replica count must stay below PostgreSQL `max_connections` (or use PgBouncer). `open-in-view: false` so no connection is held while rendering responses. Row-lock waits are bounded to 5 s (

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

`jakarta.persistence.lock.timeout`). The external call is made with **no** connection held (see transaction boundaries).

Useful SQL

```
-- the invariant that must always hold: balance equals the signed sum of ledger entries (empty result
= healthy)
SELECT a.id, a.balance, COALESCE(SUM(CASE WHEN e.entry_type = 'DEBIT' THEN -e.amount ELSE e.amount
END), 0) AS ledger_sum
FROM accounts a LEFT JOIN ledger_entries e ON e.account_id = a.id
GROUP BY a.id, a.balance
HAVING a.balance <> COALESCE(SUM(CASE WHEN e.entry_type = 'DEBIT' THEN -e.amount ELSE e.amount END),
0);

-- the attention queue operations works from
SELECT reference, status, failure_code, created_at FROM transactions
WHERE status IN ('PENDING_REVIEW','RECONCILIATION_REQUIRED','FAILED') ORDER BY created_at DESC LIMIT
50;

-- who holds locks / who is blocked right now (investigating lock contention)
SELECT blocked.pid AS blocked_pid, blocking.pid AS blocking_pid, blocked.query AS blocked_query,
blocking.query AS blocking_query
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocking ON blocking.pid = ANY (pg_blocking_pids(blocked.pid))
WHERE blocked.wait_event_type = 'Lock';

-- slowest statements (needs the pg_stat_statements extension)
SELECT calls, round(mean_exec_time::numeric, 2) AS mean_ms, left(query, 100) FROM pg_stat_statements
ORDER BY mean_exec_time DESC LIMIT 10;
```

Data retention considerations

Data	Retention in this implementation	Production consideration
idempotency records	7 days (scheduled purge)	must outlive the longest client retry window
audit events	kept (append-only)	regulatory retention (often 5-7 years) -> partition by month, archive cold partitions to cheap storage, never delete from the hot table
transactions, ledger	kept	partition `transactions` by `created_at`; archive after the reporting window
outbox	`DISPATCHED` rows accumulate	delete or partition-drop rows older than N days
PII	masked in views and logs; taxpayer fragment never returned	encryption at rest (KMS), field-level encryption for identifiers, right-to-erasure design

Oracle in an enterprise deployment (conceptual - Oracle is not run here)

Many enterprises standardise on Oracle. The design ports with these differences (no second database was added because there is no implementation need):

Topic	PostgreSQL (this repo)	Oracle
Identity / sequences	UUID in app; `GENERATED ALWAYS AS IDENTITY`	`GENERATED ALWAYS AS IDENTITY` (12c+) or sequences; `RAW(16)` for UUIDs
Pagination	`LIMIT n OFFSET m`	`OFFSET m ROWS FETCH NEXT n ROWS ONLY` (12c+)
Locking read	`SELECT ... FOR UPDATE [NOWAIT / SKIP LOCKED]`	`SELECT ... FOR UPDATE [NOWAIT / WAIT n / SKIP LOCKED]`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Topic	PostgreSQL (this repo)	Oracle
Default isolation	READ COMMITTED (statement snapshot)	READ COMMITTED (statement-level read consistency via undo) - same anomalies, `ORA-08177` on SERIALIZABLE conflicts
Empty string	distinct from NULL	`'' IS NULL` - validation and uniqueness logic must account for it
Boolean	`BOOLEAN`	no SQL `BOOLEAN` before 23ai: `NUMBER(1)` + check
Timestamps	`TIMESTAMPTZ`	`TIMESTAMP WITH TIME ZONE`
Partial indexes	yes	no - function-based indexes (e.g. index on `CASE WHEN status='OPEN' THEN transaction_id END`)
DDL in transactions	transactional	DDL auto-commits - migrations are not atomic
Hints / plans	planner statistics, `EXPLAIN`	CBO, `EXPLAIN PLAN`, AWR/ASH reports, SQL plan baselines
JPA dialect	`PostgreSQLDialect`	`OracleDialect`; `NUMBER(19,4)` for money, `VARCHAR2`

In Spring Boot the port is: add the Oracle JDBC driver, set the dialect, provide an `oracle` Flyway vendor folder next to `postgresql`/`h2`, replace partial indexes with function-based ones, and re-run the same concurrency tests.

MongoDB: where it would (and would not) fit

Not used for the ledger - financial invariants want relational constraints and multi-row ACID. Good fits around the edges: a **read model** of denormalised account activity for fast customer-facing history, **semi-structured documents** (KYC document metadata, notification payloads), and an **event/audit sink** with flexible schema and TTL indexes. Rule: one system of record per fact; Mongo copies are projections rebuilt from events, never the source of truth.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Customer and account lifecycle

Domain model in one paragraph

A **Customer** owns one or more **Accounts**. Money moves through **Transactions** (deposit, withdrawal, internal transfer). A transaction posts **LedgerEntries** (immutable debit/credit lines); an account's stored balance must always equal the signed sum of its ledger entries. Everything that matters is recorded as an **AuditEvent**. Anything that has to be delivered later is written as an **OutboxEvent**, and anything whose external outcome is uncertain gets a **ReconciliationRecord**.

Customer and account lifecycle

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

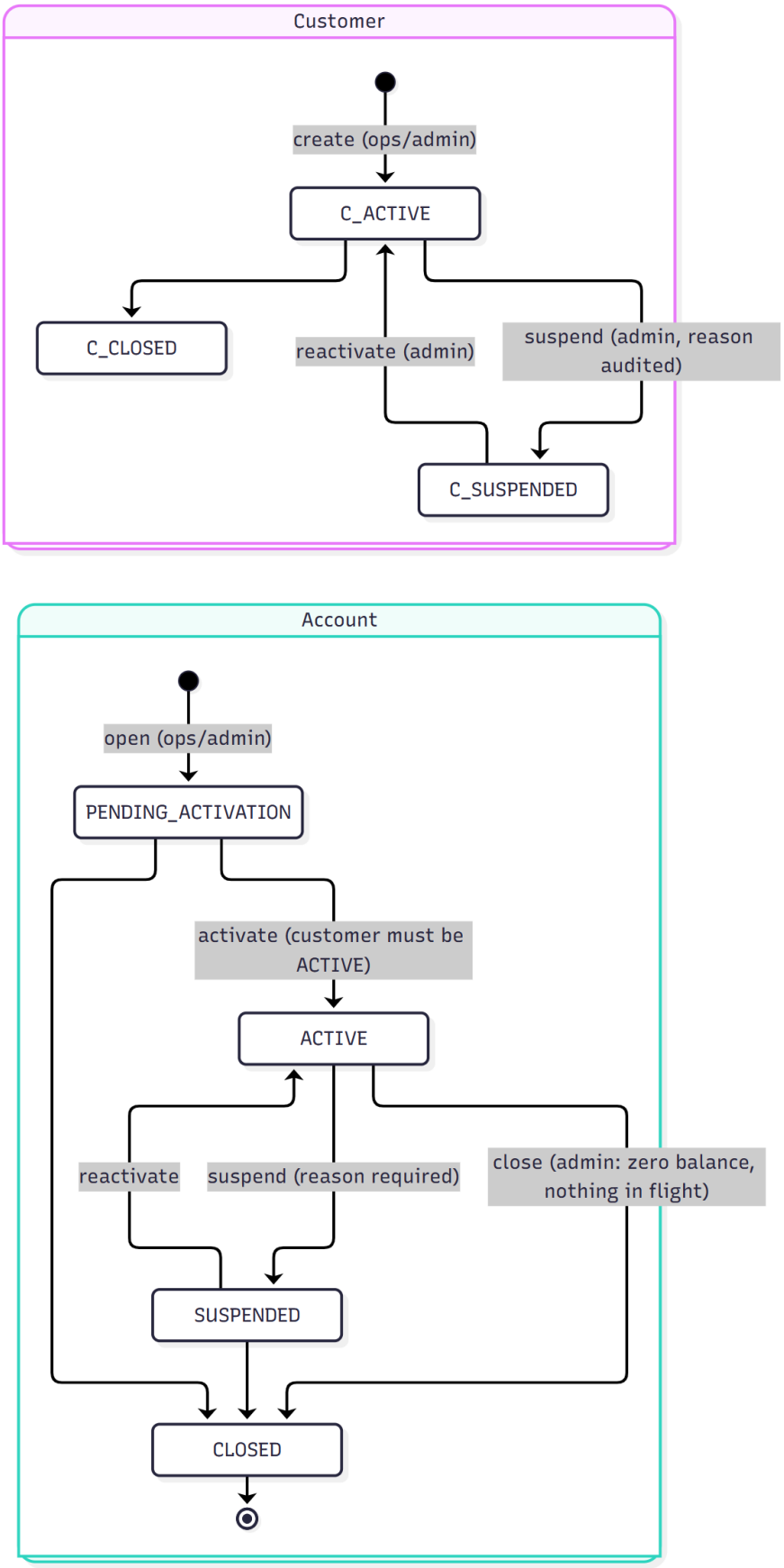


Figure 5. Customer and account lifecycle

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Transition	Who	Rules enforced in code
customer created -> `ACTIVE`	operations, admin	e-mail unique (case-insensitive) -> `409 DUPLICATE_EMAIL`; `CUS-xxxxxxxx` reference generated; taxpayer fragment stored but never returned
customer `ACTIVE -> SUSPENDED`	admin	reason required and audited; suspended customers cannot transact (`CUSTOMER_NOT_ACTIVE`)
account opened -> `PENDING_ACTIVATION`	operations, admin	customer must be `ACTIVE`; currency must be supported; default minimum balance / daily limit come from `AccountType`
`PENDING_ACTIVATION/SUSPENDED -> ACTIVE`	operations, admin	owning customer must be `ACTIVE`
`ACTIVE -> SUSPENDED`	operations, admin	reason required (audited)
`* -> CLOSED`	admin	balance must be exactly zero **and** no transaction in flight; the row is locked while checking so nobody can credit it in between
any other move	-	`409 INVALID_STATE_TRANSITION` from `AccountStatus.canTransitionTo`

Account access restrictions are layered: role (who may call the endpoint) -> object ownership (`AccessPolicy`) -> account status (validation and again under the row lock) -> customer status.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Transaction lifecycle and state machine

Transaction lifecycle and state machine

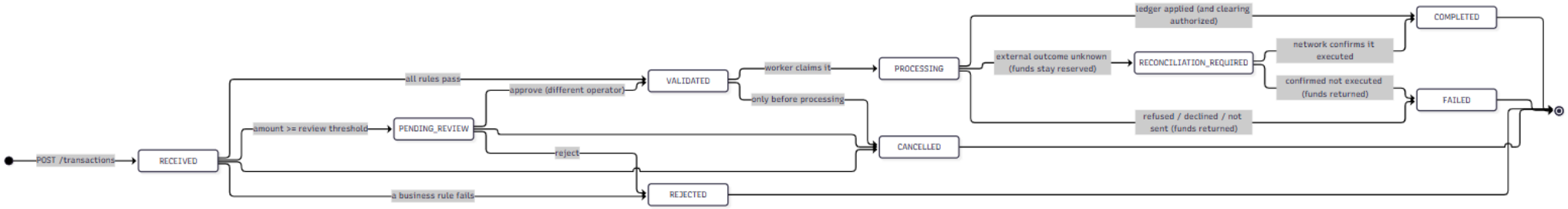


Figure 6. Transaction state machine

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

`TransactionStatus` is the only definition of which moves are legal; `FinancialTransaction.transitionTo` refuses every `INVALID_STATE_TRANSITION`. `TransactionStatusTest` re-states the whole 9 x 9 matrix independently and compares eve changing the spec on purpose.

State	Meaning	W
`RECEIVED`	persisted, not yet validated (transient: moves inside the submit transaction)	T
`VALIDATED`	all rules passed; an outbox event announces it	d
`PENDING_REVIEW`	amount >= 50,000: waits for an independent approver	a
`PROCESSING`	claimed by the worker; funds reserved (withdrawal)	p
`COMPLETED`	terminal success; ledger posted	p
`REJECTED`	terminal; a business rule failed at validation (or a reviewer rejected it)	s
`FAILED`	terminal; failed during processing (insufficient funds at lock time, decline, network unavailable); reserved funds returned	p
`RECONCILIATION_REQUIRED`	the external outcome is unknown; **funds stay reserved**	p
`CANCELLED`	terminal; only valid before processing starts	c

Validation rules (every rule has a code)

Rule	Code(s)
Amount > 0, at most 15.4 digits, at most the currency's minor units (**never silently rounded**), at most 1,000,000	`AMOUNT_NOT_POSITIVE`, `AMOUNT_PRECISION`, `AM`
Currency supported and equal to the account currency	`UNSUPPORTED_CURRENCY`, `CURRENCY_MISMATCH`
Source/target account `ACTIVE`; owning customer `ACTIVE`	`SOURCE_ACCOUNT_NOT_ACTIVE`, `TARGET_ACCOUN`
Transfer endpoints differ	`SAME_ACCOUNT_TRANSFER`
Available balance (balance - minimum balance) covers the amount	`INSUFFICIENT_FUNDS`
Rolling 24 h withdrawals + amount <= daily limit (feature built test-first, see the TDD chapter)	`DAILY_LIMIT_EXCEEDED`
Review threshold	`REVIEW_REQUIRED` (outcome `PENDING_REVIEW`
Ownership / authorization	`404 ACCOUNT_NOT_FOUND` (object-level), `403`
Duplicate submission	idempotency key replay / `422 IDEMPOTENCY`

Validation at submit is advisory; the authoritative check is repeated under the row lock when the balanc Between the two, other transactions may have spent the money, so the second check is the one that protects the in concurrent-withdrawal scenario demonstrates (20 requests pass validation against a balance that only covers 10).

Money rules

- Java `BigDecimal` everywhere; the API serialises money as **strings** (`"1250.00"`) so JavaScript never touches it as a string manipulation (`MoneyPipe`).
- Storage scale 4, display scale = the currency's minor units (2); input with more fractional digits than the currency never rounded.
- Where rounding is unavoidable (percentages, allocation) the policy is `HALF_EVEN` and allocation never loses or crea
- Comparisons use `compareTo`, never `equals` (scale-sensitive).
- The database refuses `balance < 0`, `amount <= 0` and type/account mismatches even if application code is wrong.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Account transfer sequence diagram

Account transfer: end-to-end sequence

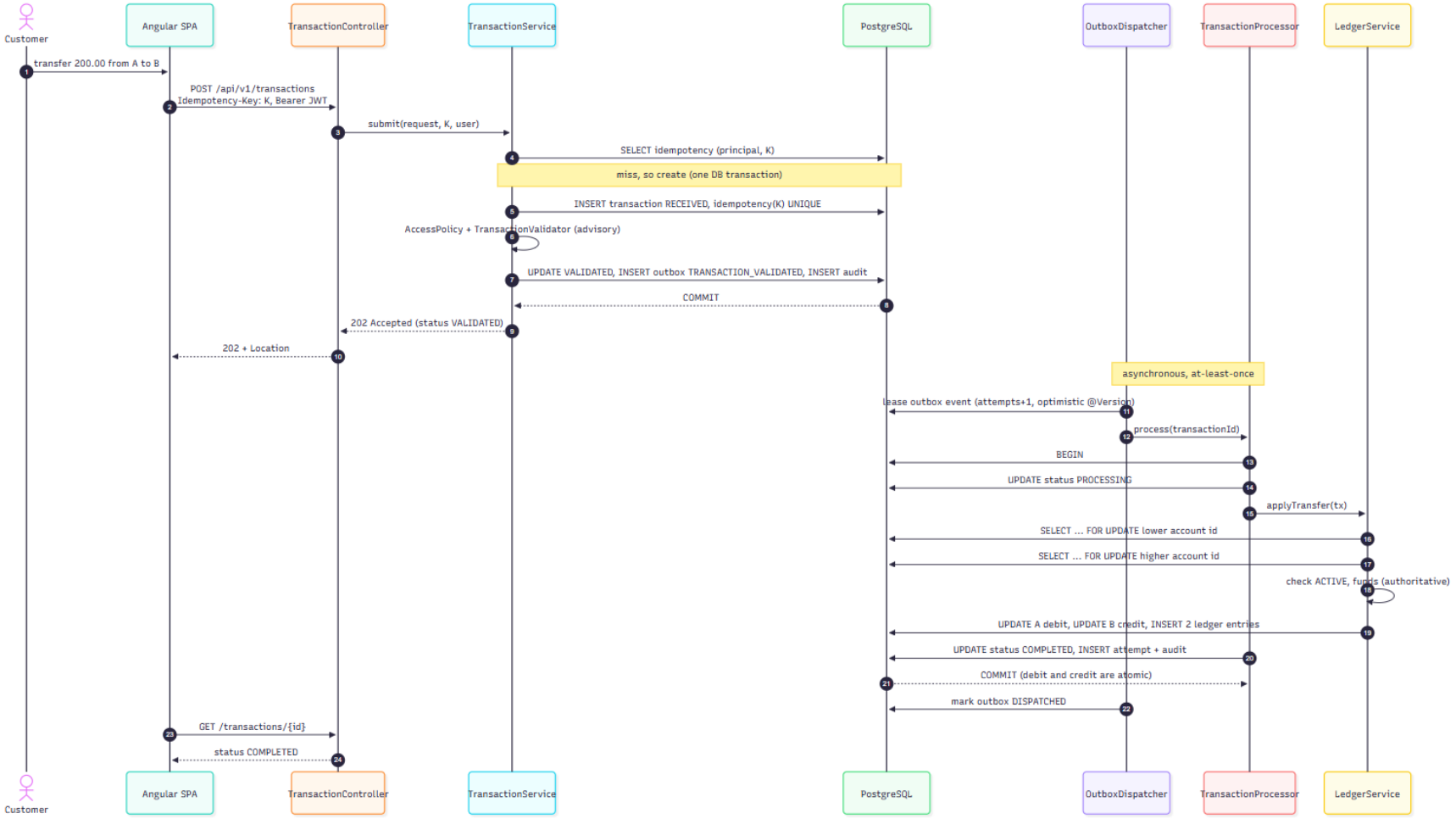


Figure 7. Account transfer sequence

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

1. The SPA generates (or reuses) an `Idempotency-Key` and posts the transfer.
2. In **one database transaction** the service looks up the key, inserts the transaction as `RECEIVED` and the idempot
   the transaction to `VALIDATED`, writes the outbox event and the audit rows, and commits. The client receives `202 A`
3. After commit, the dispatcher leases the outbox event and calls the processor.
4. The processor opens **one** database transaction, moves `VALIDATED -> PROCESSING`, locks the two account rows **in a**
   funds, debits and credits, writes two ledger entries, moves to `COMPLETED`, records the attempt and the audit event
   same commit: it is impossible to observe (or crash into) a half-applied transfer.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Idempotency and concurrency-control flow

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Idempotency and concurrency control

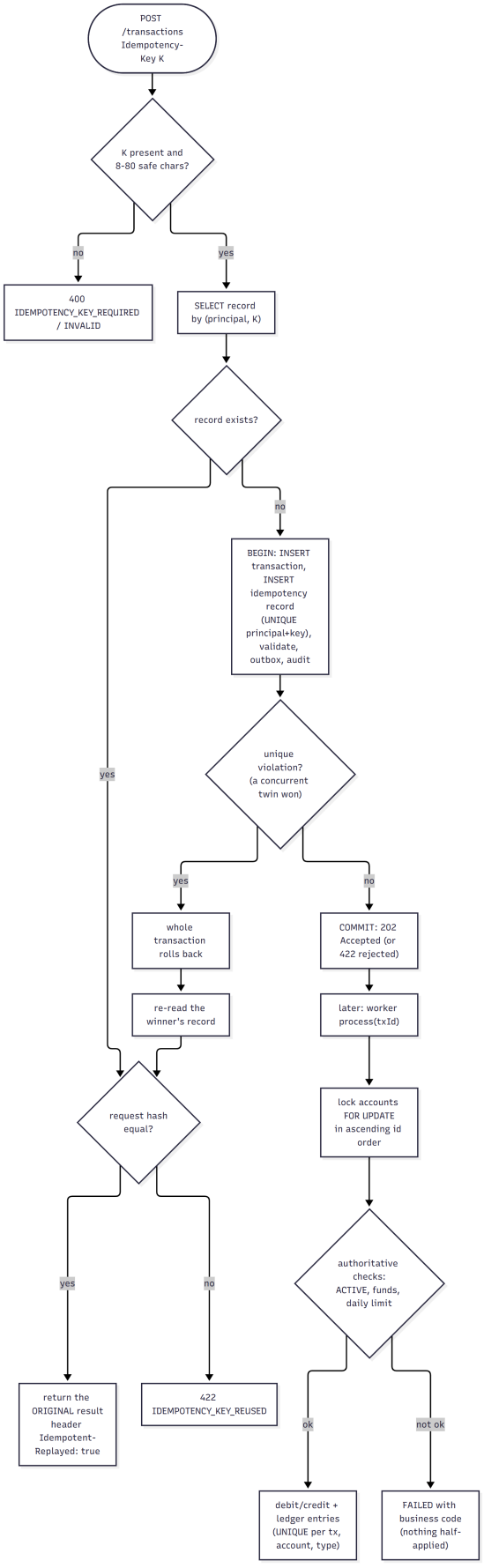


Figure 8. Idempotency and concurrency flow

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Idempotency

- The key is scoped per principal: `UNIQUE (principal, idempotency_key)`. The **unique constraint is the arbiter**: the application does a fast read first, but correctness never depends on that read.
- When two requests with the same key race, both try to insert; the loser's *whole* transaction (transaction row, idempotency row, outbox row, audit rows) rolls back on the unique violation, then it re-reads the winner's record and replays it. Result: exactly one transaction, every caller gets the same answer (`IdempotencyIT.concurrentSameKeyCreatesOneTransaction`, 12 threads).
- The record stores a SHA-256 fingerprint of the canonical request. Same key + different payload -> `422 IDEMPOTENCY_KEY_REUSED` instead of silently returning stale data for a different request.
- Replays return the original status (`202`, or `422` for a rejection) plus `Idempotent-Replayed: true`.
- Retention: records are purged after 7 days (`TransactionMaintenance`); the retention period is the window in which a client may safely retry.

Why the processor is idempotent too

The outbox guarantees *at-least-once* delivery. A second delivery finds the transaction no longer `VALIDATED` and returns without effect; the `UNIQUE (transaction_id, account_id, entry_type)` constraint on ledger entries is a second, database-level guard against applying the same posting twice.

Locking strategy

Mechanism	Used for	Why
Pessimistic row lock (`SELECT ... FOR UPDATE`, 5 s bounded wait) on `accounts`	every balance change	an account is a hot, contended row; a read-modify-write under optimistic locking would retry-storm under contention and could starve; with the lock the second request simply waits milliseconds and then sees the new balance
Optimistic `@Version` on customers, accounts, transactions, outbox, reconciliation	everything not under the row lock: cancel vs process, double approval, double resolution, concurrent profile edits	cheap, no lock held, conflicts are rare and the loser can re-read -> `409 CONCURRENT_MODIFICATION`
Unique constraints	idempotency, ledger postings, references, account numbers	the database is the only component that can decide a race between two application instances
Lease + `@Version` on the outbox	multiple dispatchers	claim by bumping `attempts` and pushing `available_at`; two claimers collide on `@Version`, only one wins; a crashed claimer's lease simply expires
Lock ordering (ascending account id)	transfers	a transfer A->B and B->A would otherwise each hold one row and wait for the other: the textbook deadlock. A total order makes the lock-wait graph acyclic (`LedgerServiceTest.locksInAscendingIdOrder`, `ConcurrencyScenarios.oppositeTransfersDoNotDeadlock`)

Optimistic versus pessimistic - how to choose. Optimistic locking wins when conflicts are rare and retries are cheap (profile edits, status changes). Pessimistic wins when the same row is *expected* to be contended and the critical section is short (balances). The cost of pessimistic locking is lock waits and the *risk of deadlock*, which is handled by ordering and by a bounded wait (a timeout becomes a retryable `503`, never a pile-up that exhausts the connection pool).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Isolation levels and the race conditions they allow

The platform runs on the database default (`READ COMMITTED` on PostgreSQL) and gets its guarantees from row locks and constraints rather than from raising the isolation level. For reference:

Isolation	Prevents	Still allows	Relevance here
READ COMMITTED (PostgreSQL default)	dirty reads	non-repeatable reads, phantoms, **lost updates / write skew**	acceptable *because* every balance change is a locked read-modify-write
REPEATABLE READ (snapshot in PostgreSQL)	non-repeatable reads, phantoms	write skew; transactions can fail with serialization errors	would turn contention into retries; not needed with row locks
SERIALIZABLE (SSI in PostgreSQL)	all anomalies	none, at the price of aborts and retry logic	the "check balance, then debit" race is the textbook case SERIALIZABLE protects - we protect it explicitly with `FOR UPDATE`

Race conditions the design closes, with the evidence that closes them:

Race	Closed by	Test
two withdrawals overspend one balance	row lock + authoritative re-check	`concurrentWithdrawalsNeverOverspend` (20 requests, balance for 10 -> exactly 10 complete)
daily limit exceeded by parallel withdrawals	limit evaluated **under the account lock**	`concurrentWithdrawalsRespectDailyLimit`
duplicate submission	unique idempotency key	`concurrentSameKeyCreatesOneTransaction`
lost update on deposits	locked read-modify-write	`concurrentDepositsAreNotLost` (30 parallel, none lost)
deadlock on opposing transfers	ordered locks	`oppositeTransfersDoNotDeadlock`
cancel vs process	`@Version` on the transaction	`TransactionFlowIT.cancellation`
double delivery of an event	status check + unique ledger entry	`duplicateDeliveryIsHarmless`

Deadlocks can still arise if some future code path locks accounts in a different order. Mitigations: all balance changes go through `LedgerService` (one place), lock waits are bounded (5 s), and PostgreSQL's own deadlock detector aborts one victim (surfacing as a retryable error). **Recovery strategy** for the failure modes:

Failure	What happens	Recovery
process crashes after commit of submit	outbox row exists	next dispatcher tick processes it
crash during processing tx	transaction rolled back by the DB	outbox lease expires -> redelivery
crash after funds reserved, before outcome (withdrawal)	`PROCESSING` + reserved funds	recovery job moves it to `RECONCILIATION_REQUIRED` after 5 min; reconciler asks the network
poison event	repeated failures	`DEAD` after 5 attempts, alert, admin requeue
DB outage	requests fail fast (3 s connection timeout)	readiness probe fails, traffic drains; on return the outbox continues

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Authentication and authorization

Authentication and authorization flow

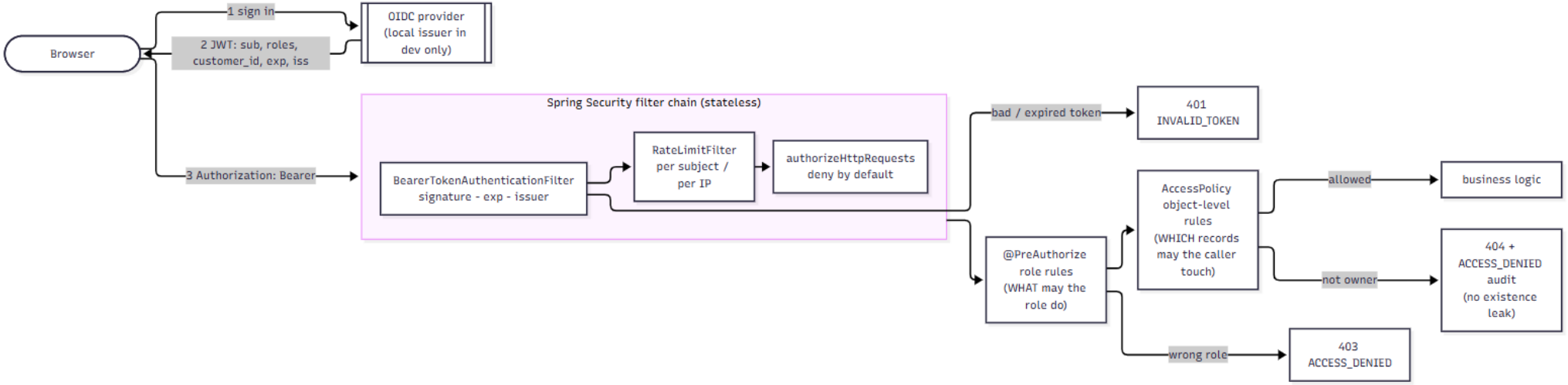


Figure 9. Authentication and authorization

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Authentication (OAuth 2.0 / OIDC compatible)

`platform-service` is an **OAuth 2.0 resource server**. It never sees passwords: clients present `Authorization: Beare` **signature, expiry and issuer**. Two interchangeable configurations:

Mode	Used for	How it validates
OIDC (production-style)	any shared environment	`spring.security.oauth2.resourceserver.jwt.issuer-uri` (`OI` and JWKS are fetched and cached; asymmetric signatures
Local issuer	laptop demo and tests only	`LocalIssuerConfig` + `LocalAuthController`: an HS256 key (r is supplied) signs tokens for seven synthetic users; the app with a `prod`/`production` profile; absent from the Kubernetes

Claims used: `sub` (principal; also the audit actor and idempotency scope), `roles` (array), `customer_id` (links a `CUSTOME` names are **ignored** (least privilege) - a token granting `SUPER_ADMIN` grants nothing (`SecurityIT.unknownRoleGrantsNo`

Token handling in the SPA: the access token is kept **in memory only** (never `localStorage`/`sessionStorage`), so XSS c the tab; trade-off: a reload requires signing in again - production would use authorization-code + PKCE (ideally via a cookie) for silent renewal. The interceptor attaches the token only to `/api/` URLs.

Authorization - two layers

1. **Role-level** (`@PreAuthorize`, `SecurityConfig`): *what may this role do?* e.g. only `ADMIN` can close an account or drain
2. **Object-level** (`AccessPolicy`, pure functions): *which records may this caller touch?* A `CUSTOMER` may only see accou `customer_id` claim; a transfer must stay inside one customer; staff may look but only operators may change; **mal** submission regardless of role. Denials return `404` (existence is not revealed) and write an `ACCESS_DENIED` audit eve survives the rollback of the failed request).

The authorization matrix is exhaustively unit-tested (`AccessPolicyTest`) and exercised end to end (`SecurityIT`: ~18 t balances, history, ledger, transaction read/cancel/approve, search scoping, submission against others' accounts, orp

Deny by default and hardening

Control	Implementation
Deny by default	`authorizeHttpRequests`: only listed paths are permitted; `anyRequest().denyAll()`
Stateless, no session	`SessionCreationPolicy.STATELESS`; CSRF disabled **deliberately**: tokens travel in the `Authoriza` automatically - CSRF is a cookie problem; the code comment says to re-enable it if cookies ar
CORS	explicit origin allow-list (`CORS_ALLOWED_ORIGINS`), no wildcard, no credentials, listed headers; pr
Security headers	`X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, HSTS, CSP, `Ca`
Request limits	64 KB body limit incl. chunked; sort whitelist; page size cap 100
Rate limiting	token bucket per subject (API) and per address (login)
Input validation	Bean Validation on DTOs, allow-list regex for memo, `@Digits`, strict JSON (unknown fields reje
SQL injection	JPA/JPQL with bound parameters and the Criteria API only; search strings are data (`SecurityIT`
Error handling	one envelope; no stack traces, SQL or payload echo; correlation id for support
Sensitive data	emails/phones masked for support agents; tax-id fragment never returned; account numbers not secrets but are never logged with identities beyond ids
Audit	every state change and every access denial; append-only trigger on PostgreSQL
Secrets	none committed; compose requires them from `.env` (no defaults for secret values); Kubernete leaks (gitleaks)
Dependencies	OWASP Dependency-Check (CVSS >= 7 fails), `npm audit`, Trivy image scan, Dependabot-read

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Control	Implementation
Containers	non-root, read-only filesystem, all capabilities dropped, `no-new-privileges`, restricted pod sec

OWASP mapping

The OWASP Top 10 (2021) and OWASP API Security Top 10 (2023) mapping to concrete controls and tests is in `docs/`
as part of the threat-model chapter).

Educational implementation versus a production financial system

This repository demonstrates the *engineering* of a secure design with synthetic data. It is **not** security-reviewed for
still needs: an independent security assessment and penetration test; formal threat modelling with the organisation'
(HSM/KMS), field-level encryption and tokenisation of identifiers; mTLS or a service mesh between services; WAF and
alerting on audit events; privileged-access management and break-glass procedures; a distributed rate limiter; SCA/
data-residency and retention policy; compliance validation (e.g. SOC 2, PCI DSS if card data were ever in scope, GDP

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

REST API examples

The contract is machine-readable in `specs/openapi.json` (generated from the running application by springdoc and checked into the repository; `ApiContractIT` fails the build when code and contract drift apart). Real request/response pairs captured from the running system are in `docs/api/examples.md`.

Resource model and endpoints

Resource	Endpoints (all under /api/v1)	Roles
Identity	`GET /me`; `POST /auth/token` (**local profile only**)	any authenticated; anonymous for login
Customers	`POST /customers`, `GET /customers`, `GET /customers/{id}`, `PATCH /customers/{id}`, `POST /customers/{id}/suspend`, `POST /customers/{id}/reactivate`	create/update: operations, admin; list: staff; get: staff or the customer themself; suspend: admin
Accounts	`POST /customers/{customerId}/accounts`, `GET /accounts`, `GET /accounts/{id}`, `GET /accounts/{id}/balance`, `POST /accounts/{id}/activate`, `.../suspend`, `.../close`	search/get/balance: owner or staff; lifecycle: operations/admin (close: admin)
Transactions	`POST /transactions`, `GET /transactions`, `GET /transactions/{id}`, `GET /accounts/{id}/transactions`, `POST /transactions/{id}/cancel`, `.../approve`, `.../reject`, `GET /transactions/{id}/attempts`, `GET /transactions/{id}/audit`	submit: customer (own accounts), operations, admin; approve/reject: a *different* operator or admin; attempts/audit: staff
Operations	`GET /ops/summary`, `/ops/health`, `/ops/audit-events`, `/ops/reconciliations`, `POST /ops/reconciliations/{id}/resolve`, `GET /ops/ledger-integrity`, `GET /ops/accounts/{id}/ledger`, `GET /ops/outbox`, `POST /ops/outbox/{id}/requeue`, `POST /ops/processing/drain`	staff; resolve: operations/admin; outbox/drain: admin
Platform	`/actuator/health/{liveness, readiness}`, `/actuator/prometheus` (management port in containers)	probes / Prometheus

Design rules applied: resource-oriented nouns (`/accounts/{id}/transactions`), verbs only for state-changing *commands* that are not CRUD (`/approve`, `/cancel`, `/activate`), plural collection names, UUID identifiers (not guessable sequential ids), DTOs everywhere (entities are never serialised), unknown JSON properties rejected (mass-assignment defence).

HTTP semantics and status codes

Situation	Status	code
created resource (`POST` customer/account)	`201` + `Location`	-
transaction accepted for asynchronous processing	`202` + `Location`	-
business rejection of a *valid* request (transaction persisted as `REJECTED`)	`422`	`TRANSACTION_REJECTED` (+ `details.failureCode`)
malformed JSON / missing header / type mismatch / bean-validation failure	`400`	`MALFORMED_REQUEST`, `IDEMPOTENCY_KEY_REQUIRED`, `INVALID_PARAMETER`, `VALIDATION_ERROR` (+ `violations[]`)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Situation	Status	code
unauthenticated / bad or expired token	`401` + `WWW-Authenticate: Bearer`	`UNAUTHENTICATED`, `INVALID_TOKEN`
authenticated but wrong role / maker-checker violation	`403`	`ACCESS_DENIED`, `MAKER_CHECKER_VIOLATION`, `TRANSACTION_NOT_PERMITTED`
not found **or not yours** (object-level authorization)	`404`	`ACCOUNT_NOT_FOUND`, `TRANSACTION_NOT_FOUND`, ...
state conflict, duplicate natural key, concurrent modification	`409`	`INVALID_STATE_TRANSITION`, `DUPLICATE_EMAIL`, `CONCURRENT_MODIFICATION`
semantic error on an otherwise valid request (idempotency key reuse, unsupported combination)	`422`	`IDEMPOTENCY_KEY_REUSED`, ...
body too large	`413`	`PAYLOAD_TOO_LARGE`
unsupported method / media type	`405` / `415`	`METHOD_NOT_ALLOWED`, `UNSUPPORTED_MEDIA_TYPE`
rate limited	`429` + `Retry-After`	`RATE_LIMITED`
lock wait timed out (retryable)	`503` + `Retry-After: 1`	`LOCK_TIMEOUT`
unexpected failure	`500`	`INTERNAL_ERROR` (generic message + correlation id only)

Why 404 and not 403 for someone else's object? A `403` confirms the record exists. The platform answers `404` for objects the caller may not see (and writes an `ACCESS_DENIED` audit event) so ids cannot be enumerated (OWASP API1). Role failures are `403` because the endpoint's existence is not secret.

The error envelope

Every non-2xx response has the same shape, produced by `GlobalExceptionHandler` (and by `ErrorResponses` for filter-level errors):

```
{
  "timestamp": "2026-10-18T12:00:00Z",
  "status": 400,
  "code": "VALIDATION_ERROR",
  "message": "Request validation failed",
  "path": "/api/v1/transactions",
  "correlationId": "7f3c1e52-...",
  "violations": [ { "field": "amount", "message": "must be greater than zero" } ],
  "details": { }
}
```

`message` never contains stack traces, SQL, class names or the offending payload (`ServerErrorIT` throws an exception carrying a fake JDBC URL and password and asserts none of it reaches the client).

Pagination, filtering, sorting

`?page=0&size=20&sort=createdAt, desc` on every collection. `size` is capped at 100 (`spring.data.web.pageable.max-page-size`); `sort` accepts a **whitelist** per resource (`PageableSupport.restrictSort`) and a deterministic tie-break on `id` is appended, so paging is stable and clients cannot sort by internal columns or trigger unindexed sorts. Responses use a stable envelope (`items, page, size, totalElements, totalPages`) instead of leaking Spring Data's `Page`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

JSON. Transaction search filters: `status` (repeatable), `type`, `accountId`, `reference`, `from`, `to`, `minAmount`, `maxAmount`, `currency`. For very deep histories the documented evolution is keyset pagination (`created_at, id`) - offset paging is O(offset).

Idempotency, correlation, limits

- `POST /transactions` **requires** `Idempotency-Key` (8-80 characters of `[A-Za-z0-9._:-]`). See the integrity chapter.
- `X-Correlation-Id` is accepted if it is a safe token (`[A-Za-z0-9._-]{8, 64}`, which blocks log injection) or generated, put in the logging MDC, echoed on the response and stored in audit rows. `X-API-Version: 1` is returned on every response.
- Request bodies larger than 64 KB are rejected before parsing (`RequestSizeLimitFilter`, counts bytes for chunked bodies too).
- Rate limiting: token bucket per authenticated subject (240/min default) and, much stricter, per client address for login (10/min). In-memory per instance; the production substitute is a shared store or the API gateway (threat model).

Versioning and backward compatibility

- **URI versioning** (`/api/v1`): simple, cache- and gateway-friendly, visible in logs. Alternatives (header/media-type versioning) are more "pure" but harder to test by hand and to route.
- **Compatibility rules** (a change is *non-breaking* if clients written against v1 keep working): *allowed in v1:* add optional request fields, add response fields, add endpoints, add new enum values **only where clients are told to tolerate unknown values**, add new error codes; *breaking (needs v2):* remove or rename a field, change a type or the meaning/scale of a value, make an optional field required, tighten validation that rejects previously valid input, change status codes of existing flows.
- **Tolerant reader:** the SPA ignores unknown fields; the API itself is *strict* on input (`FAIL_ON_UNKNOWN_PROPERTIES`) because it is the system of record - the asymmetry is deliberate (be liberal in what you accept only when ambiguity cannot cost money).
- **Contract tests:** `ApiContractIT` compares the committed `specs/openapi.json` with the live document (operations and schemas); CI also fails on `git diff specs/openapi.json` after the build, so an unreviewed API change cannot merge.
- **Deprecation process:** announce in the changelog, add `Deprecation`/`Sunset` response headers and an OpenAPI `deprecated: true`, run both versions side by side, monitor usage per client from metrics, then remove.

Example requests and responses

Success, validation failure, authentication failure, authorization failure, conflict, business rejection and server error are all shown with real captured output in the "REST API examples" chapter (generated into `docs/api/examples.md`).

Captured examples (real output of the running application)

Generated by `scripts/capture-api-examples.mjs` against the local stack - every response below is real output; JWTs and correlation ids are redacted. Base URL `http://localhost:18480`. Money amounts are strings. See `specs/openapi.json` for the full contract.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Customer management: create a customer (operations/admin)

Customers are created by staff. `taxIdLast4` is accepted but never returned (`****`).

```
POST /api/v1/customers
Content-Type: application/json
Authorization: Bearer <jwt>

{
  "legalName": "Frida Example",
  "email": "frida-1792335216662@example.test",
  "phone": "+1-555-0199",
  "taxIdLast4": "9999"
}
```

```
HTTP/1.1 201
location: /api/v1/customers/0d4f1be8-4b2c-45c7-b338-5564f8939040
x-correlation-id: <generated or echoed>

{
  "id": "0d4f1be8-4b2c-45c7-b338-5564f8939040",
  "customerRef": "CUS-591713A5",
  "legalName": "Frida Example",
  "email": "frida-1792335216662@example.test",
  "phone": "+1-555-0199",
  "taxIdLast4": "****",
  "status": "ACTIVE",
  "version": 0,
  "createdAt": "2026-10-18T14:53:36.781345800Z",
  "updatedAt": "2026-10-18T14:53:36.781345800Z"
}
```

Account management: open an account

New accounts start as `PENDING_ACTIVATION`; money amounts are JSON **strings**.

```
POST /api/v1/customers/0d4f1be8-4b2c-45c7-b338-5564f8939040/accounts
Content-Type: application/json
Authorization: Bearer <jwt>

{
  "accountType": "BROKERAGE",
  "currency": "USD",
  "dailyWithdrawalLimit": "5000.00"
}
```

```
HTTP/1.1 201
location: /api/v1/accounts/adde6851-a9c6-4092-80ab-502c4608b19b
x-correlation-id: <generated or echoed>

{
  "id": "adde6851-a9c6-4092-80ab-502c4608b19b",
  "accountNumber": "ACC-1688427101",
  "customerId": "0d4f1be8-4b2c-45c7-b338-5564f8939040",
  "accountType": "BROKERAGE",
  "status": "PENDING_ACTIVATION",
  "currency": "USD",
  "balance": "0.00",
  "availableBalance": "0.00",
  "minimumBalance": "0.00",
  "dailyWithdrawalLimit": "5000.00",
  "version": 0,
  "openedAt": "2026-10-18T14:53:36.916622200Z",
  "updatedAt": "2026-10-18T14:53:36.916622200Z"
}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Account lifecycle: activate

```
POST /api/v1/accounts/adde6851-a9c6-4092-80ab-502c4608b19b/activate
Content-Type: application/json
Authorization: Bearer <jwt>

{}
```

```
HTTP/1.1 200
x-correlation-id: <generated or echoed>

{
  "id": "adde6851-a9c6-4092-80ab-502c4608b19b",
  "accountNumber": "ACC-1688427101",
  "customerId": "0d4f1be8-4b2c-45c7-b338-5564f8939040",
  "accountType": "BROKERAGE",
  "status": "ACTIVE",
  "currency": "USD",
  "balance": "0.00",
  "availableBalance": "0.00",
  "minimumBalance": "0.00",
  "dailyWithdrawalLimit": "5000.00",
  "version": 1,
  "openedAt": "2026-10-18T14:53:36.916622Z",
  "updatedAt": "2026-10-18T14:53:36.981921900Z"
}
```

Transaction submission: deposit (accepted for asynchronous processing)

`202 Accepted` + `Location`. The `Idempotency-Key` header is mandatory.

```
POST /api/v1/transactions
Content-Type: application/json
Idempotency-Key: docs-c1e824a7-52e9-4ce1-8c57-d19ebb17cf54
Authorization: Bearer <jwt>

{
  "type": "DEPOSIT",
  "targetAccountId": "adde6851-a9c6-4092-80ab-502c4608b19b",
  "amount": "1500.00",
  "currency": "USD",
  "memo": "Opening funding"
}
```

```
HTTP/1.1 202
location: /api/v1/transactions/cf614759-44ec-4b31-9dd4-4b37be565472
x-correlation-id: <generated or echoed>

{
  "id": "cf614759-44ec-4b31-9dd4-4b37be565472",
  "reference": "TXN-C0904B0222FD",
  "type": "DEPOSIT",
  "status": "VALIDATED",
  "targetAccountId": "adde6851-a9c6-4092-80ab-502c4608b19b",
  "amount": "1500.00",
  "currency": "USD",
  "memo": "Opening funding",
  "submittedBy": "adam.admin",
  "version": 1,
  "createdAt": "2026-10-18T14:53:37.074041300Z",
  "updatedAt": "2026-10-18T14:53:37.074041300Z"
}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Transaction status

Poll the resource (or watch the dashboard) - processing is asynchronous.

```
GET /api/v1/transactions/cf614759-44ec-4b31-9dd4-4b37be565472
Content-Type: application/json
Authorization: Bearer <jwt>
```

```
HTTP/1.1 200
x-correlation-id: <generated or echoed>

{
  "id": "cf614759-44ec-4b31-9dd4-4b37be565472",
  "reference": "TXN-C0904B0222FD",
  "type": "DEPOSIT",
  "status": "COMPLETED",
  "targetAccountId": "adde6851-a9c6-4092-80ab-502c4608b19b",
  "amount": "1500.00",
  "currency": "USD",
  "memo": "Opening funding",
  "submittedBy": "adam.admin",
  "version": 2,
  "createdAt": "2026-10-18T14:53:37.074041Z",
  "updatedAt": "2026-10-18T14:53:37.415339Z",
  "completedAt": "2026-10-18T14:53:37.415339Z"
}
```

Account balance query

```
GET /api/v1/accounts/adde6851-a9c6-4092-80ab-502c4608b19b/balance
Content-Type: application/json
Authorization: Bearer <jwt>
```

```
HTTP/1.1 200
x-correlation-id: <generated or echoed>

{
  "accountId": "adde6851-a9c6-4092-80ab-502c4608b19b",
  "currency": "USD",
  "ledgerBalance": "1500.00",
  "minimumBalance": "0.00",
  "availableBalance": "1500.00",
  "asOf": "2026-10-18T14:53:38.675366600Z"
}
```

Transaction history (paginated, sorted)

Page size is capped at 100; only whitelisted sort properties are accepted.

```
GET /api/v1/accounts/adde6851-a9c6-4092-80ab-502c4608b19b/transactions?
page=0&size=5&sort=createdAt,desc
Content-Type: application/json
Authorization: Bearer <jwt>
```

```
HTTP/1.1 200
x-correlation-id: <generated or echoed>

{
  "items": [
    {
      "id": "cf614759-44ec-4b31-9dd4-4b37be565472",
      "reference": "TXN-C0904B0222FD",
      "type": "DEPOSIT",
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
      "status": "COMPLETED",
      "targetAccountId": "adde6851-a9c6-4092-80ab-502c4608b19b",
      "amount": "1500.00",
      "currency": "USD",
      "memo": "Opening funding",
      "submittedBy": "adam.admin",
      "version": 2,
      "createdAt": "2026-10-18T14:53:37.074041Z",
      "updatedAt": "2026-10-18T14:53:37.415339Z",
      "completedAt": "2026-10-18T14:53:37.415339Z"
    }
  ],
  "page": 0,
  "size": 5,
  "totalElements": 1,
  "totalPages": 1
}
```

Transaction search with filters

```
GET /api/v1/transactions?status=COMPLETED&type=DEPOSIT&minAmount=1000.00&size=2&sort=amount,desc
Content-Type: application/json
Authorization: Bearer <jwt>
```

```
HTTP/1.1 200
x-correlation-id: <generated or echoed>

{
  "items": [
    {
      "id": "cf614759-44ec-4b31-9dd4-4b37be565472",
      "reference": "TXN-C0904B0222FD",
      "type": "DEPOSIT",
      "status": "COMPLETED",
      "targetAccountId": "adde6851-a9c6-4092-80ab-502c4608b19b",
      "amount": "1500.00",
      "currency": "USD",
      "memo": "Opening funding",
      "submittedBy": "adam.admin",
      "version": 2,
      "createdAt": "2026-10-18T14:53:37.074041Z",
      "updatedAt": "2026-10-18T14:53:37.415339Z",
      "completedAt": "2026-10-18T14:53:37.415339Z"
    }
  ],
  "page": 0,
  "size": 2,
  "totalElements": 1,
  "totalPages": 1
}
```

Idempotency: first submission

```
POST /api/v1/transactions
Content-Type: application/json
Idempotency-Key: docs-058cf590-9d99-4153-835f-d9c38e9b0a8f
Authorization: Bearer <jwt>

{
  "type": "WITHDRAWAL",
  "sourceAccountId": "adde6851-a9c6-4092-80ab-502c4608b19b",
  "amount": "200.00",
  "currency": "USD"
}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
HTTP/1.1 202
location: /api/v1/transactions/5bce73dd-7609-479c-b93e-6ec9961cddee
x-correlation-id: <generated or echoed>

{
  "id": "5bce73dd-7609-479c-b93e-6ec9961cddee",
  "reference": "TXN-E93EC32A3837",
  "type": "WITHDRAWAL",
  "status": "VALIDATED",
  "sourceAccountId": "adde6851-a9c6-4092-80ab-502c4608b19b",
  "amount": "200.00",
  "currency": "USD",
  "submittedBy": "adam.admin",
  "version": 1,
  "createdAt": "2026-10-18T14:53:38.796576900Z",
  "updatedAt": "2026-10-18T14:53:38.796576900Z"
}
```

Idempotency: the same request again returns the original result

Note the `idempotent-replayed: true` header - no second withdrawal is created.

```
POST /api/v1/transactions
Content-Type: application/json
Idempotency-Key: docs-058cf590-9d99-4153-835f-d9c38e9b0a8f
Authorization: Bearer <jwt>

{
  "type": "WITHDRAWAL",
  "sourceAccountId": "adde6851-a9c6-4092-80ab-502c4608b19b",
  "amount": "200.00",
  "currency": "USD"
}
```

```
HTTP/1.1 202
location: /api/v1/transactions/5bce73dd-7609-479c-b93e-6ec9961cddee
idempotent-replayed: true
x-correlation-id: <generated or echoed>

{
  "id": "5bce73dd-7609-479c-b93e-6ec9961cddee",
  "reference": "TXN-E93EC32A3837",
  "type": "WITHDRAWAL",
  "status": "VALIDATED",
  "sourceAccountId": "adde6851-a9c6-4092-80ab-502c4608b19b",
  "amount": "200.00",
  "currency": "USD",
  "submittedBy": "adam.admin",
  "version": 1,
  "createdAt": "2026-10-18T14:53:38.796577Z",
  "updatedAt": "2026-10-18T14:53:38.796577Z"
}
```

Conflict: same key, different payload

```
POST /api/v1/transactions
Content-Type: application/json
Idempotency-Key: docs-058cf590-9d99-4153-835f-d9c38e9b0a8f
Authorization: Bearer <jwt>

{
  "type": "WITHDRAWAL",
  "sourceAccountId": "adde6851-a9c6-4092-80ab-502c4608b19b",
  "amount": "201.00",
  "currency": "USD"
}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
HTTP/1.1 422
x-correlation-id: <generated or echoed>

{
  "timestamp": "2026-10-18T14:53:38.888693800Z",
  "status": 422,
  "code": "IDEMPOTENCY_KEY_REUSED",
  "message": "This Idempotency-Key was already used with a different request payload",
  "path": "/api/v1/transactions",
  "correlationId": "7f3c1e52-0000-4000-8000-00000000c0de"
}
```

Business rejection: insufficient funds

A valid request that violates a business rule is persisted as `REJECTED` and answered with `422` + the transaction id.

```
POST /api/v1/transactions
Content-Type: application/json
Idempotency-Key: docs-15a203c7-3cb0-4395-8194-d6a8649061bc
Authorization: Bearer <jwt>

{
  "type": "WITHDRAWAL",
  "sourceAccountId": "adde6851-a9c6-4092-80ab-502c4608b19b",
  "amount": "999999.00",
  "currency": "USD"
}
```

```
HTTP/1.1 422
x-correlation-id: <generated or echoed>

{
  "timestamp": "2026-10-18T14:53:38.957937300Z",
  "status": 422,
  "code": "TRANSACTION_REJECTED",
  "message": "The transaction was rejected by business rules",
  "path": "/api/v1/transactions",
  "correlationId": "7f3c1e52-0000-4000-8000-00000000c0de",
  "details": {
    "transactionId": "1ee6b012-d5ba-47fd-92ef-ea8989373171",
    "failureCode": "INSUFFICIENT_FUNDS",
    "reference": "TXN-34AB990A9A59",
    "status": "REJECTED",
    "failureDetail": "INSUFFICIENT_FUNDS: Available balance is insufficient; DAILY_LIMIT_EXCEEDED: The
daily withdrawal limit for this account would be exceeded"
  }
}
```

Validation failure: malformed input

Every violation is listed with its field.

```
POST /api/v1/transactions
Content-Type: application/json
Idempotency-Key: docs-62358026-f739-4af0-85e1-9200bd43e3a6
Authorization: Bearer <jwt>

{
  "type": "DEPOSIT",
  "amount": "-5",
  "currency": "usd",
  "memo": "<script>"
}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
HTTP/1.1 400
x-correlation-id: <generated or echoed>

{
  "timestamp": "2026-10-18T14:53:38.993130100Z",
  "status": 400,
  "code": "VALIDATION_ERROR",
  "message": "Request validation failed",
  "path": "/api/v1/transactions",
  "correlationId": "7f3c1e52-0000-4000-8000-00000000c0de",
  "violations": [
    {
      "field": "amount",
      "message": "must be greater than zero"
    },
    {
      "field": "accountsConsistentWithType",
      "message": "accounts do not match the transaction type (DEPOSIT: target only, WITHDRAWAL: source
only, TRANSFER: both)"
    },
    {
      "field": "memo",
      "message": "contains unsupported characters"
    },
    {
      "field": "currency",
      "message": "must be a 3-letter ISO currency code"
    }
  ]
}
```

Validation failure: missing Idempotency-Key

```
POST /api/v1/transactions
Content-Type: application/json
Authorization: Bearer <jwt>

{
  "type": "DEPOSIT",
  "targetAccountId": "adde6851-a9c6-4092-80ab-502c4608b19b",
  "amount": "5.00",
  "currency": "USD"
}
```

```
HTTP/1.1 400
x-correlation-id: <generated or echoed>

{
  "timestamp": "2026-10-18T14:53:39.021006600Z",
  "status": 400,
  "code": "IDEMPOTENCY_KEY_REQUIRED",
  "message": "Required parameter or header is missing: Idempotency-Key",
  "path": "/api/v1/transactions",
  "correlationId": "7f3c1e52-0000-4000-8000-00000000c0de"
}
```

Authentication failure: no token

```
GET /api/v1/accounts
Content-Type: application/json
```

```
HTTP/1.1 401
x-correlation-id: <generated or echoed>
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
{
  "timestamp": "2026-10-18T14:53:39.039591Z",
  "status": 401,
  "code": "UNAUTHENTICATED",
  "message": "Authentication is required",
  "path": "/api/v1/accounts",
  "correlationId": "7f3c1e52-0000-4000-8000-00000000c0de"
}
```

Authentication failure: invalid token

```
GET /api/v1/accounts
Content-Type: application/json
Authorization: Bearer <jwt>
```

```
HTTP/1.1 401
x-correlation-id: <generated or echoed>

{
  "timestamp": "2026-10-18T14:53:39.053946Z",
  "status": 401,
  "code": "INVALID_TOKEN",
  "message": "The access token is missing, expired or invalid",
  "path": "/api/v1/accounts",
  "correlationId": "7f3c1e52-0000-4000-8000-00000000c0de"
}
```

Authorization failure: wrong role (403)

A customer may not create customers.

```
POST /api/v1/customers
Content-Type: application/json
Authorization: Bearer <jwt>

{
  "legalName": "X",
  "email": "x@example.test"
}
```

```
HTTP/1.1 403
x-correlation-id: <generated or echoed>

{
  "timestamp": "2026-10-18T14:53:39.069964700Z",
  "status": 403,
  "code": "ACCESS_DENIED",
  "message": "You do not have permission to perform this operation",
  "path": "/api/v1/customers",
  "correlationId": "7f3c1e52-0000-4000-8000-00000000c0de"
}
```

Authorization failure: object-level (404, no existence leak)

Bob's account requested by Alice. The attempt is written to the audit trail.

```
GET /api/v1/accounts/10000000-0000-4000-8000-000000000004
Content-Type: application/json
Authorization: Bearer <jwt>
```

```
HTTP/1.1 404
x-correlation-id: <generated or echoed>
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
{
  "timestamp": "2026-10-18T14:53:39.094751800Z",
  "status": 404,
  "code": "ACCOUNT_NOT_FOUND",
  "message": "Account not found",
  "path": "/api/v1/accounts/10000000-0000-4000-8000-000000000004",
  "correlationId": "7f3c1e52-0000-4000-8000-00000000c0de"
}
```

(setup) large withdrawal needing review

amounts >= 50,000 wait for an independent approver

```
POST /api/v1/transactions
Content-Type: application/json
Idempotency-Key: docs-283f8724-f07b-469b-af9d-4fba7bf6b7f7
Authorization: Bearer <jwt>

{
  "type": "DEPOSIT",
  "targetAccountId": "adde6851-a9c6-4092-80ab-502c4608b19b",
  "amount": "60000.00",
  "currency": "USD"
}
```

```
HTTP/1.1 202
location: /api/v1/transactions/046c3d44-66f2-43ba-a061-afc15d51ef43
x-correlation-id: <generated or echoed>

{
  "id": "046c3d44-66f2-43ba-a061-afc15d51ef43",
  "reference": "TXN-7099DD8A62FE",
  "type": "DEPOSIT",
  "status": "PENDING_REVIEW",
  "targetAccountId": "adde6851-a9c6-4092-80ab-502c4608b19b",
  "amount": "60000.00",
  "currency": "USD",
  "submittedBy": "omar.ops",
  "version": 1,
  "createdAt": "2026-10-18T14:53:39.114793600Z",
  "updatedAt": "2026-10-18T14:53:39.114793600Z"
}
```

Maker-checker: an operator cannot approve their own transaction

```
POST /api/v1/transactions/046c3d44-66f2-43ba-a061-afc15d51ef43/approve
Content-Type: application/json
Authorization: Bearer <jwt>

{}
```

```
HTTP/1.1 403
x-correlation-id: <generated or echoed>

{
  "timestamp": "2026-10-18T14:53:39.161124100Z",
  "status": 403,
  "code": "MAKER_CHECKER_VIOLATION",
  "message": "Approval requires a different operator than the one who submitted the transaction",
  "path": "/api/v1/transactions/046c3d44-66f2-43ba-a061-afc15d51ef43/approve",
  "correlationId": "7f3c1e52-0000-4000-8000-00000000c0de"
}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Conflict: duplicate e-mail

```
POST /api/v1/customers
Content-Type: application/json
Authorization: Bearer <jwt>

{
  "legalName": "Dup",
  "email": "alice@example.test"
}
```

```
HTTP/1.1 409
x-correlation-id: <generated or echoed>

{
  "timestamp": "2026-10-18T14:53:39.185130800Z",
  "status": 409,
  "code": "DUPLICATE_EMAIL",
  "message": "A customer with this e-mail already exists",
  "path": "/api/v1/customers",
  "correlationId": "7f3c1e52-0000-4000-8000-00000000c0de"
}
```

Invalid state transition

Cancelling a completed transaction.

```
POST /api/v1/transactions/cf614759-44ec-4b31-9dd4-4b37be565472/cancel
Content-Type: application/json
Authorization: Bearer <jwt>

{}
```

```
HTTP/1.1 409
x-correlation-id: <generated or echoed>

{
  "timestamp": "2026-10-18T14:53:39.205838200Z",
  "status": 409,
  "code": "INVALID_STATE_TRANSITION",
  "message": "A COMPLETED transaction can no longer be cancelled",
  "path": "/api/v1/transactions/cf614759-44ec-4b31-9dd4-4b37be565472/cancel",
  "correlationId": "7f3c1e52-0000-4000-8000-00000000c0de"
}
```

Unsupported sort property

```
GET /api/v1/transactions?sort=submittedBy,asc
Content-Type: application/json
Authorization: Bearer <jwt>
```

```
HTTP/1.1 400
x-correlation-id: <generated or echoed>

{
  "timestamp": "2026-10-18T14:53:39.230088500Z",
  "status": 400,
  "code": "INVALID_SORT",
  "message": "Unsupported sort property: submittedBy",
  "path": "/api/v1/transactions",
  "correlationId": "7f3c1e52-0000-4000-8000-00000000c0de",
  "details": {
    "allowed": [
      "amount",
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
      "completedAt",
      "createdAt",
      "reference",
      "status"
    ]
  }
}
```

Operations: dashboard summary

Staff only (support, operations, admin).

```
GET /api/v1/ops/summary
Content-Type: application/json
Authorization: Bearer <jwt>
```

```
HTTP/1.1 200
x-correlation-id: <generated or echoed>

{
  "generatedAt": "2026-10-18T14:53:39.252045600Z",
  "transactionsByStatus": {
    "RECEIVED": 0,
    "VALIDATED": 0,
    "PROCESSING": 1,
    "COMPLETED": 1,
    "REJECTED": 1,
    "FAILED": 0,
    "PENDING_REVIEW": 1,
    "RECONCILIATION_REQUIRED": 0,
    "CANCELLED": 0
  },
  "completedVolumeLast24h": [
    {
      "type": "DEPOSIT",
      "currency": "USD",
      "count": 1,
      "volume": "1500.00"
    }
  ],
  "completedLast24h": 1,
  "failedLast24h": 0,
  "pendingReview": 1,
  "reconciliationRequired": 0,
  "openReconciliationRecords": 0,
  "transactionFailureRateLast24h": 0,
  "processingLatency": {
    "sampleSize": 1,
    "averageMs": 341,
    "p50Ms": 341,
    "p95Ms": 341,
    "p99Ms": 341
  },
  "http": {
    "totalRequests": 30,
    "serverErrors": 0,
    "errorRate": 0
  },
  "accountsByStatus": {
    "ACTIVE": 9,
    "PENDING_ACTIVATION": 1,
    "SUSPENDED": 1
  },
  "totalBalances": [
    {
      "currency": "EUR",
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
      "total": "170000.00"
    },
    {
      "currency": "GBP",
      "total": "7500.00"
    },
    {
      "currency": "INR",
      "total": "500000.00"
    },
    {
      "currency": "USD",
      "total": "124200.00"
    }
  ]
}
```

Operations: platform health

```
GET /api/v1/ops/health
Content-Type: application/json
Authorization: Bearer <jwt>
```

```
HTTP/1.1 200
x-correlation-id: <generated or echoed>

{
  "status": "UP",
  "checkedAt": "2026-10-18T14:53:39.537684100Z",
  "components": [
    {
      "name": "database",
      "status": "UP",
      "detail": "round trip 8 ms"
    },
    {
      "name": "clearing-network",
      "status": "UP",
      "detail": "reachable=true, circuit=CLOSED"
    },
    {
      "name": "outbox",
      "status": "UP",
      "detail": "pending=1, dead-lettered=0"
    },
    {
      "name": "reconciliation",
      "status": "UP",
      "detail": "open records=0"
    }
  ]
}
```

Operations: audit events

```
GET /api/v1/ops/audit-events?action=TRANSACTION_COMPLETED&size=2
Content-Type: application/json
Authorization: Bearer <jwt>
```

```
HTTP/1.1 200
x-correlation-id: <generated or echoed>

{
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
  "items": [
    {
      "id": 14,
      "occurredAt": "2026-10-18T14:53:37.429305Z",
      "actor": "system:processor",
      "actorRoles": "",
      "action": "TRANSACTION_COMPLETED",
      "entityType": "TRANSACTION",
      "entityId": "cf614759-44ec-4b31-9dd4-4b37be565472"
    }
  ],
  "page": 0,
  "size": 2,
  "totalElements": 1,
  "totalPages": 1
}
```

Operations: ledger integrity check

Must always return an empty list.

```
GET /api/v1/ops/ledger-integrity
Content-Type: application/json
Authorization: Bearer <jwt>
```

```
HTTP/1.1 200
x-correlation-id: <generated or echoed>

[]
```

Administration: drain the background loops once (admin only)

```
POST /api/v1/ops/processing/drain
Content-Type: application/json
Authorization: Bearer <jwt>

{}
```

```
HTTP/1.1 200
x-correlation-id: <generated or echoed>

{
  "dispatched": 0,
  "reconciled": 0,
  "recoveredStuck": 0
}
```

Administration: dead-lettered outbox events

```
GET /api/v1/ops/outbox?status=DEAD
Content-Type: application/json
Authorization: Bearer <jwt>
```

```
HTTP/1.1 200
x-correlation-id: <generated or echoed>

{
  "items": [],
  "page": 0,
  "size": 20,
  "totalElements": 0,
  "totalPages": 0
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
}
```

Server error (500) - produced by ServerErrorIT through the real handler

A test-only controller throws `IllegalStateException("secret internal detail: jdbc:postgresql://... password=...")`; the client sees only this:

```
HTTP/1.1 500 Internal Server Error

{
  "timestamp" : "2026-10-18T14:39:27.165157700Z",
  "status" : 500,
  "code" : "INTERNAL_ERROR",
  "message" : "An unexpected error occurred. Quote the correlation id when contacting support.",
  "path" : "/api/v1/test-only/boom",
  "correlationId" : "08843174-13d2-41f5-bd74-1ff13f05a566"
}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Database transaction boundaries

Database transaction boundaries (processing)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

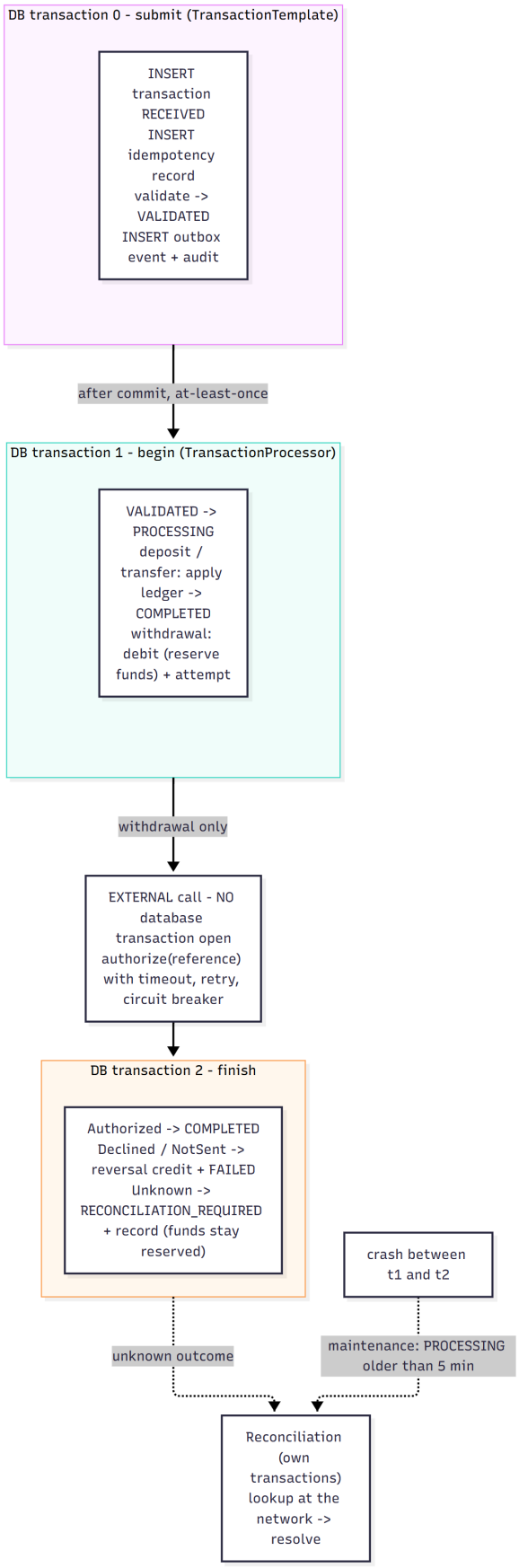


Figure 10. Transaction boundaries

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

The external call is deliberately **outside** any database transaction. Holding row locks and a pooled connection across a network call that may take seconds (or hang) is how connection pools are exhausted and how one slow dependency takes the whole system down. Instead the processor commits "funds reserved" first (so the money cannot be spent twice), calls the network with a hard timeout, and then commits the outcome in a second short transaction.

> A database transaction does **not** provide distributed exactly-once processing across external systems. What the design provides instead is explicit states, idempotent calls (the network deduplicates on our `reference`), at-least-once internal delivery with idempotent consumers, and reconciliation for the unknowable case.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Error-handling and recovery flows

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Error handling and recovery for the external call

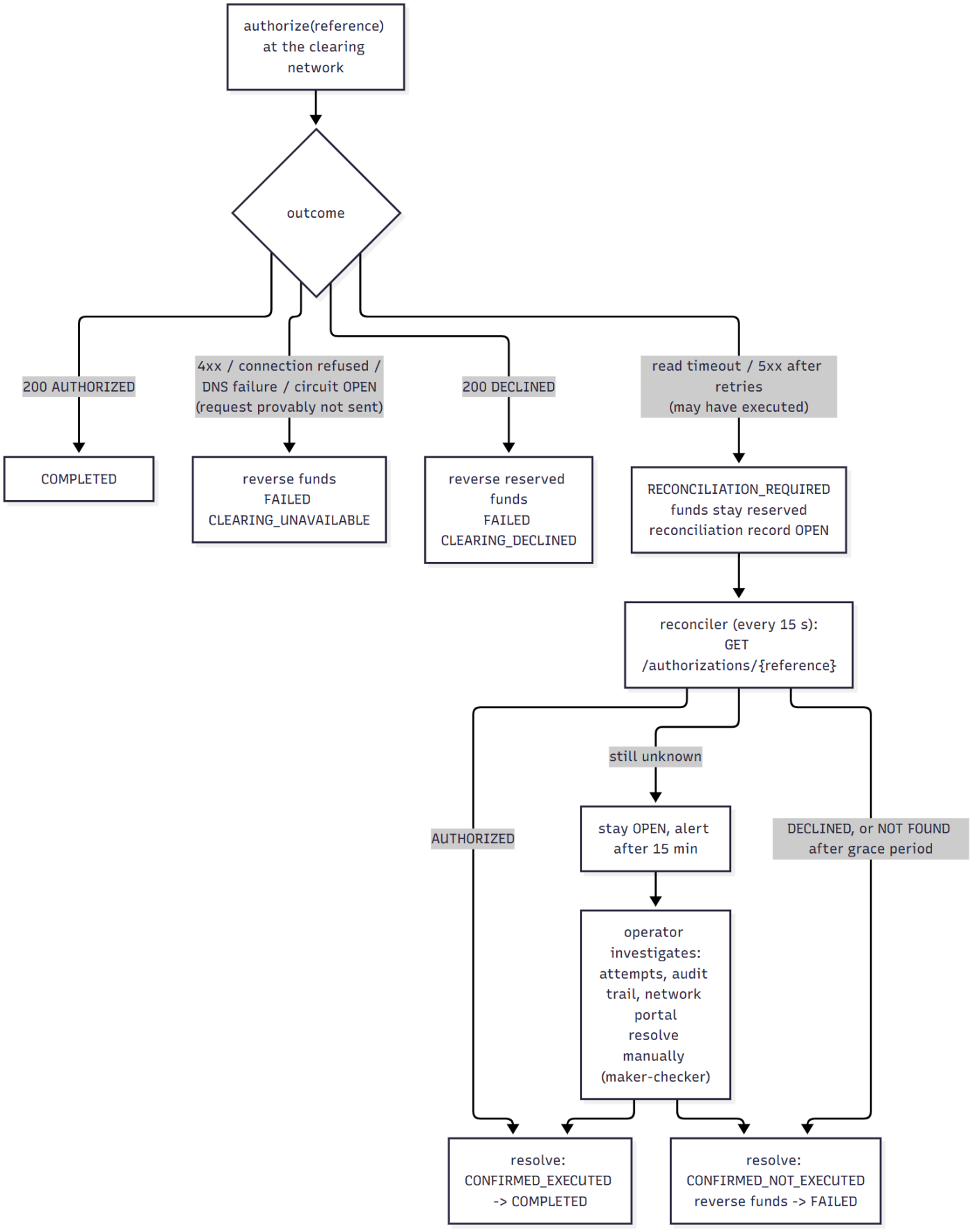


Figure 11. Error and recovery flow

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

The key discipline is the three-way classification of the call's result:

Result	Evidence	Action
Authorized / Declined	a definite answer from the network	complete, or refund + `FAILED`
NotSent	connection refused, DNS failure, 4xx, circuit open - the request provably never ran	refund immediately + `FAILED` `CLEARING_UNAVAILABLE` (safe to fail fast)
Unknown	read timeout, 5xx after retries - the network may have executed it	**do not refund**; `RECONCILIATION_REQUIRED`; reconcile

Treating "unknown" as "failed" would pay the customer twice (refund now, payout later); treating it as "success" could hold funds for a payout that never happened. Only reconciliation - asking the network - may decide.

Retries are applied **only** to the idempotent authorize call (the network deduplicates on `reference`), with exponential backoff and *full jitter* so many instances do not retry in lock-step against a recovering dependency. A circuit breaker (50 % failures over 10 calls, 10 s open) converts a sustained outage into immediate, cheap `NotSent` failures instead of every request waiting out its timeouts; the end-to-end run with the chaos switch exercised exactly this (docs/validation).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

CI/CD pipeline

CI/CD pipeline

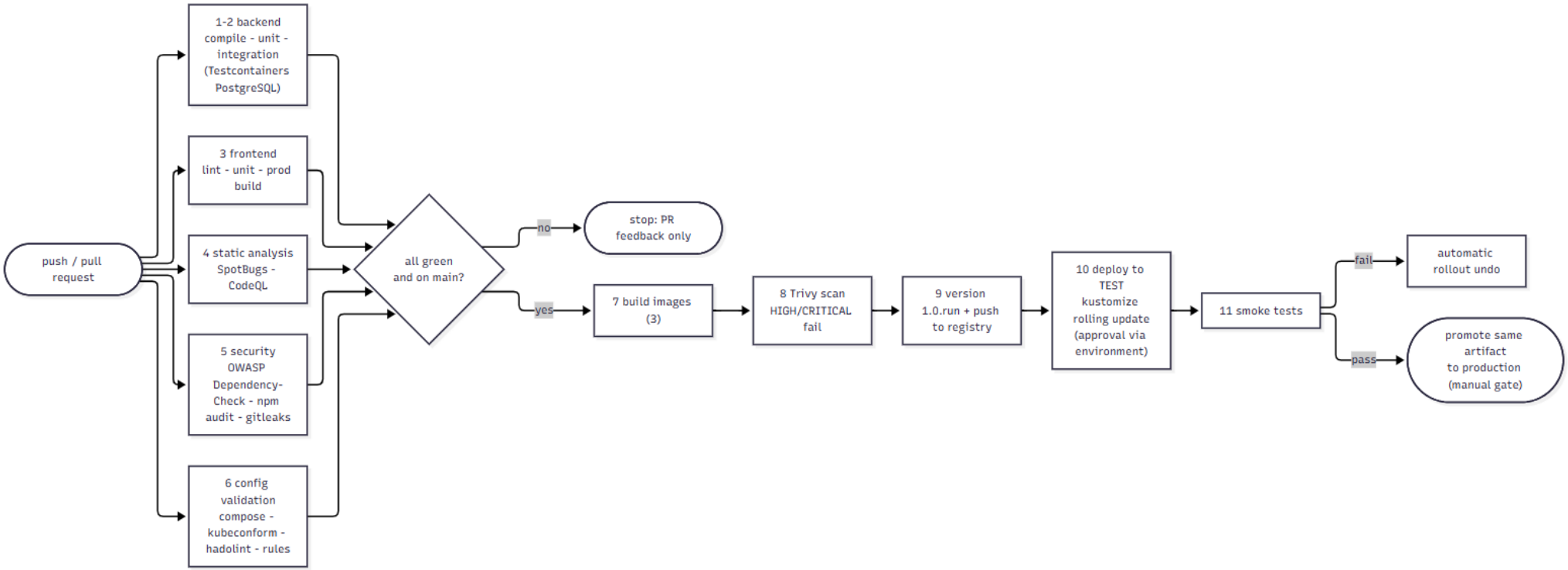


Figure 12. CI/CD pipeline

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Implemented in `.github/workflows/ci.yml` (GitHub Actions). **Status: reviewed and every constituent command allowed, but the workflow itself has not been run on a GitHub runner** (stated plainly in the validation report

#	Stage	Job
1	Java compilation + unit tests (JUnit 5, Mockito)	`backend`
2	Integration tests (Spring context, H2, embedded/Testcontainers PostgreSQL)	`backend` (failsafe `*IT`)
3	Angular lint, unit tests (Karma/Jasmine, headless Chrome), production build	`frontend`
4	Static analysis	`static-analysis`: SpotBugs (medium+), CodeQL (Java + Typ
5	Dependency vulnerability scanning + secret scanning	`security-scan`: OWASP Dependency-Check (CVSS >= 7), `np` gitleaks
6	Configuration validation	`config-validation`: `scripts/validate-infra.sh` (compose, kubeconform, hadolint, alert rules vs. metrics, dashboards, patterns)
7	Container image build (3 images, layer-cached)	`images`
8	Container image scanning	Trivy, fails on HIGH/CRITICAL with a fix available
9	Artifact versioning + publish	`1.0.<run number>` and `sha-<short>` tags to the registry, **mai**
10	Deployment to a test environment	`deploy-test`: kustomize overlay, rolling update, `rollout sta` skipped without credentials
11	Post-deployment smoke tests	`scripts/smoke-test.sh`: liveness, 401 on protected endpoint security headers, actuator not exposed

Design points: least-privilege `permissions:`; concurrency group cancels superseded runs; build **once**, promote the sa rebuild per environment); the OpenAPI contract is re-generated by the tests and `git diff --exit-code specs/openap` contract is stale; credentials for cluster access exist only as environment secrets (prefer OIDC federation to the clou production promotion is a manual approval of the *same* artefact. Branch protection requires the `backend`, `frontend`, `s`

Local infrastructure

`infrastructure/docker-compose.yml` runs PostgreSQL, the platform, the simulator, the nginx-served SPA, Prometheus and Jaeger. Hardening applied even locally: the database is not published to the host, secrets must come from `.env` containers are read-only with all capabilities dropped, networks are split (`backend`, `frontend`, `observability`), the app `stop_grace_period` longer than Spring's shutdown phase. **Not executed by the author** (no Docker on the machine

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Cloud deployment architecture

Kubernetes

`infrastructure/k8s/base`: namespace with the *restricted* pod-security level, ConfigMap (no secrets), Deployments, Services, HPA (2-8 replicas at 65 % CPU), PodDisruptionBudget, Ingress with TLS, and NetworkPolicies (default deny; the API accepts traffic only from the frontend and, on the management port, from the monitoring namespace). `overlays/test` adds the simulator and a single replica.

- **Probes:** startup (3 minutes for Flyway/warm-up), liveness, readiness on the management port.
- **Rolling updates:** `maxUnavailable: 0, maxSurge: 1` - a ready pod is never removed before its replacement is ready.
- **Graceful shutdown:** `preStop: sleep 10` lets the load balancer deregister the pod, then `SIGTERM` makes Spring stop accepting requests and drain for up to 20 s (`server.shutdown=graceful`, scheduler `await-termination`); `terminationGracePeriodSeconds: 45`.
- **Resources:** CPU request, **memory limit only** (CPU limits throttle JVMs and create latency spikes), JVM `MaxRAMPercentage=75`.
- **Secrets:** only `secretKeyRef`s; the Secret is created by External Secrets Operator from AWS Secrets Manager/Vault (`external-secret.example.yaml`).
- **Autoscaling caveat:** the dispatcher/reconciler run in every replica; they are safe to run concurrently (lease + optimistic locking), so scale-out never double-processes.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

AWS reference architecture

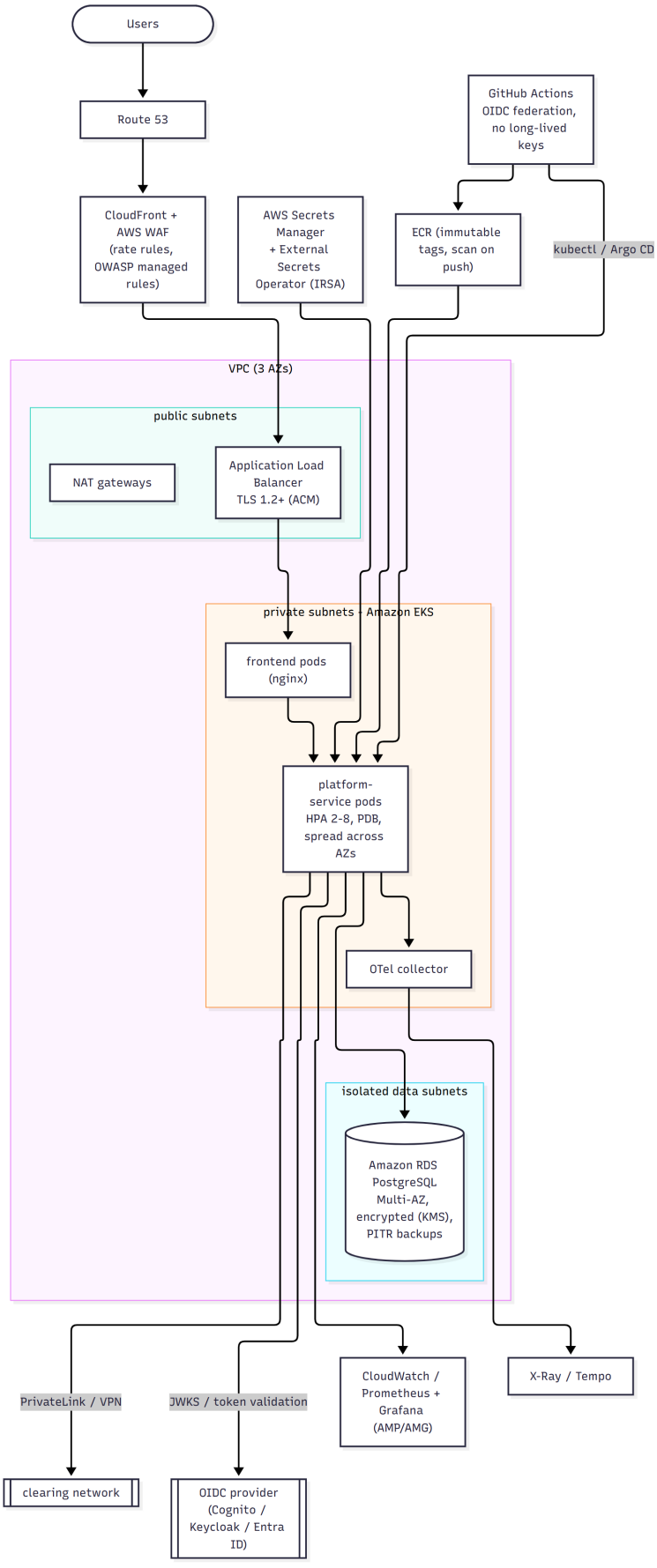


Figure 13. AWS deployment

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **Edge:** Route 53 -> CloudFront + **AWS WAF** (managed OWASP rule sets, rate rules) -> ALB (TLS via ACM).
- **Compute:** Amazon **EKS** across three AZs (private subnets); nodes patched via managed node groups; images in **ECR** (immutable tags, scan on push). ECS Fargate is a lighter alternative for a team without Kubernetes expertise.
- **Data:** Amazon **RDS for PostgreSQL**, Multi-AZ, encrypted with KMS, automated backups with point-in-time recovery, parameter group tuned (`max_connections`, `lock_timeout`, `statement_timeout`), read replica for reporting; connection pooling via the app (Hikari) or RDS Proxy.
- **Secrets:** AWS Secrets Manager (rotation) surfaced by External Secrets with **IRSA** (pod-level IAM, no node-wide credentials).
- **Identity:** an OIDC provider (Cognito, Keycloak, Entra ID) issues JWTs; the API only needs the issuer URL.
- **Observability:** Prometheus (Amazon Managed Prometheus) + Grafana (AMG), CloudWatch Logs for ECS-format JSON, X-Ray or Tempo via the OpenTelemetry Collector.
- **Pipeline:** GitHub Actions with OIDC federation to AWS (no static keys), pushes to ECR, deploys with `kubectl`/Argo CD.
- **Environment-specific configuration:** one ConfigMap per environment through kustomize overlays; behaviour flags (`app.processing.*`, rate limits, tracing sampling) are plain environment variables; nothing environment-specific is baked into the image.
- **Scaling:** HPA on CPU (add a custom metric on `platform_outbox_backlog` for processing throughput), RDS vertical scaling plus replicas, CloudFront/WAF absorbing abusive traffic.
- **Rollback:** application: `kubectl rollout undo` (previous ReplicaSet, seconds) or Argo CD revert; **database: forward-only migrations with the expand/contract pattern, so the previous application version always works against the new schema** (rolling the schema back is a last resort and means point-in-time recovery); feature flags/config for risky behaviour.
- **No cloud credentials are needed for the default development workflow** - the whole system runs locally with H2 and the simulator.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Observability architecture

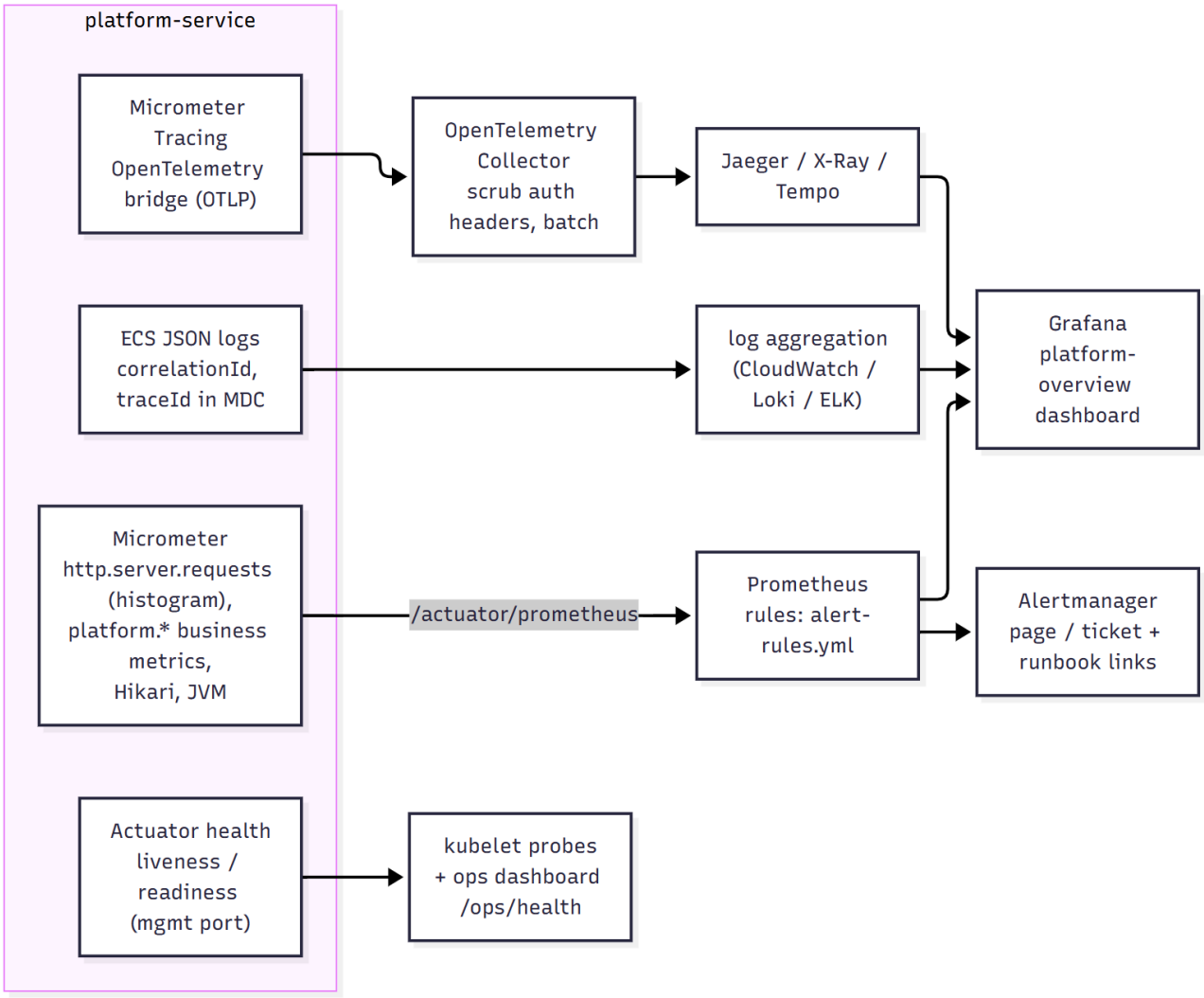


Figure 14. Observability architecture

Three pillars plus health, all wired to the same correlation id.

Logs

- Console logs; in containers `LOG_STRUCTURED_FORMAT=ecs` switches Spring Boot's built-in **structured logging** to Elastic Common Schema JSON (no extra dependency). Local runs keep a readable pattern: `%5p [%X{correlationId}] ...`.
- `CorrelationIdFilter` puts `correlationId` in the MDC for every request (safe-token validation blocks log injection), the structured format includes MDC entries and the trace id, the error envelope returns it, and `audit_events.correlation_id` stores it - one id joins the UI action, the log lines, the trace and the audit trail.
- Logging discipline: no PII, no tokens, no request bodies; account numbers masked (`Masking`); exceptions are logged once server-side at the point they are mapped to a response.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Metrics

Prometheus scrape of `/actuator/prometheus` (management port `9090` in containers, never routed through the ingress).

Metric	Source	Why
`http_server_requests_seconds_{count, sum, bucket}` (histogram + 0.5/0.95/0.99 + SLO buckets 100 ms/300 ms/1 s)	Spring Boot	latency percentiles, request and 5xx **error rate** (RED)
`platform_transactions_submitted_total{type, result}`	`PlatformMetrics`	submission volume and validation outcomes
`platform_transactions_finished_total{type, status}`	`PlatformMetrics`	terminal outcomes, failure rate
`platform_transactions_end_to_end_seconds` (histogram)	`PlatformMetrics`	**transaction-processing latency** (submit -> terminal state)
`platform_reconciliation_open`, `platform_outbox_backlog`, `platform_outbox_dead`	gauges (DB counts)	**queue metrics**: uncertain outcomes, pending work, dead letters
`platform_clearing_circuit_open`	gauge	circuit breaker state
`hikaricp_connections_{active, idle, pending, max}`, `hikaricp_connections_acquire_seconds`	HikariCP	**connection-pool** saturation
`jvm_memory_*`, `jvm_gc_pause_seconds_*`, `jvm_threads_*`, `process_cpu_usage`	Micrometer JVM binders	memory, GC, thread starvation

The Grafana dashboard `observability/grafana/dashboards/platform-overview.json` (provisioned automatically in Compose) charts all of the above; `observability/prometheus/alert-rules.yml` defines 11 alerts, each linking to a runbook section (a validation check fails the build if an alert has no runbook).

Traces

Micrometer Tracing with the OpenTelemetry bridge exports OTLP (`OTLP_TRACES_ENDPOINT`) to an OpenTelemetry Collector (which scrubs `authorization`/`cookie` attributes) and then to Jaeger (X-Ray/Tempo in AWS). Sampling is `TRACING_SAMPLING` (1.0 in dev, 0.1 in the test overlay) and export is off unless `OTLP_EXPORT_ENABLED=true` - tracing is opt-in per environment. Outgoing REST calls to the clearing network are instrumented automatically by the Spring `RestClient` observation, so the trace shows the external hop and its retries.

Health

`/actuator/health/liveness` and `/readiness` (Kubernetes probes). The staff-only `/api/v1/ops/health` is the **business** view that the dashboard shows: database round trip, clearing network reachability and circuit state, outbox backlog/dead letters, open reconciliation records - status `UP`, `DEGRADED` (dead letters or a degraded dependency) or `DOWN` (database unreachable).

From signal to action

Alert -> runbook section (`docs/operations/runbooks.md`) -> dashboard panel -> correlation id -> logs/trace/audit trail. The investigation guides (`docs/operations/performance-troubleshooting.md`) walk from each symptom (high memory, long GC, thread starvation, slow SQL, pool exhaustion, lock contention, elevated latency, database outage, partial failure) to its cause.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Security threat model

Method: STRIDE per trust boundary, ranked by impact on money and data, each threat mapped to a concrete control **and** to evidence (test or configuration). Scope: the platform as implemented; assets: account balances and ledger integrity, customer PII, the audit trail, credentials/tokens, availability of transaction processing.

Trust boundaries and actors

1. Internet -> edge (WAF, TLS) -> Angular SPA / API (untrusted callers, including authenticated-but-malicious customers).
2. SPA/API -> identity provider (tokens are trusted only after signature/expiry/issuer validation).
3. API -> database (trusted, but least-privilege account; SQL built only with bound parameters).
4. API -> external clearing network (semi-trusted: it can be slow, wrong, or unreachable).
5. Operators/administrators -> admin endpoints (privileged insiders can err or abuse).

Threat register

#	Threat (required topic)	STRIDE	Impact	Controls	Evidence
T1	**Broken object-level authorization** - a customer reads or changes another customer's accounts/ transactions by guessing ids	E, I	critical	UUID ids; `AccessPolicy` on every read/command; `404` not `403`; customer scope forced server-side for list/search; audit of denials	`SecurityIT` (accounts, balances, history, ledger, tx read/cancel/ approve, searches, submit/transfer into others' accounts, orphan token), `AccessPolicyTest`
T2	**Unauthorized transaction submission** - wrong role, someone else's account, support agent moving money, forged token	S, E	critical	`@PreAuthorize`; `canSubmit` per type (transfer must stay in one customer); JWT signature/expiry/ issuer; unknown roles ignored	`SecurityIT` (forged token, expired, tampered, support cannot submit, unknown role), `AccessPolicyTest`
T3	**Duplicate transaction requests** - client retry, double click, replay by an attacker	T, R	high	mandatory `Idempotency-Key`, unique `(principal,key)`, payload fingerprint, replay header, 7-day retention	`IdempotencyIT` (incl. 12 concurrent), e2e duplicate scenario, load baseline (25 keys x ~1,100 requests -> exactly 25 deposits)
T4	**SQL injection**	T, I	critical	JPA/Criteria with bound parameters only; no native string concatenation; typed path variables; sort whitelist	`SecurityIT.inputHardening` (injection strings in search params are inert), code review rule

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

#	Threat (required topic)	STRIDE	Impact	Controls	Evidence
T5	**Credential exposure** - secrets in the repo, logs, images, client bundle	I	critical	nothing committed (`.env.example` placeholders only); compose requires secrets from `.env`; Kubernetes `secretKeyRef` + External Secrets; token in memory only in the SPA; gitleaks in CI; secret-pattern scan in `validate-infra.py`; local issuer key random per start	`validate-infra.py`, CI `security-scan` job
T6	**Privilege escalation** - role claim manipulation, mass assignment, maker-checker bypass	E	critical	roles only from a *validated* token; unknown roles ignored; strict JSON (unknown fields rejected); maker-checker in `AccessPolicy.canReview`; admin endpoints `@PreAuthorize`	`SecurityIT` (role rules, mass assignment), `TransactionFlowIT.reviewThresholdAndMakerChecker`
T7	**Sensitive data in logs and errors**	I	high	one error envelope, generic 500 message; no PII/tokens/bodies logged; masking utilities; OTel collector strips auth headers	`ServerErrorIT` (fake JDBC URL + password never reach the client), `SecurityIT.noInformationLeakage`, `MaskingAndPagingTest`
T8	**Race conditions** - overspend, lost update, deadlock, double posting	T	critical	ordered row locks, authoritative checks under lock, unique ledger postings, optimistic versions, DB `CHECK balance >= 0`	`ConcurrencyIT` + `PostgresConcurrencyIT`, `SchemaConstraintsIT`, `LedgerServiceTest`
T9	**Denial of service** - request floods, huge bodies, expensive queries, retry storms, slow dependency	D	high	per-subject and per-address rate limits; 64 KB body limit; page cap 100; sort whitelist; Hikari 3 s acquire timeout; 5 s lock wait; timeouts + circuit breaker + jittered backoff on the external call; HPA	`RateLimitIT`, `ApiContractIT` (413, page cap), chaos e2e
T10	**Insecure administrative operations** - drain, requeue, resolve, suspend, close	E, R	high	admin-only `@PreAuthorize`; maker-checker on approvals and reconciliation resolutions; every action audited with actor and correlation id; management port separate from the public port	`SecurityIT.roleRules`, `TransactionFlowIT` (support cannot resolve)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

#	Threat (required topic)	STRIDE	Impact	Controls	Evidence
T11	Tampering with the audit trail	T, R	high	append-only; PostgreSQL trigger rejects UPDATE/DELETE; audit rows written in the same transaction as the change	`PostgresFeaturesIT.auditTrailIsImmutable`
T12	Token theft / XSS	S, I	high	token in memory only, strict CSP, no `innerHTML`, Angular sanitisation on, short TTL (30 min), HTTPS + HSTS	unit test asserts nothing in web storage; nginx CSP
T13	CSRF	T	medium	stateless bearer tokens in a header (not cookies); CORS allow-list; documented trigger to re-enable CSRF if cookies are introduced	`SecurityIT.cors`
T14	Replay of an old token after logout	S	medium	short TTL; (production) token revocation / refresh-token rotation at the IdP	design note
T15	Enumeration and user probing via login	I	medium	identical `401` message for unknown user and wrong password, constant-time comparison path (decoy hash), login rate limit	`SecurityIT.localLogin`, `RateLimitIT.loginIsThrottled`
T16	Supply-chain compromise (dependency, base image, CI action)	T	high	pinned versions, OWASP Dependency-Check, `npm audit`, Trivy, minimal dependencies (no Spring Cloud/broker/Redis), non-root distroless-style images, pin actions by SHA in production	CI pipeline
T17	Lateral movement inside the cluster	E	high	default-deny NetworkPolicy, per-service allow rules, restricted pod security, no service-account tokens mounted, read-only root FS	`validate-infra.py` asserts these on the manifests
T18	Uncertain external outcome exploited (double payout / lost money)	T	critical	three-way outcome classification, funds reserved before the call, reconciliation with maker-checker	`TransactionProcessorTest`, `TransactionFlowIT`, chaos e2e

Residual risks and next steps (production)

- In-memory rate limiting is per instance - use a shared store or the gateway.
- Local issuer HS256 secrets are acceptable for a demo only; real deployments use the IdP's asymmetric keys.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- No field-level encryption or tokenisation of identifiers; no key rotation procedure for a data-encryption key.
- No anomaly detection on audit events (e.g. repeated `ACCESS_DENIED` from one subject); recommended SIEM rule.
- Four-eyes is implemented for approval and reconciliation, not yet for customer suspension or account closure.
- An independent penetration test and a data-protection impact assessment are prerequisites for any real use.

OWASP mapping

OWASP Top 10 (2021)

Risk	Control in this repository	Evidence
A01 Broken Access Control	role rules + object-level `AccessPolicy`, deny-by-default, 404 for foreign objects, maker-checker	`SecurityIT`, `AccessPolicyTest`
A02 Cryptographic Failures	TLS at the edge + HSTS; JWT signature validation (asymmetric with an IdP); BCrypt for the demo issuer; secrets via environment/ secret manager; passwords never logged	`SecurityConfig`, `LocalIssuerConfig`, k8s manifests
A03 Injection	bound parameters only (JPA/Criteria), allow-list validation, strict JSON, memo regex, sort whitelist	`SecurityIT.inputHardening`
A04 Insecure Design	threat model, state machine, idempotency, maker-checker, reconciliation, rate limits, ADRs	`docs/security/threat-model.md`
A05 Security Misconfiguration	secure headers, CORS allow-list, deny-by-default, management port separated, hardened containers, restricted pod security, no default credentials (secrets required)	`validate-infra.py`, `SecurityIT`
A06 Vulnerable and Outdated Components	pinned versions; OWASP Dependency-Check, `npm audit`, Trivy; minimal dependency set	CI `security-scan`, `images`
A07 Identification and Authentication Failures	JWT validation (signature/expiry/issuer), login throttling, generic login error, short token TTL, in-memory token in the SPA	`SecurityIT`, `RateLimitIT`
A08 Software and Data Integrity Failures	CI builds once and promotes the same image; Flyway checksums (`validate-on-migrate`); append-only audit; contract test	CI, `ApiContractIT`
A09 Security Logging and Monitoring Failures	audit trail incl. denials and logins; correlation ids; structured logs; alert rules (dead letters, reconciliation, circuit open, 5xx)	`AuditService`, `alert-rules.yml`
A10 Server-Side Request Forgery	no user-controlled URLs; the only outbound call goes to a configured base URL with fixed paths	`RestClearingClient`

OWASP API Security Top 10 (2023)

Risk	Control	Evidence
API1 Broken Object Level Authorization	`AccessPolicy` on every object access; scope forced for list/search	`SecurityIT` (BOLA tests)
API2 Broken Authentication	JWT validation, login throttle, no session, short TTL	`SecurityIT`, `RateLimitIT`
API3 Broken Object Property Level Authorization	DTOs only; PII masking per role; tax id never returned; strict input (no mass assignment)	`SecurityIT.piiMasking`, `inputHardening`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Risk	Control	Evidence
API4 Unrestricted Resource Consumption	rate limit, body limit, page cap, sort whitelist, timeouts, pool limits	`RateLimitIT`, `ApiContractIT`
API5 Broken Function Level Authorization	`@PreAuthorize` per endpoint; admin-only operations	`SecurityIT.roleRules`
API6 Unrestricted Access to Sensitive Business Flows	idempotency, review threshold, maker-checker, daily limit, rate limit on submission	`TransactionFlowIT`, `IdempotencyIT`
API7 Server Side Request Forgery	see A10	-
API8 Security Misconfiguration	see A05; error envelope without internals	`ServerErrorIT`
API9 Improper Inventory Management	single versioned API, committed OpenAPI contract, docs endpoints only when `app.docs.public=true` (local)	`ApiContractIT`
API10 Unsafe Consumption of APIs	the clearing client treats the remote as untrusted: timeouts, strict response mapping (unknown status -> `Unknown`), idempotent calls, circuit breaker	`RestClearingClientTest`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Test strategy

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

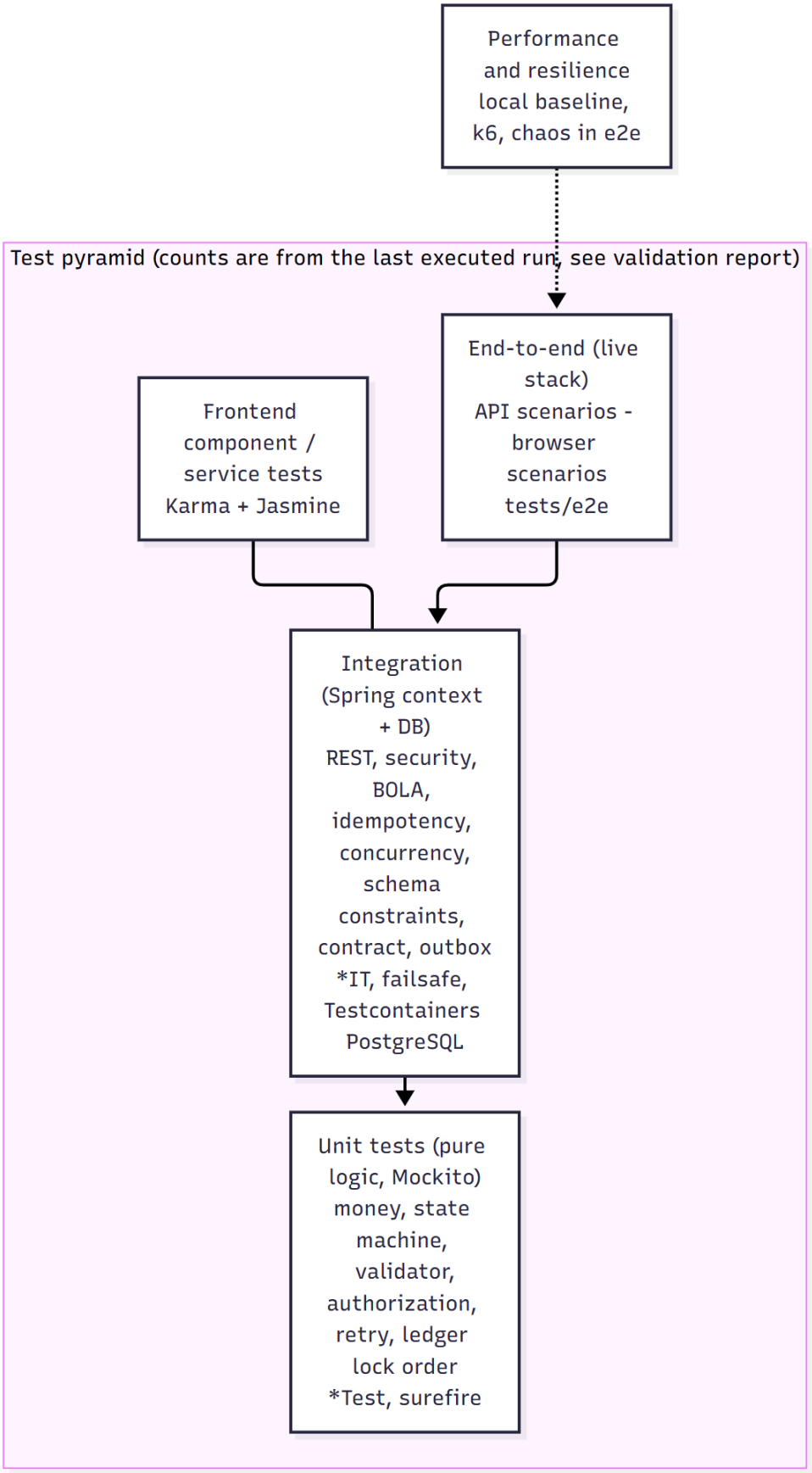


Figure 15. Test strategy

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Principles

1. **Test the money invariants hardest.** The properties that must never break are: balance never negative, balance == sum of ledger, a transfer is all-or-nothing, a key is applied once, no cross-customer access. They are asserted after *every* concurrency scenario (`assertLedgerIntegrity()`), not just where a test is "about" them.
2. **Pure logic gets unit tests; wiring gets integration tests; journeys get end-to-end tests** - and each layer is kept small enough to run often. Business rules are deliberately written as pure functions (`TransactionValidator`, `AccessPolicy`, `DailyLimitPolicy`, `MoneyPolicy`) so each rule is one assertion.
3. **Real infrastructure where behaviour depends on it.** Locking, isolation, constraints, migrations and query plans are tested against a real database (H2 in PostgreSQL mode for speed, a real PostgreSQL 14 embedded in the test JVM for fidelity, Testcontainers in CI).
4. **Do not claim what was not executed.** Test counts in this document come from executed runs recorded in `docs/validation/validation-report.md`; anything not run is listed there as not run.

Layers

Layer	Tooling	Location	What it proves
Unit (backend)	JUnit 5, AssertJ, Mockito	`src/test/**/*Test.java` (surefire)	money policy (precision, HALF_EVEN, allocation, equals vs compareTo), 81-pair state machine, validator rules, authorization matrix, retry/jitter, token bucket, daily-limit policy (TDD), ledger **lock ordering** and compensation, processor outcome mapping (Declined/ NotSent/Unknown), outbox dispatcher lease/backoff/ dead-letter, idempotency fingerprint, REST clearing client against a real in-JVM HTTP server (retries, timeouts, circuit breaker, outcome classification)
Integration (backend)	Spring Boot Test, MockMvc, JdbcTemplate, H2 + Flyway + seed data	`*IT.java` (failsafe)	full request path with **real JWT validation**: BDD scenarios (`TransactionFlowIT`), security and BOLA (`SecurityIT`), idempotency incl. 12-thread race (`IdempotencyIT`), concurrency (`ConcurrencyIT`), DB constraints (`SchemaConstraintsIT`), outbox dead-letter (`OutboxDeadLetterIT`), contract + error envelopes + metrics exposure (`ApiContractIT`), rate limit (`RateLimitIT`), server-error sanitisation (`ServerErrorIT`)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Layer	Tooling	Location	What it proves
Integration on **real PostgreSQL**	embedded PostgreSQL (zonky) - no Docker; Testcontainers variant for CI	`PostgresFlowIT`, `PostgresConcurrencyIT`, `PostgresFeaturesIT`, `TestcontainersPostgresConcurrencyIT`	the same BDD and race scenarios on real row locks/ MVCC; the PostgreSQL migration (partial indexes), the append-only audit trigger, query plans
API / contract	committed OpenAPI vs live document; Postman collection + `.http` file	`ApiContractIT`, `tests/api`	the contract cannot drift silently; the collection can be run with Newman
Frontend unit/ component	Karma + Jasmine, `HttpTestingController`	`frontend/src/**/*.spec.ts`	services, interceptors, guards, forms, state handling, role-aware UI, money formatting
End-to-end (API)	Node scripts against the live stack incl. the clearing simulator	`tests/e2e/e2e-api.mjs` (+ `--with-chaos`)	customer workflows, negative authorization, retries/timeouts/declines, reconciliation, circuit breaker and recovery
End-to-end (browser)	puppeteer-core driving Edge/Chrome headless against the production bundle	`tests/e2e/ui-e2e.mjs`	sign-in, customer journey, operations journey, support masking, mobile layout, no console errors
Performance	local baseline script (executed), k6 scripts (written)	`performance/`	invariants under load, relative behaviour; **not** capacity claims
Regression	every defect found by a test keeps its test	see "Defects the tests found" below	

Defects found by the test suite during development (kept as regression tests)

Found by	Defect	Fix
integration tests	`VARCHAR(20)` column too short for `CONFIRMED_NOT_EXECUTED` (Flyway `validate` passed; the insert failed)	widened to 30; schema/enum lengths reviewed
integration tests	`CONCURRENT_MODIFICATION` on every submit: a getter returning `0` for a null `@Version` made Spring Data `save()` merge a new entity	wrapper `Long` version, comment explains why
unit test	`UUID.compareTo` is *signed*; the test assumed unsigned order	test uses same-sign UUIDs; the doc states any total order works if used consistently
browser E2E	horizontal overflow of the page at 390 px (toolbar)	toolbar items hidden/ellipsised on small screens
browser E2E	fonts fell back to a serif face (Material theme referenced an unavailable `Roboto`)	explicit system font stack in the theme
load baseline	processing rate capped by "one batch per tick" scheduling (~23 tx/s)	work-conserving drain loop (~34 tx/s measured, same machine)
review	`@Primary` on one of two `TransactionTemplate` beans would silently override name-based injection	removed; documented

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Found by	Defect	Fix
test code	latch deadlock: a pool smaller than the task count with a start gate	one thread per task

Test data

Synthetic seed data (`db/seed/V1000__synthetic_seed_data.sql`: five customers, ten accounts in four currencies, one suspended customer, one suspended account, one account awaiting activation) is loaded in `local` and `test` profiles only. Integration tests create their **own accounts** (via the public API) so tests never interfere with each other, and the context is cached across test classes for speed.

Coverage

JaCoCo runs in the build (`target/site/jacoco*`); coverage is a *signal*, not a target - the invariants above matter more than a percentage. The numbers of the last run are recorded in the validation report.

How to run

```
mvn -f backend/pom.xml verify                      # unit + integration (incl. embedded PostgreSQL) +
coverage
mvn -f backend/pom.xml test                        # unit tests only (fast)
( cd frontend && npx ng test --watch=false --browsers=ChromeHeadlessNoSandbox )
scripts/run-local.sh && node tests/e2e/e2e-api.mjs --with-chaos && node tests/e2e/ui-e2e.mjs
```

Complete TDD example: daily withdrawal limit

Feature (product request): "*A customer must not be able to withdraw more than the account's daily limit in any rolling 24-hour window, including withdrawals still in flight.*"

0. Acceptance criteria (written before any code)

#	Criterion
AC1	A withdrawal below the remaining allowance is accepted.
AC2	A withdrawal that exactly reaches the limit is accepted.
AC3	One cent over the limit is rejected with code `DAILY_LIMIT_EXCEEDED`.
AC4	Earlier withdrawals in the window (completed, processing, or awaiting reconciliation) count against the limit.
AC5	A limit of zero blocks every withdrawal.
AC6	Comparison ignores `BigDecimal` scale (`1000.0` equals `1000.0000`).
AC7	Under concurrency the limit is still respected (two parallel withdrawals cannot both slip under it).
AC8	Cancelled, rejected and failed withdrawals do **not** count.

1. RED - write the failing test first

`DailyLimitPolicyTest` (six tests mirroring AC1-AC6) was written against a stub whose body is `throw new UnsupportedOperationException("not implemented")`. Running it (real output of the run, scratch module):

```
[ERROR] Tests run: 6, Failures: 0, Errors: 6, Skipped: 0, Time elapsed: 0.243 s <<< FAILURE! --
in ...DailyLimitPolicyTest
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
[ERROR] ...DailyLimitPolicyTest.cumulativeWithdrawalsCount ...
java.lang.UnsupportedOperationException: not implemented
[ERROR] ...DailyLimitPolicyTest.scaleDoesNotMatter           ...
java.lang.UnsupportedOperationException: not implemented
[ERROR] ...DailyLimitPolicyTest.belowLimitIsAllowed          ...
java.lang.UnsupportedOperationException: not implemented
[ERROR] ...DailyLimitPolicyTest.exactlyAtLimitIsAllowed      ...
java.lang.UnsupportedOperationException: not implemented
[ERROR] ...DailyLimitPolicyTest.oneCentOverIsRejected        ...
java.lang.UnsupportedOperationException: not implemented
[ERROR] ...DailyLimitPolicyTest.zeroLimitBlocksEverything    ...
java.lang.UnsupportedOperationException: not implemented
```

A red test that fails for the *right reason* ("not implemented", not a typo or a compile error) is the point of this step.

2. GREEN - the minimum correct behaviour

```
public static boolean wouldExceed(BigDecimal dailyLimit, BigDecimal alreadyWithdrawn, BigDecimal
requested) {
    return alreadyWithdrawn.add(requested).compareTo(dailyLimit) > 0;   // compareTo: 1000.0 ==
1000.0000
}
```

```
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 -- in ...DailyLimitPolicyTest      BUILD
SUCCESS
```

Nothing else was written: no repository, no controller, no configuration. One line satisfies AC1-AC6.

3. Wire it in, still test-first

- `TransactionValidatorTest.dailyLimit` (AC3/AC4 at the rule level) - the validator receives `withdrawnInWindow` as an *input*, so the rule stays pure.
- `TransactionRepository.sumWithdrawnSince(accountId, since, statuses, excludeId)` provides the sum; the statuses that count are an explicit set (`PROCESSING, COMPLETED, RECONCILIATION_REQUIRED` - AC4/AC8: cancelled/rejected/failed are absent).
- `account.daily_withdrawal_limit` column (migration), `AccountType` defaults, `OpenAccountRequest.dailyWithdrawalLimit`.

4. REFACTOR - with the tests green

- The check existed in two places (advisory at submit, authoritative at processing). Both now call `DailyLimitPolicy.wouldExceed`, so the rule has one definition; the 6 + 10 unit tests stayed green throughout.
- The authoritative evaluation moved **under the account row lock** inside `LedgerService.reserveWithdrawal` (AC7) - the refactor that matters: without it two parallel withdrawals could each read "40 withdrawn so far" and both pass.

5. Integration tests

Test	Proves
`TransactionFlowIT.dailyLimitEnforced` (H2) and `PostgresFlowIT` (PostgreSQL)	the HTTP path: 60 accepted, then 40.01 rejected with `DAILY_LIMIT_EXCEEDED`, then 40.00 accepted
`ConcurrencyScenarios.concurrentWithdrawalsRespectDailyLimit`	AC7: eight parallel 40.00 withdrawals against a 100.00 limit -> exactly two complete (80.00), balance 4,920.00, ledger intact
`TransactionFlowIT.dailyLimitIgnoresCancelledAndFailed`	AC8: a cancelled 60.00 and a declined (failed) 60.13 consume no allowance - the full 100.00 is still available

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Test	Proves
`LedgerServiceTest.reserveWithdrawal`	the limit is enforced inside the locked section and nothing is mutated on rejection

6. Documentation and acceptance

AC1-AC8 are covered as listed above; the BDD wording is in `tests/bdd/transactions.feature` (`Daily withdrawal limit` outline); the validation rule is in the domain chapter; the API reports the failure as `422 TRANSACTION_REJECTED` with `details.failureCode = DAILY_LIMIT_EXCEEDED`.

What the example demonstrates

Small steps, a failing test that fails for the right reason, the simplest implementation, refactoring under a green bar, pushing the rule to where it is *safe* (under the lock), and letting the concurrency test - not the unit test - prove the property that actually matters in production.

BDD scenarios

The Gherkin text is in `tests/bdd/transactions.feature`. It is kept as the **specification** and every scenario is bound to automated tests by name (there is deliberately no Cucumber runtime - the scenarios run as JUnit integration tests and as live end-to-end checks, so wording and behaviour cannot drift apart inside a glue-code layer nobody maintains).

Scenario (required)	Given / When / Then (summary)	Automated by
A valid withdrawal	account 1,000.00; withdraw 250.00 -> 202 `VALIDATED`, then `COMPLETED`, balance 750.00, ledger integrity holds	`TransactionFlowIT.validWithdrawal`, `PostgresFlowIT` (same), e2e `valid withdrawal`
A withdrawal exceeding the available balance	account 100.00; withdraw 100.01 -> 422 `TRANSACTION_REJECTED` / `INSUFFICIENT_FUNDS`, recorded and audited, balance unchanged	`TransactionFlowIT.withdrawalExceedingBalance`, e2e `insufficient funds`
A duplicate transaction submission	same key twice or 3x concurrently -> one transaction, replay header, applied once; same key + different amount -> 422 `IDEMPOTENCY_KEY_REUSED`	`IdempotencyIT` (6 tests incl. 12 threads), e2e `duplicate submission`
A transfer between two accounts	A 500.00, B 50.00; transfer 200.00 -> A 300.00, B 250.00, one DB transaction	`TransactionFlowIT.transferBetweenAccounts`, e2e `transfer`
Concurrent requests against the same account	20 x 10.00 against 100.00 -> exactly 10 complete, 10 `INSUFFICIENT_FUNDS`, balance 0.00; opposing transfers never deadlock	`ConcurrencyScenarios` on H2 and on real PostgreSQL; e2e `concurrent withdrawals`
An unauthorized account-access attempt	Alice requests Bob's account/balance/history or submits against it -> 404, `ACCESS_DENIED` audited, balance unchanged	`SecurityIT` (BOLA tests), e2e `unauthorized account access`
A failed transaction requiring investigation	clearing times out -> `RECONCILIATION_REQUIRED`, funds reserved, attempts + audit visible, auto/manual resolution, money paid out once or returned in full	`TransactionFlowIT.unknownOutcome*`, `stuckProcessingIsRecovered`, e2e `clearing network...` and chaos

Additional scenarios: maker-checker approval, daily limit (outline with examples), cancellation vs processing, clearing decline/refund, not-sent refund, closing an account, search/pagination/sorting, dashboard data.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Acceptance-criteria template used for every feature

```
Feature: <business capability>
  Rule: <one business rule>
    Scenario: <example>   Given <state> When <action> Then <observable outcome> And <invariant that must still hold>
```

Every scenario ends with an **invariant** clause (ledger integrity, nothing half-applied, audit written). That habit is what turns a happy-path demo into a specification of a financial system.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

AI-assisted development workflow

A practical, tool-agnostic setup for working with GitHub Copilot (or any coding assistant) **without lowering the quality bar**. It is a set of *files an assistant can read*, a *workflow* that keeps a human accountable, and *review recipes* for the specific ways AI-generated code goes wrong in a financial system.

> **Honesty note.** Nothing in this workspace was produced by an autonomous multi-agent run. The files are written artefacts (instructions, prompts, checklists, agent *definitions*, templates). The agent definitions are optional task descriptions; no claim is made that any agent executed them. The only "AI execution" claim that can be made about this repository is that an AI assistant helped author it, and its output was verified by the build, the test suites and the live runs documented in `docs/validation/validation-report.md`.

Layout

```
.github/copilot-instructions.md        always-on, SHORT: the ten critical rules + commands
.github/instructions/*.instructions.md path-scoped rules (applyTo): java, angular, migrations, tests
.github/prompts/*.prompt.md            reusable prompts: spec-driven feature, review-ai-code
ai-engineering/
  context/            modular reference files (loaded on demand, not pasted into every prompt)
    architecture-context.md  java-standards.md  angular-standards.md  api-rules.md  db-migration-
guide.md
    security-review.md  unit-test-guidance.md  integration-test-guidance.md  performance-checklist.md
    pr-review-checklist.md  definition-of-done.md  release-readiness.md
  agents/             optional multi-agent task definitions: implementation, testing, security-review,
documentation
  handoff-template.md         structured hand-off between agents/people
  review-examples.md          how to review AI output: races, authz, money, retries, tests (bad ->
good)
  workflow-example.md         the account-transfer spec walked through the 11 steps
specs/001-account-transfer.md  the concrete specification-driven example
```

Context engineering: small always-on file, modular on-demand files

Repeating the whole architecture in every prompt wastes context and dilutes the rules that matter. The design:

- **Tier 1 (always loaded, ~35 lines):** the ten rules whose violation costs money or data - `copilot-instructions.md`.
- **Tier 2 (loaded by file pattern):** `.github/instructions/*.instructions.md` with `applyTo`, 5-8 lines each, pointing at tier 3.
- **Tier 3 (loaded when the task needs it):** `ai-engineering/context/*.md`, referenced by name in prompts (`@workspace read ai-engineering/context/security-review.md`).
- **Tier 4 (the source of truth):** ADRs, the spec, the code and the tests - the assistant is told to *read the code*, not to trust prose that may have drifted.

Each fact lives in exactly one place; other files link to it.

The workflow (specification-driven, human in the loop)

1. **Requirements analysis** - the human writes or approves the spec; the assistant restates it and lists open questions.
2. **Acceptance criteria** - Given/When/Then including negative, security and concurrency cases.
3.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Technical design - classes, transaction boundaries, lock order, failure handling; the human approves *before* code.

4. **API contract** - OpenAPI-visible DTOs, status codes, compatibility check.
5. **Data-model changes** - additive migration plan (expand/contract).
6. **Implementation** - small slices, tests first; assistant output is a *draft*.
7. **Test generation** - from criteria, not from the implementation; mutation thinking ("would this fail if the code were wrong?").
8. **Security review** - `context/security-review.md` (+ the `review-ai-code` prompt).
9. **Performance review** - `context/performance-checklist.md`.
10. **Documentation** - contract, ADR, runbook.
11. **Release planning** - readiness checklist, rollback plan. A human reviews every line before merge (`context/pr-review-checklist.md`); the Definition of Done is explicit (`context/definition-of-done.md`).

Guardrails when using an assistant

- Never paste secrets, real customer data or production dumps into a prompt; use the synthetic seed data.
- Treat suggestions as untrusted input: review them like a pull request from a stranger; run the tests; read the diff.
- Prefer asking the assistant to *explain a failing test* or *propose test cases* over generating large implementations unseen.
- Dependency suggestions need justification (licence, maintenance, CVEs) and an ADR; the assistant may not add libraries on its own.
- Safe dependency upgrades: one dependency at a time, read the changelog, run the full suite, check the Dependency-Check delta, and prefer the Boot BOM to manual versions.
- Record AI involvement in the PR description; the author remains accountable.

What assistants are good and bad at here

Good at	Needs close review
boilerplate (DTOs, mappers, specs), test scaffolding, refactoring with tests green, explaining unfamiliar code, drafting docs and runbooks	concurrency and locking, authorization boundaries, money arithmetic, retry/timeout design, migration safety, anything touching security configuration

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Diagram: the specification-driven workflow

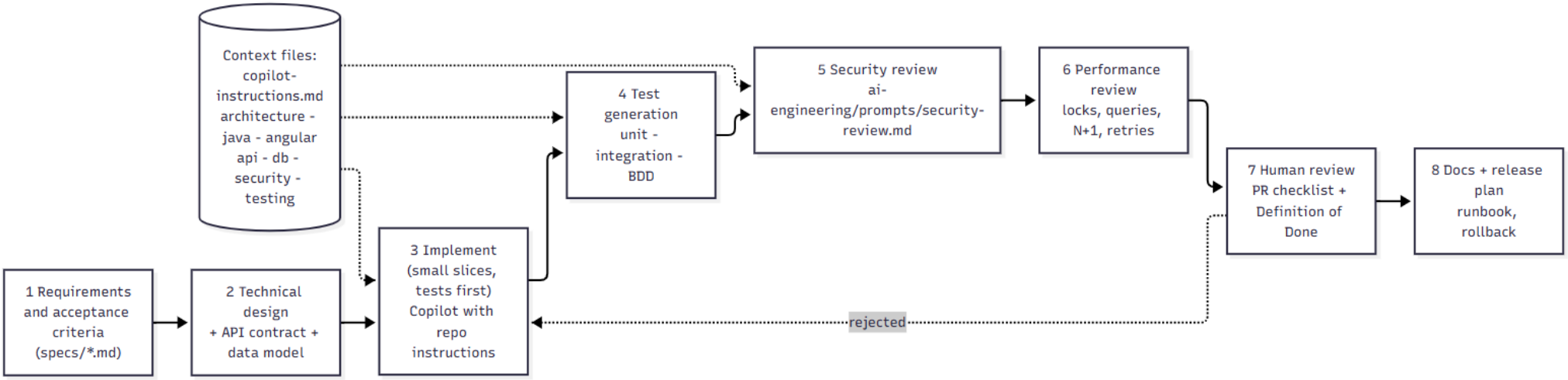


Figure 16. AI-assisted development workflow

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Workflow example: implementing the account transfer with an assistant

The specification is `specs/001-account-transfer.md`. How the eleven steps looked (what the **human** decides is in bold):

Step	With the assistant	Human decision / check
1 Requirements	"Restate spec 001, list open questions" -> asks whether transfers cross customers, whether fees apply, whether the daily limit applies	**answers: same customer only, no fees, limit applies to withdrawals only**
2 Acceptance criteria	proposes negative/security/concurrency cases (opposing transfers, foreign account, key reuse)	**accepts 12 criteria**
3 Design	proposes `applyTransfer` with ordered locks and one DB transaction; lists alternatives (saga, conditional UPDATE)	**GATE 1: approves design, rejects the saga**
4 API contract	confirms reuse of `POST /transactions` and error codes; flags `accountsConsistentWithType` validation	**approves, checks OpenAPI diff**
5 Data model	none needed - points at existing `CHECK` and unique constraints	**confirms no migration**
6 Implementation	slices: state machine -> validator -> ledger -> service -> processor, each after a failing test	reviews each diff
7 Tests	generates tests from criteria; mutation notes (remove lock-order line -> test fails)	**runs them, reads them**
8 Security review	security-review agent/prompt: BOLA on both accounts, `canSubmit` for transfers, idempotency scope	**GATE 2 input**
9 Performance review	two row locks, no I/O in the transaction, plan check; baseline script for opposing transfers	reviews the numbers and their caveats
10 Documentation	sequence diagram, API example, BDD scenario, runbook line	checks it against the code
11 Release planning	checklist, rollback (`rollout undo`, no schema change), post-release verification	**signs the readiness checklist**

Observed value: the assistant is fastest at scaffolding and at proposing test cases; every real defect found in this project (undersized column, null-version `save()` trap, signed `UUID` ordering, scheduler throughput) was found by **running** the tests, which is why the Definition of Done insists on recorded runs.

Reviewing AI-generated code: five failure patterns with before/after

Assistants produce plausible, compiling, often *tested* code that is wrong in exactly the ways below. Each example shows a realistic generated snippet, the review reasoning and the corrected version (from this repository). Use together with `.github/prompts/review-ai-code.prompt.md`.

1. Race condition

Generated:

```
@Transactional
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
public void withdraw(UUID id, BigDecimal amount) {
    Account a = accounts.findById(id).orElseThrow();
    if (a.getBalance().compareTo(amount) >= 0) { a.setBalance(a.getBalance().subtract(amount)); }
}
```

Review: "What if two requests arrive together?" `@Transactional` gives atomicity of *this* transaction, not isolation from another reading the same row: both read 100 and both subtract 80. READ COMMITTED allows it; only a lock/version/constraint prevents it. The generated unit test (single thread) passes. **Fixed:** `accounts.lockById(id)` (`FOR UPDATE`), check and mutate under the lock, DB `CHECK (balance >= 0)`, concurrency test (`ConcurrencyScenarios.concurrentWithdrawalsNeverOverspend`) on H2 **and** PostgreSQL.

2. Missing authorization

Generated:

```
@GetMapping("/accounts/{id}") public Account get(@PathVariable UUID id) { return
repo.findById(id).orElseThrow(); }
```

Review: authentication is configured globally, so it *looks* secure. The question is *whose* account - any logged-in customer can read any id (OWASP API1). It also returns the entity (field leakage) and a 500 for unknown ids. **Fixed:** service call with `AccessPolicy.canViewAccount`, `404` + `ACCESS_DENIED` audit for foreign ids, DTO mapping; tests `SecurityIT.customerCannotReadOthersAccounts` and friends.

3. Incorrect monetary calculation

Generated:

```
double fee = amount.doubleValue() * 0.015;
BigDecimal total = amount.add(BigDecimal.valueOf(fee));
if (total.equals(new BigDecimal("100.00"))) { ... }
```

Review: `double` introduces binary rounding; `valueOf(double)` bakes it in; `equals` is scale-sensitive (`100.0` != `100.00`); no rounding policy. A test with round numbers passes. **Fixed:** `MoneyPolicy.percentageOf(amount, 150, currency)` (HALF_EVEN, currency scale), `compareTo`; tests `MoneyPolicyTest` (ties, equals vs compareTo, allocation to the cent).

4. Unsafe retry

Generated:

```
@Retryable(maxAttempts = 5, backoff = @Backoff(delay = 100))
public void sendPayout(PayoutRequest r) { restTemplate.postForObject(url, r, Void.class); }
```

Review: a timeout does not mean "not executed"; retrying a non-idempotent POST can pay five times. Constant backoff without jitter makes clients retry in lock-step; no cap on total time; no circuit breaker; failure after retries is reported as *failure* although the outcome is *unknown*. **Fixed:** retry only the idempotent `authorize(reference)`; `RetryExecutor` (exponential backoff, full jitter, bounded); breaker; result classified `Authorized | Declined | NotSent | Unknown` with reconciliation for `Unknown`; tests `RestClearingClientTest`, `RetryAndRateLimitTest`, `TransactionProcessorTest`.

5. Inadequate tests

Generated:

```
@Test void withdrawalWorks() {
    when(accountRepo.findById(id)).thenReturn(Optional.of(account));
    service.withdraw(id, BigDecimal.TEN);
    verify(accountRepo).save(account);        // asserts the call, not the outcome
}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Review: the test restates the implementation; it would still pass if the balance were not reduced, if the amount were negated, or if two threads raced. No negative case, no invariant, everything mocked. **Fixed:** assert observable outcomes (`balance == 750.00`, status `COMPLETED`), boundaries (`100.01`), authorization negatives, the invariant afterwards (`assertLedgerIntegrity()`), and a concurrency test; mutation check - remove the lock line and the concurrency test must fail.

Review procedure in one paragraph

For each hunk ask: *what if it runs twice, concurrently, or dies in the middle? who is allowed, on whose data? what are the units and rounding? what does a timeout mean? which production bug would this test catch?* Then run the tests yourself.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Architecture mind map

The mind map summarises the platform and the engineering concepts it demonstrates on one page.

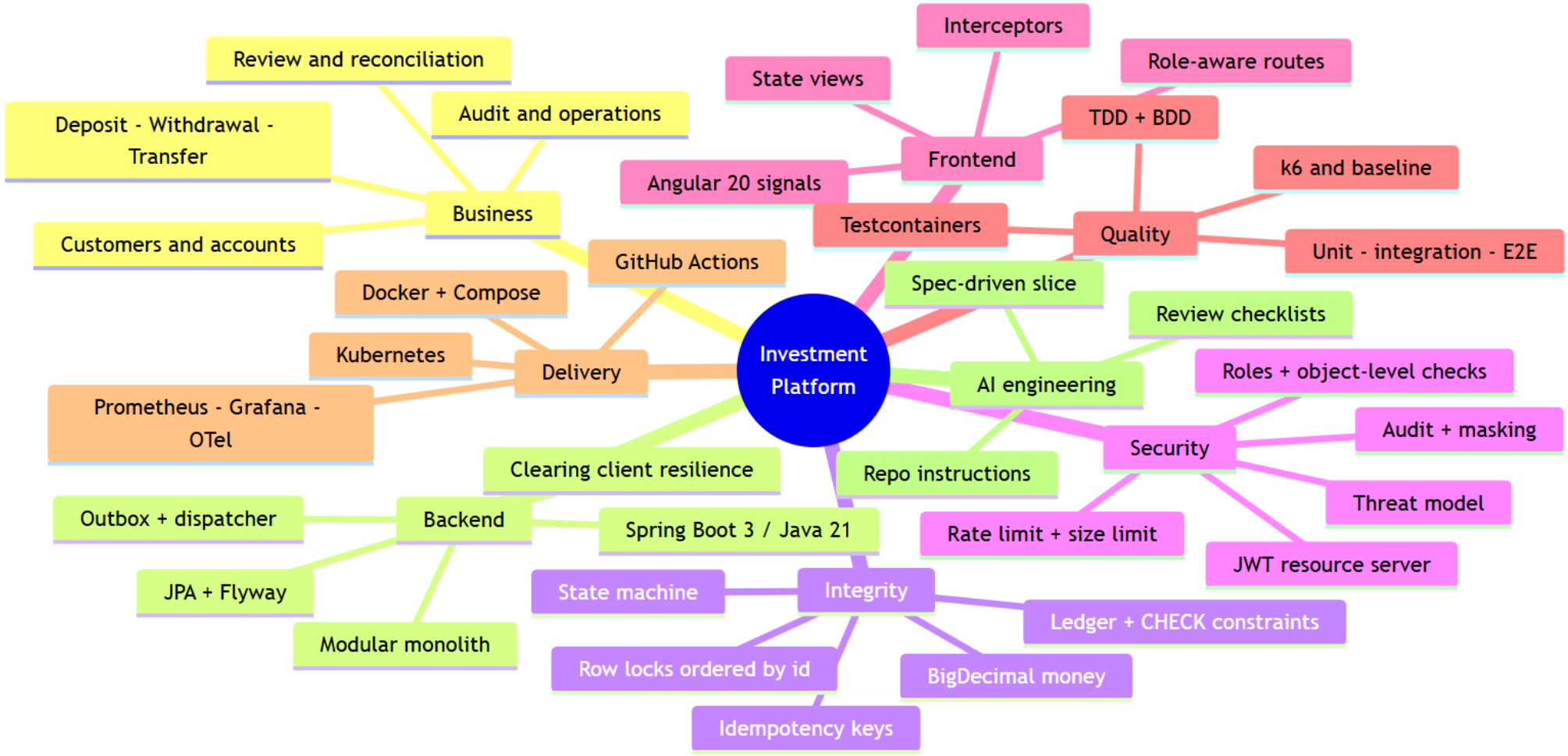


Figure 17. Platform mind map

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Troubleshooting scenarios

Realistic production problems for this platform, each with symptoms, investigation, resolution and the regression guard. (The five full *exercises* with code, trade-offs and regression tests are in `docs/leadership/troubleshooting-exercises.md`.)

#	Scenario	First clue	Investigation	Resolution	Guard
1	**Customer says money was taken twice**	two similar rows in history	compare `Idempotency-Key`s (audit `TRANSACTION_SUBMITTED` details), references, amounts; check `idempotency_records`	if same key -> a replay returned the original (no duplicate); if two keys -> the client created two logical requests; the ledger shows exactly what was posted	`IdempotencyIT`, e2e duplicate scenario
2	**Balance looks wrong**	support reports a mismatch	`GET /ops/accounts/{id}/ledger` vs `balance`; run `/ops/ledger-integrity`	integrity check empty -> the balance is the sum of postings; explain via ledger entries and reserved withdrawals (`funds_reserved`, `RECONCILIATION_REQUIRI`)	ledger integrity asserted in every concurrency test
3	**Withdrawal stuck in `RECONCILIATION_REQUIRED`**	alert after 15 min	attempts + audit trail; network reachability; look up the reference	resolve per runbook; if the network is down wait for the reconciler	`unknownOutcome*` ITs, chaos e2e
4	**Spike of 503 `LOCK_TIMEOUT`**	error-rate alert	blocking-locks query; which account; who holds the lock	find the long lock holder (external call inside a transaction? a stuck session?), kill it, fix the code path	bounded lock wait; `LedgerServiceTest` lock order
5	**Queue not draining after a deploy**	`platform_outbox_backlog` rising	dispatcher logs, `app.processing.dispatcher-enabled`, dead letters, DB latency	enable dispatcher / fix cause / requeue dead letters	`OutboxDeadLetterIT`, `OutboxDispatcherTest`
6	**Login works, every API call returns 401**	after an IdP change	token `iss` vs `OIDC_ISSUER_URI`, clock skew, JWKS reachability from the pod	align issuer/URL, fix NTP/egress	`SecurityIT` token validation tests
7	**Dashboard says "DEGRADED: outbox dead-lettered=3"**	health component	`GET /ops/outbox?status=DEAD` -> `last_error`	fix cause, requeue	alert `OutboxDeadLetters`
8	**Users report slowness only for one account**	p95 fine, one account slow	lock wait chain on that account; many parallel requests on a hot account	expected serialisation; if abusive -> rate limit/ suspend; if legitimate -> batching design	concurrency tests; ADR-005 consequences
9	**Pods restart every few minutes**	`OOMKilled`	heap-after-GC, RSS vs limit, heap dump	right-size limit or fix the leak	JVM heap alert

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

#	Scenario	First clue	Investigation	Resolution	Guard
10	**Flyway fails at startup in one environment**	`Validate failed: checksum mismatch`	someone edited an applied migration	restore the original file, add a *new* migration for the change	`validate-on-migrate`, review rule "never edit a released migration"

Performance and reliability investigation guide

General method: **observe -> hypothesise -> measure -> change one thing -> re-measure**. Start from the user-visible symptom (latency, errors), find the saturated resource (CPU, memory, threads, connections, locks, I/O), then the code or query responsible. Never tune without a baseline; never claim an improvement without a re-measurement (the repository's own example: the dispatcher change moved drain rate from ~23 to ~34 transactions/s on the *same* machine, measured in `docs/validation/perf-baseline*.json`).

High JVM memory usage

1. Distinguish *footprint* from *leak*: heap-after-GC trend (`jvm_memory_used_bytes{area="heap"}` low points) rising across hours = leak; flat but high = under-sized.
2. Container view: `process_resident_memory` vs. the pod limit - heap is only part of RSS (metaspace, thread stacks, direct buffers, code cache). `MaxRAMPercentage=75` leaves 25 % for non-heap.
3. Evidence: heap dump (`jcmd <pid> GC.heap_dump /tmp/h.hprof`) -> Eclipse MAT "Leak suspects"/ dominator tree. Typical findings in this kind of service: unbounded query results (`findAll()` without paging), caches without eviction, per-request objects held by static fields, large audit/log strings.
4. Fix, then verify by repeating the load while watching the heap-after-GC line.

Excessive garbage collection

- Symptoms: `jvm_gc_pause_seconds_sum` rate high, latency spikes aligned with pauses, CPU in GC threads.
- Read the GC log (`-Xlog:gc*`) or the metric labels: frequent young collections = high allocation rate; Full GC / humongous allocations = heap too small or large arrays; "to-space exhausted" = evacuation failure.
- Order of action: reduce allocation (avoid per-request large collections, streaming instead of materialising, paging), right-size heap and pod, *then* consider collector flags. G1 is the default and is right for most services; ZGC/Shenandoah only with evidence of pause-bound latency SLOs.

Thread starvation

- Symptoms: requests queue, CPU low, `jvm_threads_live` at the Tomcat max (`server.tomcat.threads.max` = 100), `http.server.requests` rising with flat throughput.
- Take 2-3 thread dumps 10 s apart (`jcmd <pid> Thread.print`); look for many threads `WAITING` on the same monitor, on a Hikari `getConnection`, or inside a remote call.
- Causes: pool exhaustion (threads wait for connections), a slow dependency without a timeout (every request thread blocks), synchronized sections, blocking calls on shared pools.
- Fixes: bounded timeouts everywhere (done: connect 500 ms / read 2 s / pool 3 s / lock 5 s), bulkheads (the asynchronous processor isolates the external call from request threads), right-size pools, virtual threads (`spring.threads.virtual.enabled`) only after pool limits are understood - they remove thread scarcity but not connection scarcity.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Slow SQL

1. Find it: `pg_stat_statements` (mean/total time, calls), slow-query log (`log_min_duration_statement`), traces. 2. `EXPLAIN (ANALYZE, BUFFERS)` on the exact statement with real parameters. 3. Look for: sequential scans on large tables, `Rows Removed by Filter` >> rows returned, sorts spilling to disk, nested loops with huge outer sets, stale statistics (`ANALYZE`), bloat.
2. Fixes in order: correct predicate/index (composite order matching the filter then the sort), keyset pagination instead of deep `OFFSET`, split `OR` across indexes with `UNION ALL`, avoid N+1 (`default_batch_fetch_size`, fetch joins), cache only after the query is right. The captured plans in the database chapter show the OR-predicate trade-off concretely.

Connection-pool exhaustion

See the runbook section; investigative questions: is `active == max` with `pending > 0`? what are the active sessions doing (`pg_stat_activity`)? are transactions held open across slow work (the anti-pattern ADR-003 forbids)? is the pool size justified (concurrent DB capacity, not request volume)? Enable `leak-detection-threshold`; track `hikaricp_connections_acquire_seconds` p95 as an early warning (it rises before errors appear).

Lock contention

- Symptoms: latency that grows with concurrency on the *same* account, `LOCK_TIMEOUT` 503s, long `Lock` waits in `pg_stat_activity`.
- Find the head of the chain with the blocking-locks query (database chapter); examine what the lock holder is doing - a lock should be held for milliseconds. Long holders = work done inside the transaction that should not be (external call, large computation, user think time).
- Remedies: shorten the critical section, move non-essential work (audit enrichment, notifications) out, reduce hot-row contention (per-account serial processing is inherent; a hot "house" account needs a different design such as sharded counters or batching), keep lock order consistent, keep lock waits bounded.

Elevated API latency

Top-down: (1) which endpoint (`http_server_requests` by `uri`); (2) percentile vs. average (a fat tail = queuing or GC; uniformly slow = work per request); (3) trace: where is the time - DB, external call, application; (4) saturation signals - pool pending, threads, CPU throttling (a CPU **limit** causes throttling; the manifests set none for that reason), GC; (5) recent change / traffic shape; (6) fix and re-measure. USE (utilisation, saturation, errors) for resources, RED (rate, errors, duration) for endpoints.

Database outage

Expected behaviour: connections fail within 3 s, readiness fails (traffic drains from the pod), requests return 5xx/503 quickly, **no data is lost** (the outbox lives in the database; submitted-but-unprocessed work resumes). On recovery Hikari reconnects, the dispatcher resumes, stuck `PROCESSING` rows move to reconciliation after 5 minutes. Procedure: confirm at the provider (RDS events), fail over (Multi-AZ is automatic), watch connection storms on recovery (jittered reconnects; the pool is capped), verify ledger integrity and the outbox backlog. Practise with a game day (stop PostgreSQL in the test environment).

Partial service failures

The clearing network is the partial-failure case: **slow** (latency up, circuit may open), **flaky** (retries with jitter absorb it), **down** (breaker open -> fast `CLEARING_UNAVAILABLE`), **lying by omission** (timeouts -> `RECONCILIATION_REQUIRED`). The chaos end-to-end scenario (`e2e-api.mjs --with-chaos`) exercises all four against the simulator. Design questions to ask of any dependency: what is its timeout, is the call idempotent, what does the caller do on *unknown*, is there a fast-fail path, and

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

who gets paged?

Measured results versus targets

Only `docs/validation/perf-baseline*.json` contains measurements (developer machine, H2, not a benchmark). `TARGETS` in `performance/k6/lib.js` and the alert thresholds are illustrative starting points to be tuned from real production-like measurements.

Production runbooks

Every alert in `observability/prometheus/alert-rules.yml` links to a section here (a validation check fails if one is missing). General first steps for **any** incident: (1) acknowledge, note the time; (2) open the Grafana *Platform overview* dashboard and `GET /api/v1/ops/health`; (3) find a representative failing request's **correlation id** and follow it through logs, trace and audit trail; (4) decide *stop the bleeding* (scale, roll back, disable a feature, shed load) before *find the root cause*; (5) communicate; (6) write the post-incident review (template in this folder).

Platform down

Alert `PlatformServiceDown`*, page.* **Symptoms:** Prometheus cannot scrape; readiness fails; users get 502/503 from the ingress.

1. `kubectl -n investment-platform get pods` / `describe pod` - `CrashLoopBackOff`, `OOMKilled`, `ImagePullBackOff`, failing probes?
2. `kubectl logs --previous` - startup failure causes seen so far: DB unreachable or wrong credentials; **Flyway validation failure** (a migration was edited after release); no `OIDC_ISSUER_URI` (the service fails closed on purpose).
3. A recent deploy? `kubectl rollout history` -> `kubectl rollout undo deployment/platform-service` (the previous version works against the current schema by the expand/contract rule).
4. Database down? see *Database outage* in `performance-troubleshooting.md`.
5. Verify: readiness green, `/ops/health` UP, outbox backlog drains, error rate back to baseline.

Elevated 5xx

Alert `HighServerErrorRate`*, page.* 1. Dashboard: which `uri` and `status`? 2. Logs filtered by `level=ERROR` and the correlation id from a failing call (the 500 body contains only the id). 3. Common causes: database connectivity/pool exhaustion (`ConnectionPoolSaturated`), lock timeouts (503 `LOCK_TIMEOUT`), a bad deploy, the clearing network (should surface as FAILED/RECONCILIATION states, *not* 5xx - if it does, that is a bug). 4. Mitigate: roll back the last release; scale out if CPU-bound; check the DB. 5. Add a regression test for the cause.

Slow API

Alert `HighApiLatencyP95`*, ticket.* Follow the *Elevated API latency* flow in `performance-troubleshooting.md`: which endpoint (histogram by `uri`), DB time vs. app time (traces), slow SQL (`pg_stat_statements`), lock waits, pool wait (`hikaricp_connections_acquire_seconds`), GC pauses, CPU throttling, noisy neighbour.

Reconciliation backlog

Alert `ReconciliationBacklog`*, ticket (open records for 15 min).* These are withdrawals whose external outcome is unknown; **customer funds are reserved, not lost**.

1. Operations console -> *Investigations* -> *Reconciliation* lists the records with the reason (`CLEARING_OUTCOME_UNKNOWN`, `PROCESSING_TIMEOUT`).
2. Is the clearing network reachable? `/ops/health` -> `clearing-network`. If the network is down the reconciler cannot ask; wait for recovery (it retries every 15 s).
3.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

If it is up and records persist: look up the reference in the network's own portal/tools. *Executed* -> resolve **"the network executed it"** (transaction completes). *Not executed* -> resolve **"not executed"** (funds returned).

4. You cannot resolve a transaction you submitted yourself (maker-checker): ask a colleague.
5. Record the decision in the resolution note; it is written to the audit trail.

Dead-letter outbox events

Alert `OutboxDeadLetters`*, page.* Validated transactions are **not being processed** for the affected ids.

1. `GET /api/v1/ops/outbox?status=DEAD` (admin) shows `last_error` per event.
2. Fix the cause first (bug, data problem, dependency). Requeueing before fixing just dead-letters it again.
3. `POST /api/v1/ops/outbox/{id}/requeue`; watch `platform_outbox_backlog`. The processor is idempotent, so requeueing an event that was actually handled is harmless.
4. If the transaction itself is poisoned (bad data), cancel it (`/transactions/{id}/cancel` if still cancellable) or fail it through reconciliation, and file a defect.

Outbox backlog

Alert `OutboxBacklogGrowing`*, page (> 100 pending for 5 min).* Work is arriving faster than it is processed (or the dispatcher is not running).

1. Is `app.processing.dispatcher-enabled` true in every pod? Are scheduled jobs failing (`ERROR Scheduled job 'outbox dispatch' failed`)?
2. Processing time per transaction (`platform_transactions_end_to_end_seconds`), DB latency, lock contention on a hot account (one popular account serialises its own transactions).
3. Scale out replicas (safe: lease + optimistic locking) - or run a dedicated worker deployment (ADR-001 migration path).
4. Last resort: `POST /ops/processing/drain` (admin) runs one synchronous pass.

Transaction failures

Alert `HighTransactionFailureRate`*, page (> 10 % FAILED for 10 min).* Break down by `failureCode` in the dashboard/`/ops/summary` and the Investigations page: `INSUFFICIENT_FUNDS` at processing time = normal contention; `CLEARING_DECLINED` = network policy/risk; `CLEARING_UNAVAILABLE` = network or circuit breaker (see next section); `*_NOT_ACTIVE` = account/customer status changes; anything else = defect. Confirm refunds happened (`/ops/ledger-integrity` must be empty).

Clearing network outage

Alert `ClearingCircuitOpen`*, page.* **Expected behaviour:** requests fail fast as `CLEARING_UNAVAILABLE` with funds returned; those already in flight become `RECONCILIATION_REQUIRED` (funds reserved) and resolve automatically after recovery.

1. Confirm at the provider's status page; check egress/DNS/VPN. 2. Do **not** restart the platform to "reset" the breaker - it half-opens by itself after 10 s and closes on success.
2. Communicate to support: withdrawals are failing safely, no customer money is lost, deposits and transfers are unaffected. 4. After recovery verify the reconciliation backlog drains and the ledger integrity check is empty.

Connection pool exhaustion

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Alert `ConnectionPoolSaturated`*, page (threads waiting for a connection).* 1. `hikaricp_connections_active == max`, `pending > 0`, acquire time rising. 2. **Why are connections held?** slow queries or long transactions (look at `pg_stat_activity` for `state = 'active'`/`idle in transaction`), lock waits (see the blocking-locks query in the database chapter), a connection leak (`leak-detection-threshold` logs a stack trace after 30 s), or genuine load. 3. Mitigate: kill the blocking session if safe; scale replicas *only if the database has headroom* (each replica adds `DB_POOL_MAX` connections); never raise the pool above what PostgreSQL can run concurrently. 4. Remember: an external call must never run inside a transaction (ADR-003) - if the pool saturates when the clearing network is slow, that rule has been broken.

High memory

Alert `JvmHeapHigh`*, ticket.* 1. Is it growth (leak) or a high steady state? Look at the heap after GC (`jvm_memory_used_bytes` low points). 2. Heap dump on OOM is enabled (`-XX:+HeapDumpOnOutOfMemoryError`, `/tmp`); for a live pod use `jcmd <pid> GC.heap_dump` and open in Eclipse MAT: look for retained sets (large `ArrayList` of entities from an unbounded query, caches, `ThreadLocal`s). 3. Typical causes here: an unpaged query, a huge page size, an unbounded map (the rate limiter is bounded), large JSON in logs. 4. Mitigate with a rolling restart while the cause is fixed.

Excessive GC

Alert `LongGcPauses`*, ticket.* 1. `jvm_gc_pause_seconds` by `cause`/`action`; G1 young pauses are normal, frequent **Full GC** or "to-space exhausted" are not. 2. Allocation rate vs. heap size (the pod memory limit of 768 MiB with `MaxRAMPercentage=75` leaves little room: raise the limit before tuning flags). 3. Check for allocation hot spots in a flight-recorder profile (`jcmd <pid> JFR.start duration=60s`). 4. Tune last (`-XX:MaxGCPauseMillis`), after reducing allocation.

Operational how-tos

Task	How
Rolling restart	`kubectl -n investment-platform rollout restart deployment/platform-service`
Roll back	`kubectl -n investment-platform rollout undo deployment/platform-service`
Scale	`kubectl scale deployment/platform-service --replicas=4` (the HPA will take over again)
See who did what	`GET /api/v1/ops/audit-events?entityId=<id>`; correlation id search in logs
Verify money integrity	`GET /api/v1/ops/ledger-integrity` must return `[]`
Suspend a misbehaving account	`POST /api/v1/accounts/{id}/suspend` with a reason (audited)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Architecture decisions and trade-offs

Thirteen Architecture Decision Records. Each states the context, the decision, the alternatives that were rejected and the consequences.

ADR-001: Modular monolith instead of microservices

- **Status:** accepted * **Deciders:** architecture lead * **Date:** project start

Context

The platform needs customer/account management, transaction processing, audit and reporting. The brief asks for separate deployable services only where the separation gives a meaningful benefit and warns against distributed transactions "merely to demonstrate microservices". The core invariants (balance == ledger, a transfer is all-or-nothing, idempotent submission) need a single transaction boundary.

Decision

Deliver one Spring Boot deployable (`platform-service`) organised as modules with enforced one-directional dependencies, plus one small separate service (`clearing-simulator`) because it models a genuinely separate external system.

Alternatives considered

1. **Microservices per module** (customer, account, transaction, audit) - atomic transfers would require a saga/2PC; invariants become eventually consistent; 4x the pipelines, dashboards, contracts and failure modes for a one-team scope. Rejected.
2. **Monolith + separate worker deployment** - cheap and compatible with the design (`app.processing.dispatcher-enabled`); kept as the *first* extraction step.
3. **Event-sourced core** - powerful audit but heavy; the append-only ledger and audit trail give most of the benefit. Rejected for scope.

Consequences

- one local ACID transaction protects every money invariant; simple operations; fast tests; trivial local run.
- one deployable means one blast radius (mitigated by rolling updates, probes, async worker, and the option to run a dedicated worker pool);
- scaling is all-or-nothing (the database is the real bottleneck anyway).

Migration path

Dispatcher/processor as a separate deployment of the same artefact -> audit/ops read models behind events -> `customer` -> only then, with measurements, anything touching balances. The `AccountClosureGuard` port already keeps `account` independent of `transaction` persistence.

ADR-002: PostgreSQL as the primary relational database

- **Status:** accepted

Context

Financial state needs strong consistency, constraints and predictable locking. The assignment allows PostgreSQL or MySQL and wants the choice justified, with a discussion of Oracle for enterprise use.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Decision

PostgreSQL 16 in deployment; H2 (PostgreSQL mode) for zero-install local runs and fast tests; Flyway for migrations with a `{vendor}` folder for database-specific DDL.

Why PostgreSQL over MySQL

Partial and expression indexes (one OPEN reconciliation per transaction; a tiny "attention" index), transactional DDL (atomic migrations), richer `CHECK` constraints, `SELECT ... FOR UPDATE SKIP LOCKED`, `pg_stat_statements`/`EXPLAIN (ANALYZE, BUFFERS)` ergonomics, MVCC without gap-lock surprises at READ COMMITTED.

Alternatives

- **MySQL/InnoDB** - fine technically; weaker partial-index story; gap locks at REPEATABLE READ are a common source of unexpected deadlocks.
- **Oracle** - common in enterprises; the design ports (see database chapter: `'' IS NULL`, no partial indexes, DDL auto-commit, `FETCH FIRST`). Not added as a second running database - no implementation need.
- **MongoDB** - unsuited for the ledger; fits read models/projections (documented, not implemented).

Consequences

- constraints are a second line of defence (negative balance, double posting, duplicate keys are *impossible*, not just unlikely);
- H2 cannot express every PostgreSQL feature, so PostgreSQL-specific behaviour is tested on a real PostgreSQL (embedded locally, Testcontainers in CI);
- two vendor folders must be kept in step (a test asserts the PostgreSQL one ran against PostgreSQL).

ADR-003: Explicit transaction boundaries; the external call is never inside a database transaction

- **Status:** accepted

Context

A withdrawal calls an external clearing network that can be slow, fail, or succeed without telling us. A database transaction cannot include that call. Holding row locks and a pooled connection across a network call risks pool exhaustion and cascading failure.

Decision

1. All changes to *our* data happen in short local ACID transactions with explicit boundaries (`TransactionTemplate`), `MANDATORY` propagation on code that must never run without one (`LedgerService`, `AuditService.record`, `OutboxService.enqueue`).
2. Processing is split: **Tx1** `VALIDATED -> PROCESSING` + (deposit/transfer: apply ledger -> `COMPLETED`; withdrawal: reserve funds); **external call with no transaction open**; **Tx2** record the outcome.
3. The result of the external call is classified `Authorized / Declined / NotSent / Unknown`; `Unknown` never refunds and never completes - it goes to reconciliation (funds stay reserved).
4. Business failures inside Tx1 roll it back completely and are then recorded in a fresh transaction; *technical* failures (lock timeout, DB down) propagate so the outbox retries.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Alternatives

- `@Transactional` on the whole processor including the HTTP call - simple, dangerous (connection/lock hold times = external latency).
- Saga with a broker and compensations for every step - overkill when only one step is external.
- Two-phase commit with the network - not offered by the counter-party and not available over HTTP.

Consequences

- locks are held for milliseconds; an outage of the network cannot exhaust the pool; every intermediate state is explicit and queryable;
- more states to reason about and test (`RECONCILIATION_REQUIRED`, reserved funds) - covered by unit, integration and chaos tests;
- a crash between Tx1 and Tx2 leaves `PROCESSING` + reserved funds, handled by the stuck-transaction recovery job.

ADR-004: Idempotency keys enforced by a database unique constraint

- **Status:** accepted

Context

Clients retry (timeouts, double clicks, flaky networks); gateways replay. A money movement must be applied at most once per logical request.

Decision

`POST /transactions` requires an `Idempotency-Key` header. The record `(principal, key) -> transaction, SHA-256(request)` lives in `idempotency_records` with `UNIQUE(principal, idempotency_key)` and is inserted in the **same transaction** as the transaction row, the outbox event and the audit rows. A fast read handles the common replay; a unique violation (concurrent twin) rolls the whole attempt back and the loser replays the winner's result. Same key + different payload -> `422 IDEMPOTENCY_KEY_REUSED`. Records are kept for 7 days.

Alternatives

- In-memory/Redis key cache - not atomic with the database write; loses keys on restart; adds a system.
- Natural-key deduplication (same amount/accounts within N seconds) - false positives (two legitimate equal payments); kept only as a possible *review* signal.
- Server-generated ids with client polling - does not solve duplicate creation.

Consequences

- correctness does not depend on the pre-check - the constraint decides races, across any number of instances; replay returns the identical outcome (including rejections);
- one extra index and a purge job; the retention window defines how long a client may safely retry;
- the key namespace is per principal (a leaked key cannot replay someone else's request).

ADR-005: Pessimistic row locks (ordered) for balances, optimistic versioning elsewhere

- **Status:** accepted

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Context

Balances are hot rows; concurrent withdrawals must not overspend; opposing transfers must not deadlock; other entities see rare conflicts.

Decision

- Balance changes: `SELECT ... FOR UPDATE` on the account rows (5 s bounded wait), re-check status/funds/daily limit **under the lock**, then mutate; multi-account operations lock in **ascending id order**.
- Everything else: `@Version` optimistic locking (cancel vs process, double approval, double resolution, profile edits) -> `409 CONCURRENT_MODIFICATION`.
- Database constraints as the last line (`CHECK balance >= 0`, unique ledger postings).
- Default isolation (READ COMMITTED); no blanket SERIALIZABLE.

Alternatives

Option	Verdict
Optimistic locking on balances with retries	retry storms under contention; starvation; more complex client-visible failures
`UPDATE accounts SET balance = balance - :x WHERE id = :id AND balance - :x >= :min` (atomic conditional update)	a good alternative for the single-row case; chosen *against* because we also need the ledger entry, the daily-limit sum and status checks in the same critical section, and the lock model generalises to transfers
SERIALIZABLE isolation	correct but turns contention into aborts and retries everywhere
Single-writer queue per account	highest throughput per account, but needs a partitioned broker - overkill here

Consequences

- simple reasoning ("hold the row, check, change"); invariants proven by concurrency tests on H2 and on real PostgreSQL;
- lock waits are possible (bounded; become retryable 503, never a pool pile-up); a future code path that locks in a different order could deadlock -> all mutations go through `LedgerService` and the order is unit-tested;
- throughput per *account* is serial by nature (per-account TPS bounded by the transaction time, tens of ms).

ADR-006: Transactional outbox with an in-process dispatcher; no message broker, no Redis (yet)

- **Status:** accepted

Context

Submission must be fast and reliable, processing asynchronous and retryable, and the system must not lose or duplicate work. The brief mentions RabbitMQ/Kafka/Redis "where justified".

Decision

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Write an `outbox_events` row in the same transaction as the state change; a scheduled `OutboxDispatcher` claims events by **lease** (bump `attempts`, push `available_at`, optimistic `@Version`), runs the processor, marks `DISPATCHED`, retries with exponential backoff (`2 s * 2^n`, capped at 5 min) and moves poison events to `DEAD` (the dead-letter state) after 5 attempts with an audit event and an alert. Delivery is at-least-once; the consumer is idempotent. No broker and no Redis are deployed.

Why not a broker or Redis now

- One module consumes the events; a broker adds an operational system, a second failure domain, and dual-write problems that the outbox exists to avoid.
- Redis would only back the rate limiter (and a cache we do not need). A shared limiter becomes necessary when running many instances behind no gateway.

Alternatives

Kafka/RabbitMQ with the outbox publishing to it (needed for multiple independent consumers, replay, fan-out); `LISTEN/NOTIFY`; `SKIP LOCKED` polling workers; Spring Integration.

Consequences

- atomic with the business change, no extra infrastructure, simple local run, DLQ semantics visible in SQL and the ops UI;
- polling latency (500 ms default; the work-conserving drain loop removes the gap under load), database load from polling (an indexed query);
- when a second consumer, replay or cross-service fan-out appears: add a relay that publishes outbox rows to Kafka/RabbitMQ; producers do not change.

ADR-007: URI versioning, strict input, committed OpenAPI contract

- **Status:** accepted

Context

Several internal applications will consume the API and cannot all upgrade together.

Decision

Version in the URI (`/api/v1`); within a major version only non-breaking changes (add optional fields/endpoints/error codes). Money as strings, scale per currency. Strict input (`FAIL_ON_UNKNOWN_PROPERTIES`) because the API is the system of record; clients are tolerant readers. The OpenAPI document is generated from the running app, committed (`specs/openapi.json`) and compared in `ApiContractIT` and in CI (`git diff --exit-code`).

Alternatives

Header / media-type versioning (cleaner URIs, harder to route, test and see in logs); date-based versions (Stripe style: powerful, needs per-account pinning machinery); GraphQL (different trade-offs, not requested).

Consequences

- a breaking change is a visible, reviewed diff; versions run side by side; easy cache/gateway routing;
- strict input means a client that sends an extra field gets a 400 (deliberate; documented);
- deprecation requires process (headers, usage metrics, sunset date) - written in the API chapter.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

ADR-008: Stateless JWT resource server with two-layer authorization

- **Status:** accepted

Context

Customers and staff use the same API with very different rights; the most likely serious flaw in such systems is broken object-level authorization.

Decision

Spring Security as an OAuth2 resource server validating JWT (signature, expiry, issuer) from an external OIDC provider. Roles from a claim map to authorities; unknown roles are ignored. Authorization in two layers: `@PreAuthorize` for *what a role may do* and a pure `AccessPolicy` for *which objects the caller may touch* (customer scope via the `customer_id` claim, transfers inside one customer, maker-checker). Foreign objects answer `404` and write an `ACCESS_DENIED` audit event. Stateless sessions, CSRF disabled **because** tokens travel in a header (re-enable if cookies are ever used), CORS allow-list, secure headers, rate limit and body limit, deny-by-default routing. A local HS256 issuer exists for demos, refuses to start with a production profile, and is not part of the Kubernetes configuration.

Alternatives

Opaque tokens + introspection (revocation friendly, extra hop per request); session cookies + CSRF protection (fine for a BFF); a gateway-only authorization (misses object-level rules that need domain data); attribute-based access control engine (OPA/Cedar) - attractive at larger scale.

Consequences

- authorization rules are plain functions: exhaustively unit-tested and obvious to review; the same rule set applies to REST, search scoping and reads;
- the SPA keeps tokens in memory (reload = sign in again) pending an authorization-code + PKCE / BFF design;
- revocation relies on short token lifetimes (30 min) until the IdP's refresh-token rotation is integrated.

ADR-009: GitHub Actions pipeline with security gates; container images promoted unchanged; Kubernetes rolling deployments

- **Status:** accepted

Context

Releases must be frequent, reversible and safe, and a financial application must not ship known-vulnerable dependencies or leaked secrets.

Decision

One pipeline: build/unit -> integration (real PostgreSQL via Testcontainers) -> frontend lint/test/build -> SpotBugs + CodeQL -> OWASP Dependency-Check + `npm audit` + gitleaks -> config validation -> image build -> Trivy scan -> versioned publish (`1.0.<run>`, `sha-<short>`) -> deploy to test with kustomize -> smoke tests -> automatic `rollout undo` on failure. Build once, promote the *same* image; environment differences live in kustomize overlays/ConfigMaps. Rolling updates with `maxUnavailable: 0`, readiness gating, `preStop` + graceful shutdown. Database changes follow expand/contract so the previous release always works against the new schema (application rollback

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

never needs a schema rollback).

Alternatives

Jenkins (more control, more to run); GitLab CI (equivalent); blue/green or canary with a service mesh/Argo Rollouts (stronger, heavier - noted as the next step for high-risk releases); deploying with Helm (templating power vs. plain-YAML readability).

Consequences

- gates are automated and visible; a failed smoke test self-heals; no per-environment rebuilds;
- the workflow cannot be fully validated without a runner (documented honestly); pin third-party actions by SHA before production use;
- expand/contract needs discipline and an extra release for destructive changes.

ADR-010: Money as BigDecimal / NUMERIC(19,4) / JSON string, rejected (not rounded) excess precision

- **Status:** accepted

Decision

`BigDecimal` in Java, `NUMERIC(19,4)` in the database, JSON strings in the API, string-only handling in the SPA. Input with more decimals than the currency's minor units is rejected (`AMOUNT_PRECISION`) instead of being rounded; unavoidable rounding (percentages, allocation) uses `HALF_EVEN`; comparisons use `compareTo`; allocation preserves the total to the cent. Single place of policy: `MoneyPolicy`.

Alternatives

- **Minor units as `long`** (cents): fast and exact, but awkward for fractional rates and currencies with different exponents; kept as a valid alternative.
- **JSON numbers**: lose precision in JavaScript above 2^53 and invite float arithmetic in clients.
- **Silent rounding of input**: hides client bugs and creates reconciliation differences.

Consequences

- exactness end to end; DB `CHECK`s catch any bypass; tests prove the policy (`MoneyPolicyTest`);
- clients must treat amounts as decimal strings; sums in clients need a decimal library (the SPA does no arithmetic).

ADR-011: Reserve funds first; classify external outcomes; reconcile the unknown

- **Status:** accepted

Context

After a timeout we cannot know whether the clearing network executed a payout. Refunding would risk a double payment; completing would risk charging for nothing.

Decision

1. Debit (reserve) the funds in Tx1 *before* calling the network - the customer cannot spend them twice and the platform cannot over-pay.
2. Treat the call result as `Authorized | Declined | NotSent | Unknown`. Only definite results change money state (`Declined`/`NotSent` -> reversal credit + `FAILED`; `Authorized` -> `COMPLETED`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

3. `Unknown` -> `RECONCILIATION_REQUIRED` + an OPEN `reconciliation_records` row; the funds stay reserved.
4. A scheduled reconciler looks the reference up at the network (idempotent read): authorized -> completed; declined, or not found after a grace period -> refund + `FAILED`.
5. Anything still unknown after 15 minutes raises an alert; operators resolve manually with maker-checker; every step is audited. A stuck `PROCESSING` row (crash) is moved to reconciliation by a maintenance job.

Alternatives

Check funds then call then debit (race window, can overspend); assume failure on timeout (double payout risk); assume success (phantom payouts); distributed transaction (not available).

Consequences

- the platform is never wrong silently: money is either paid out exactly once or returned in full, and uncertainty is a visible, alertable state;
- customers see their balance reduced while a payout is unresolved (the correct, conservative behaviour); needs support runbooks (written);
- requires the network to deduplicate on our `reference` and to offer a lookup - a contract assumption recorded in the interface documentation.

ADR-012: Access token held in memory only in the SPA

- **Status:** accepted (with a documented production follow-up)

Context

Browser storage is readable by any script that runs in the page; a single XSS would exfiltrate a token from `localStorage`/`sessionStorage`.

Decision

Keep the access token in a signal inside `AuthService` (memory only); attach it only to `/api/` requests; strict CSP, no `innerHTML`, Angular sanitisation left on; sign out on 401.

Alternatives

`localStorage`/`sessionStorage` (convenient, XSS-readable); HttpOnly cookie via a backend-for-frontend (best practice for browsers, needs a BFF and CSRF protection); service-worker token handling.

Consequences

- nothing to steal from storage; the token dies with the tab; trivially testable (a unit test asserts storage stays empty);
- a page reload needs a new sign-in (acceptable for a back-office tool; the production answer is authorization-code + PKCE with silent renewal or a BFF).

ADR-013: Two frontends, one API contract

Status: accepted

Context

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

The platform was built with an Angular 20 console. Teams and candidates also work with React, and the same REST API should be usable from either without changing the backend. The risk is drift: two frontends that slowly disagree about the API, about security behaviour, or about how a user-visible rule is worded.

Decision

Add a React 19 console (`frontend-react/`) next to the Angular one and keep them aligned by construction:

- the API contract types (`core/models.ts`) and the idempotency-key class are the same source in both projects;
- both implement the same security behaviours (in-memory token, bearer on `/api/` only, correlation id, 401 ends the session, validated `returnUrl`);
- one browser end-to-end suite (`tests/e2e/ui-e2e.mjs --flavor angular|react`) drives both against the real backend, so any behavioural difference fails a test;
- the backend is unchanged, and so is its CORS allow-list: each edition is served same-origin behind a proxy that preserves the `Host` header (nginx in the images, `scripts/serve-frontend.mjs` and the Vite proxy locally).

Alternatives considered

- **One frontend only.** Simplest to maintain; does not serve a team that works in React.
- **Shared UI library or micro-frontends.** Overkill for ten screens and would couple the editions through a build system.
- **Generate both clients from `openapi.json`.** The right next step once the API grows; today the hand-written types are 180 lines and are checked by the end-to-end suite and the backend's `ApiContractIT`.
- **Weaken CORS (a wildcard) or add the new origin to the allow-list to make the second UI work.** Rejected: the UI is same-origin behind the proxy. The first browser run answered 403 because a proxy rewrote the `Host` header; that was fixed instead (threat T13 stays untouched).

Consequences

- Every UI change is made twice. The shared end-to-end suite and the identical wording keep the cost visible.
- A new endpoint or field needs the type changed in two places until the types are generated.
- The React edition has fewer dependencies (3 runtime) and a smaller initial bundle; the Angular edition has a batteries-included component library. Neither is "better": the choice is organisational.
- Deployment gets a second image, never a second backend.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Known limitations and future enhancements

What was executed, what was not (summary; details in the validation report)

Area	Status
Backend compile, unit tests, integration tests (H2)	**executed**
Integration and concurrency tests on a real PostgreSQL (embedded PostgreSQL 14, no Docker)	**executed**
Angular lint, unit tests, production build	**executed**
React edition: type-check, lint, 90 unit/component tests, production build, browser end-to-end (19/19)	**executed**
Live API end-to-end (incl. simulator, chaos) and browser end-to-end	**executed**
Local load baseline (Node)	**executed** (developer machine; not a benchmark)
k6 scripts	written, **not executed** (k6 not installed)
Docker images, Docker Compose stack, Prometheus/ Grafana/Jaeger	written, **not executed** (no Docker) - structure validated statically
Kubernetes manifests	written, **not applied to a cluster** - validated statically (and `kubeconform`/`hadolint` in CI)
GitHub Actions workflow	written, **not run on a runner**
Testcontainers PostgreSQL test	written, **skipped** locally (needs Docker)
Cloud deployment (AWS)	**conceptual** only
Oracle and MongoDB	**conceptual** only
Multi-agent AI definitions	**definitions only**, never executed

Functional limitations

- Single-customer transfers only; no fees, interest, FX conversion, scheduled/recurring transfers, statements, or settlement cycles.
- Withdrawals model *one* external step (a payout authorization); real flows have more (netting, settlement files, returns/chargebacks).
- The clearing-network contract (idempotent on `reference`, supports lookup) is an **assumption** that must be confirmed with a real provider; the simulator is deterministic, not realistic.
- No KYC/AML/sanctions screening, no regulatory reporting, no multi-tenancy across legal entities, no data-subject-request workflow.
- Four-eyes control applies to approvals and reconciliation only.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Technical limitations

- **In-process outbox dispatcher:** polling, single worker loop per instance, in-database queue. Measured locally at ~34 transactions/s drain rate (dev machine); a real deployment needs parallel workers (`SKIP LOCKED`) or a broker relay.
- **Per-account serialisation** is inherent to the row-lock design; a hot "house" account needs a different structure (sharded sub-accounts or batching).
- **History query** uses `source = :id OR target = :id`; adequate with the current indexes (captured plans) but the scalable form is `UNION ALL`/participants table and keyset pagination.
- **Rate limiting** is per instance and in memory; production needs a shared store or gateway.
- **Local token issuer** (HS256, demo users) exists only for development and tests; the SPA keeps the token in memory so a reload requires sign-in.
- **Embedded PostgreSQL is version 14**; Compose/CI use 16. Behaviour relied on (row locks, partial indexes, triggers) is stable across both but only 14 was executed.
- H2 cannot express PostgreSQL's partial unique index; the single-open-reconciliation rule is therefore also guarded in code.
- No schema partitioning, no read replicas, no multi-region design, no encryption of individual PII fields.
- Dashboards and alert thresholds are illustrative; they have never seen production traffic. SLO numbers are targets, not measurements.
- The frontend has no i18n, no offline mode, no automated visual regression tests; browser E2E is a script, not a full Playwright suite.
- **Two frontends (Angular and React) mean every UI change is made twice.** The API types are hand-written copies in both projects (generate them from `specs/openapi.json` next); one shared browser suite keeps behaviour aligned. The React edition was tested in Edge only, has no server-state library (React Query), no SSR, and its Docker image and Compose profile were never built or run (no Docker).
- Several GitHub Actions are referenced by tag, not by commit SHA.

Security limitations

Educational, synthetic implementation. Not penetration-tested; no formal threat-modelling workshop; no WAF/DDoS layer implemented; secrets management is specified (External Secrets, Secrets Manager) but not deployed; no SIEM rules written; no key rotation procedure for data-encryption keys (there are none). See the residual-risk list in the threat model.

Future enhancements (prioritised)

1. **Run the unexecuted pieces** where Docker/runners exist: Compose stack, Testcontainers on PostgreSQL 16, GitHub Actions, k6, kubeconform/hadolint, Grafana dashboards against real traffic.
2. Parallel outbox workers with `FOR UPDATE SKIP LOCKED`; optional relay to Kafka/RabbitMQ when a second consumer appears (ADR-006).
3. Participants table + keyset pagination for account history; table partitioning for `transactions`, `ledger_entries`, `audit_events`.
4. OIDC authorization-code + PKCE (or BFF with HttpOnly cookies) in the SPA; refresh-token rotation; MFA step-up for privileged roles.
5. Distributed rate limiting (Redis or gateway); abuse analytics on audit events.
6. Contract tests with the real clearing provider's sandbox; settlement-file reconciliation as a second detection net.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

7. Canary / progressive delivery with automated analysis; SBOM + signed images + provenance (SLSA).
8. Property-based tests for money and state-machine invariants; PIT mutation testing in CI.
9. Playwright suite replacing the puppeteer script; accessibility audit (axe) in CI.
10 Fees, FX, scheduled transfers, statements - each as a spec in `specs/` following the same workflow.
.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Appendix A: Captured PostgreSQL query plans

Generated by `PostgresFeaturesIT.queryPlans` against a real embedded PostgreSQL 14.10 (no Docker). Source: `backend/platform-service/target/query-plans.txt`, copied here after the final build.

Setup: EXPLAIN (ANALYZE, COSTS OFF) on PostgreSQL with 100,000 synthetic transactions over 20 accounts - the SHAPE of the plan matters; timings are omitted on purpose (shared dev machine)

Account history for a SPARSE account (planner switches to the per-account indexes)

```
select * from transactions where (source_account_id = 'e06be2e5-2a8a-467b-8400-802fa1203d1c' or
target_account_id = 'e06be2e5-2a8a-467b-8400-802fa1203d1c') order by created_at desc, id limit 20
```

```
Limit (actual rows=1 loops=1)
  ->  Sort (actual rows=1 loops=1)
        Sort Key: created_at DESC, id
        Sort Method: quicksort  Memory: 25kB
        ->  Bitmap Heap Scan on transactions (actual rows=1 loops=1)
              Recheck Cond: ((source_account_id = 'e06be2e5-2a8a-467b-8400-802fa1203d1c'::uuid) OR
(target_account_id = 'e06be2e5-2a8a-467b-8400-802fa1203d1c'::uuid))
              Heap Blocks: exact=1
              ->  BitmapOr (actual rows=0 loops=1)
                    ->  Bitmap Index Scan on ix_tx_source_created (actual rows=0 loops=1)
                          Index Cond: (source_account_id =
'e06be2e5-2a8a-467b-8400-802fa1203d1c'::uuid)
                    ->  Bitmap Index Scan on ix_tx_target_created (actual rows=3 loops=1)
                          Index Cond: (target_account_id =
'e06be2e5-2a8a-467b-8400-802fa1203d1c'::uuid)
```

Account history for a BUSY account (5,000 rows): newest-first walk of ix_tx_created with a filter

```
select * from transactions where (source_account_id = '63305c83-031c-43df-bf3c-7efb79c4c683' or
target_account_id = '63305c83-031c-43df-bf3c-7efb79c4c683') order by created_at desc, id limit 20
```

```
Limit (actual rows=20 loops=1)
  ->  Incremental Sort (actual rows=20 loops=1)
        Sort Key: created_at DESC, id
        Presorted Key: created_at
        Full-sort Groups: 1  Sort Method: quicksort  Average Memory: 30kB  Peak Memory: 30kB
        ->  Index Scan using ix_tx_created on transactions (actual rows=21 loops=1)
              Filter: ((source_account_id = '63305c83-031c-43df-bf3c-7efb79c4c683'::uuid) OR
(target_account_id = '63305c83-031c-43df-bf3c-7efb79c4c683'::uuid))
              Rows Removed by Filter: 491
```

Operations attention queue (partial index ix_tx_attention)

```
select * from transactions where status in ('PENDING_REVIEW','RECONCILIATION_REQUIRED','FAILED') order
by created_at desc limit 50
```

```
Limit (actual rows=17 loops=1)
  ->  Index Scan using ix_tx_attention on transactions (actual rows=17 loops=1)
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Idempotency lookup (unique index uq_idempotency_principal_key)

```
select * from idempotency_records where principal = 'alice.customer' and idempotency_key =
'k-1234567890'
```

```
Index Scan using uq_idempotency_principal_key on idempotency_records (actual rows=0 loops=1)
  Index Cond: (((principal)::text = 'alice.customer'::text) AND ((idempotency_key)::text =
'k-1234567890'::text))
```

Outbox polling (ix_outbox_status_available)

```
select id from outbox_events where status = 'NEW' and available_at <= now() order by created_at limit
20
```

```
Limit (actual rows=0 loops=1)
  ->  Sort (actual rows=0 loops=1)
        Sort Key: created_at
        Sort Method: quicksort  Memory: 25kB
        ->  Index Scan using ix_outbox_status_available on outbox_events (actual rows=0 loops=1)
              Index Cond: (((status)::text = 'NEW'::text) AND (available_at <= now()))
```

Daily withdrawal limit sum (ix_tx_source_created)

```
select coalesce(sum(amount),0) from transactions where source_account_id = '63305c83-031c-43df-
bf3c-7efb79c4c683' and transaction_type = 'WITHDRAWAL' and created_at >= now() - interval '24 hours'
and status in ('PROCESSING','COMPLETED','RECONCILIATION_REQUIRED')
```

```
Aggregate (actual rows=1 loops=1)
  ->  Bitmap Heap Scan on transactions (actual rows=4319 loops=1)
        Recheck Cond: ((source_account_id = '63305c83-031c-43df-bf3c-7efb79c4c683'::uuid) AND
(created_at >= (now() - '24:00:00'::interval)))
        Filter: (((transaction_type)::text = 'WITHDRAWAL'::text) AND ((status)::text = ANY
('{PROCESSING,COMPLETED,RECONCILIATION_REQUIRED}'::text[])))
        Heap Blocks: exact=1661
        ->  Bitmap Index Scan on ix_tx_source_created (actual rows=4319 loops=1)
              Index Cond: ((source_account_id = '63305c83-031c-43df-bf3c-7efb79c4c683'::uuid) AND
(created_at >= (now() - '24:00:00'::interval)))
```

Row lock taken when applying a transfer (primary key)

```
select * from accounts where id = '63305c83-031c-43df-bf3c-7efb79c4c683' for update
```

```
LockRows (actual rows=1 loops=1)
  ->  Index Scan using pk_accounts on accounts (actual rows=1 loops=1)
        Index Cond: (id = '63305c83-031c-43df-bf3c-7efb79c4c683'::uuid)
```

Open reconciliation records (partial unique index uq_recon_open_per_tx)

```
select * from reconciliation_records where status = 'OPEN' order by opened_at limit 50
```

```
Limit (actual rows=1 loops=1)
  ->  Index Scan using ix_recon_status on reconciliation_records (actual rows=1 loops=1)
        Index Cond: ((status)::text = 'OPEN'::text)
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Appendix B: Validation summary

All numbers below come from commands run on one Windows developer machine (8 CPUs, Java 21 GraalVM, Node 24, Microsoft Edge headless). Raw outputs are in `docs/validation/` and `.work/` (not shipped). Nothing here is a benchmark or a production measurement.

1. Executed and passing

Check	Command / source	Result
Backend full build (clean), both modules	`mvn -B clean verify` (backend/)	**BUILD SUCCESS**, 6 min 42 s
Unit tests (platform-service)	surefire	**81 passed**, 5 skipped (Testcontainers class, see section 2)
Integration tests (platform-service)	failsafe	**95 passed**, 5 skipped (same Testcontainers class)
clearing-simulator tests	surefire	**6 passed**
Real PostgreSQL 14.10 (embedded, no Docker): flow, concurrency, features	`PostgresFlowIT` 18, `PostgresConcurrencyIT` 5, `PostgresFeaturesIT` 4	**27 passed** (included in the 95)
Concurrency on H2 and PostgreSQL	`ConcurrencyIT` 5, `PostgresConcurrencyIT` 5	passed: no lost update, no overspend, no deadlock, daily limit holds, duplicate delivery harmless
Angular lint	`ng lint`	clean
Angular unit tests	Karma/Jasmine (headless)	**58 / 58 passed**
Angular production build	`ng build`	OK, about 341 kB initial bundle
React type-check and lint	`tsc --noEmit` (strict), `eslint` (typescript-eslint, react-hooks, jsx-a11y)	clean
React unit and component tests	Vitest + Testing Library, scripted fake backend	**90 / 90 passed**
React production build	`vite build`	OK, 279 kB main chunk (89 kB gzip) plus lazy route chunks
Live API end-to-end	`tests/e2e/e2e-api.mjs` against the running stack	**50 / 50** (`e2e-results.json`)
Live API end-to-end with chaos (simulator 5xx, timeouts)	`--with-chaos`	**55 / 55** (`e2e-results-with-chaos.json`)
Browser end-to-end, Angular UI (headless Edge, real backend)	`tests/e2e/ui-e2e.mjs`	**19 / 19**, 11 screenshots (`ui-e2e-results.json`, `docs/screenshots`)
Browser end-to-end, React UI (same suite, same backend, same run)	`tests/e2e/ui-e2e.mjs --flavor react`	**19 / 19**, 11 screenshots (`ui-e2e-results-react.json`, `docs/screenshots-react`)
Actuator on a separate management port	`scripts/check-management-port.mjs`	5 / 5 PASS (readiness, liveness, prometheus on management port; 404 for actuator and 401 for API on the main port)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Check	Command / source	Result
Static infrastructure validation	`scripts/validate-infra.py`	**0 failures** (compose, Kubernetes manifests, probes, security contexts, `automountServiceAccountToken: false`, alert-to-runbook mapping, no committed secrets)
Fail-closed security	start without a JWT decoder	application refuses to start (checked manually)
Local load baseline	`performance/local-baseline.mjs`	`perf-baseline.json`; all invariants held (300 completed withdrawals on a 300.00 balance, money conserved on opposing transfers, 25 idempotency keys gave 25 deposits, 0 ledger integrity violations). Outbox drain rose from about 23 to about 34 tx/s after the scheduler change (`perf-baseline-before-drain-loop.json`)
Mutation check on the concurrency tests	removed the row lock / lock ordering by hand, then restored	removing the row lock failed 4 of 5 `ConcurrencyIT` tests and the `LedgerServiceTest` lock tests; removing lock ordering failed the lock-order unit test

Defects found by the final run and fixed before packaging:

- `ServerErrorIT.lockTimeoutIs503` asserted that the body does not contain `relation`, which also matched the field name `correlationId`. The assertion now checks for the leaked SQL text. The production code was correct.
- The three PostgreSQL ITs failed once with "Gave up waiting for server to start after 10000 ms" (cold first start of the native PostgreSQL binary). The startup wait is now 120 s. The four affected classes were re-run (30 / 30), then the whole build was re-run clean.
- `validate-infra.py` crashed on the test-overlay simulator Deployment, which had no rolling-update strategy; the manifest now declares one, and the validator also asserts that no pod mounts a service-account token.

Found while adding the React edition (all fixed, then every suite re-run on the final code):

- The first browser run of the React UI got **403** on sign-in. The development proxy rewrote the `Host` header, so the API saw a cross-origin request from an origin that is not on its CORS allow-list. The proxy now preserves `Host` (as nginx does); the allow-list was **not** changed and both UIs pass with the default configuration.
- Two timing races in the browser suite itself (a page checked before its accounts had loaded; a native `<select>` read before its options arrived). The suite now waits for the data; it was re-run for both UIs.
- One React test (the first of a file, which pays for the lazy route import) failed once in a full parallel run because Testing Library's default 1 s wait was too short on a loaded machine. The wait is now 5 s (test setup only; no application code changed) and four consecutive full runs passed 90 / 90.
- Native browser validation bubbles would have pre-empted the app's own messages on two small forms in the React UI; those forms are `noValidate`, like the others.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

2. Written but NOT executed

Item	Why not
Docker image builds, Docker Compose stack (including the React image and the `react` compose profile)	Docker is not installed on this machine. Compose and Dockerfiles are validated statically only
React edition: other browsers, visual regression, automated accessibility audit	Edge only; ESLint `jsx-a11y` is static analysis, not an audit
Prometheus, Grafana, OpenTelemetry collector, Jaeger	need the Compose stack. The configuration and alert rules are validated statically; metric names were confirmed against the real `/actuator/prometheus` output
Kubernetes apply, `kubeconform`, `hadolint`	no cluster and the tools are not installed; CI runs `kubeconform` and `hadolint`
GitHub Actions workflow	never run on a runner
k6 scripts	k6 is not installed (the Node baseline above covers the same scenarios on a small scale)
Testcontainers PostgreSQL concurrency test	skipped without Docker (5 tests, reported by both Maven phases)
AWS / cloud deployment	conceptual design only
Oracle, MongoDB	conceptual sections only
Multi-agent AI workflow definitions	definitions only, never executed
PostgreSQL 16 (Compose and CI target)	only 14.10 was executed

3. Honest caveats

- Test and load numbers are from a shared developer laptop with an in-memory H2 database (load baseline) or an embedded PostgreSQL (integration tests). They show relative behaviour and invariants, not capacity.
- The clearing network is a deterministic simulator; its contract (idempotent on `reference`, supports lookup) is an assumption.
- Synthetic data only. The local token issuer is for development and tests and is disabled outside the `local` and `test` profiles.
- The application was not penetration-tested.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Appendix C: Operations console screenshots

Screenshots captured by the browser end-to-end run (headless Edge against the production bundle and the live backend; synthetic data only).

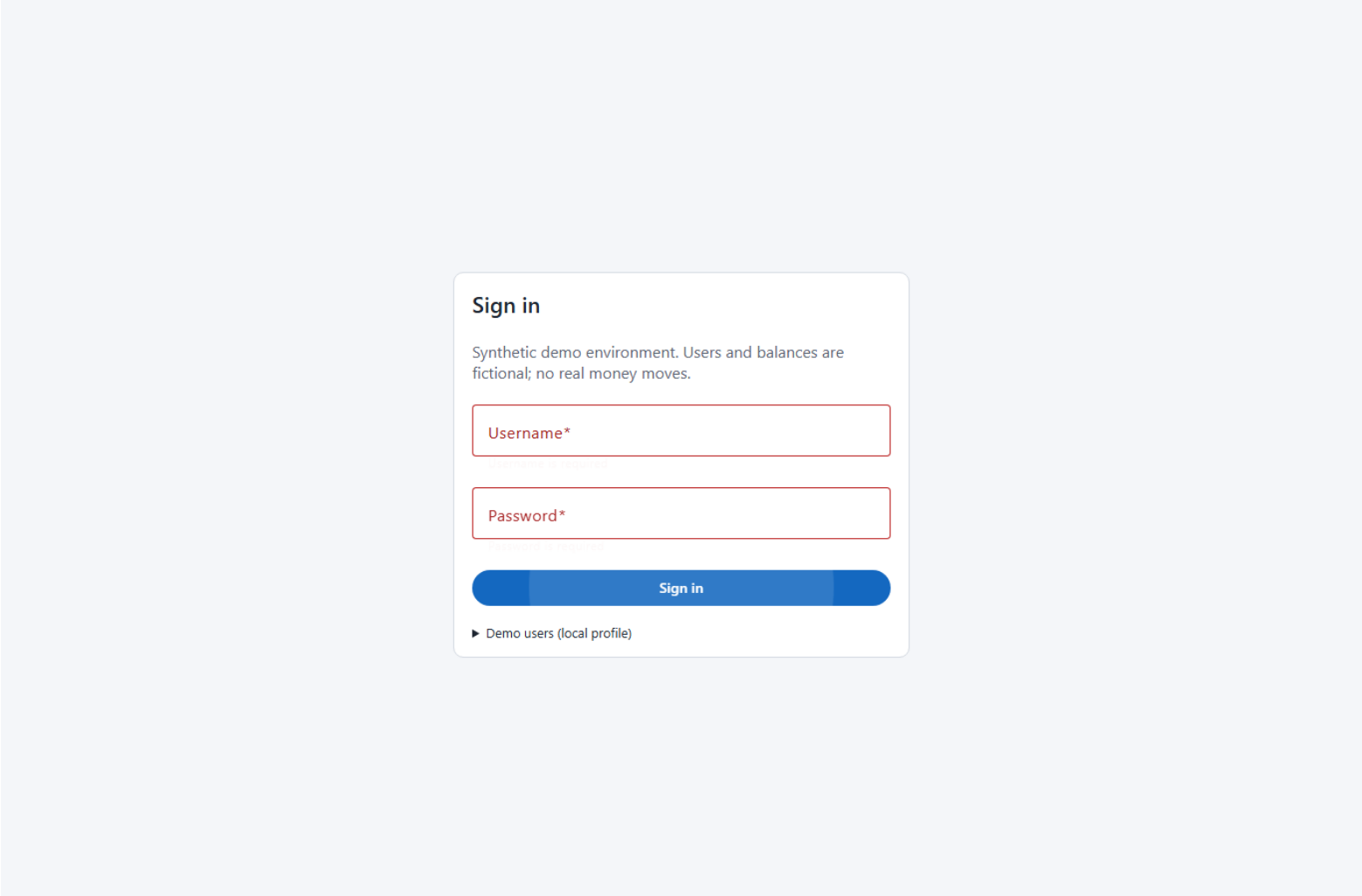


Figure 18. Sign-in screen

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

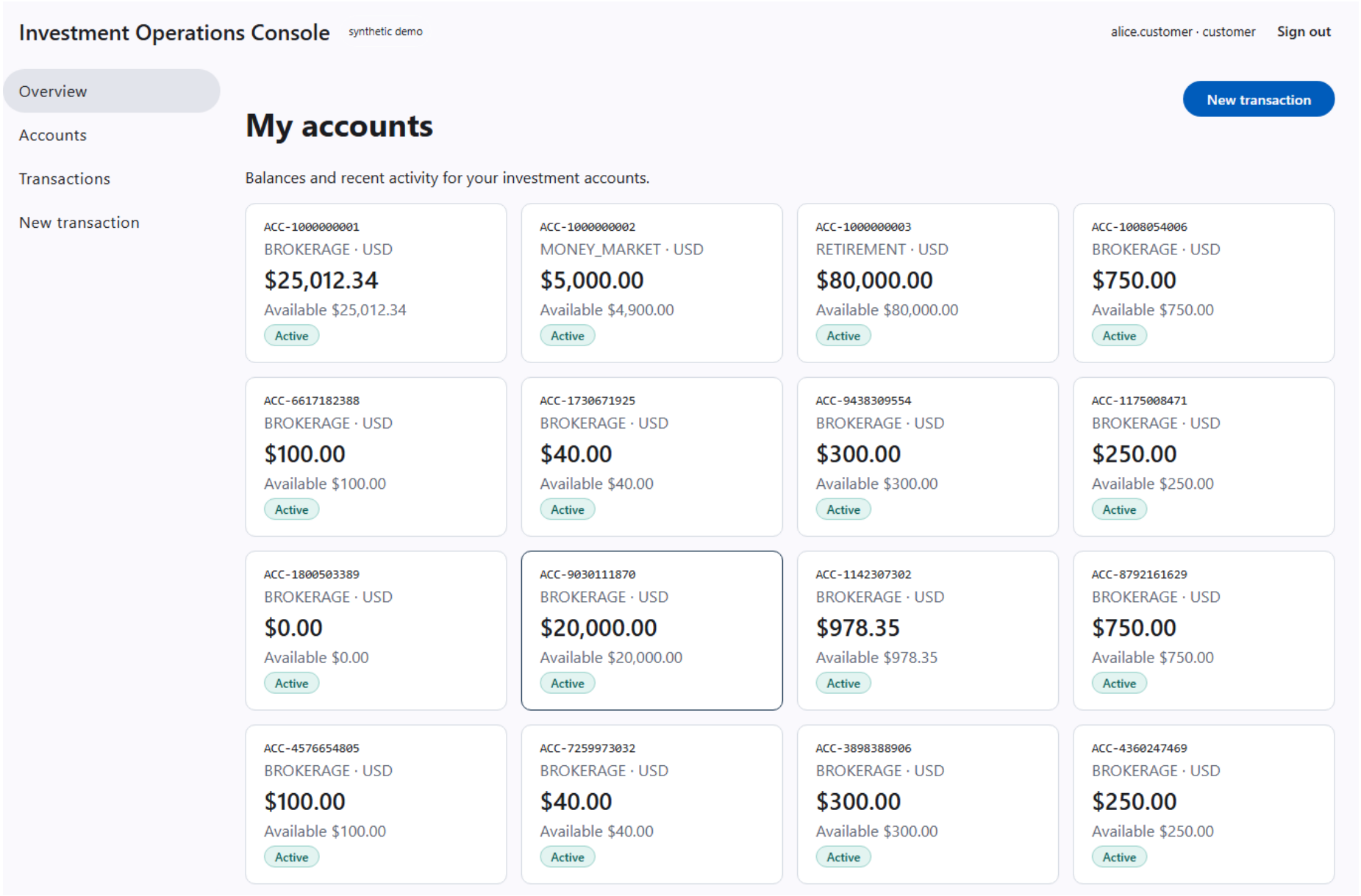


Figure 19. Customer portfolio

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

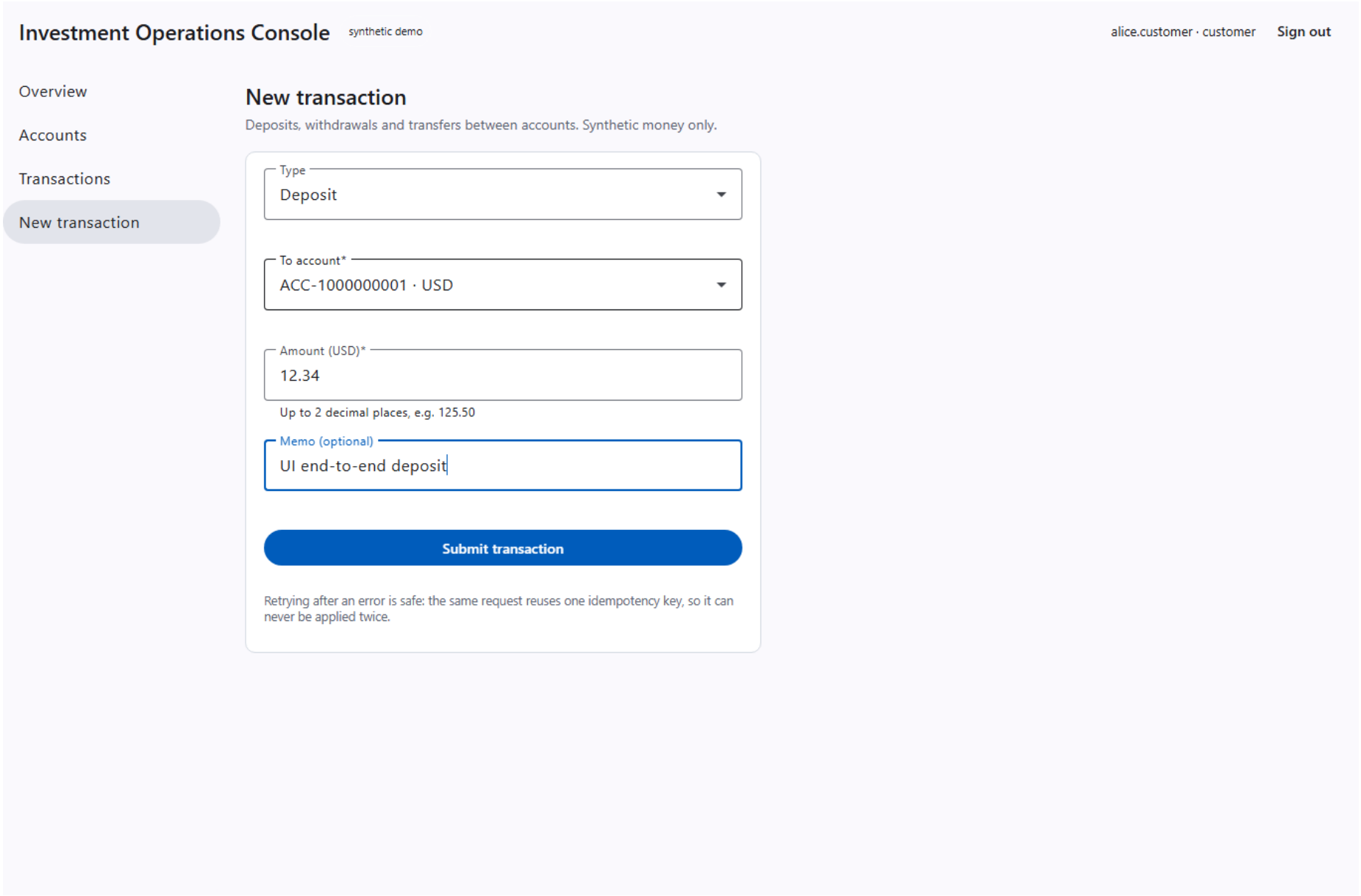


Figure 20. New transaction form

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

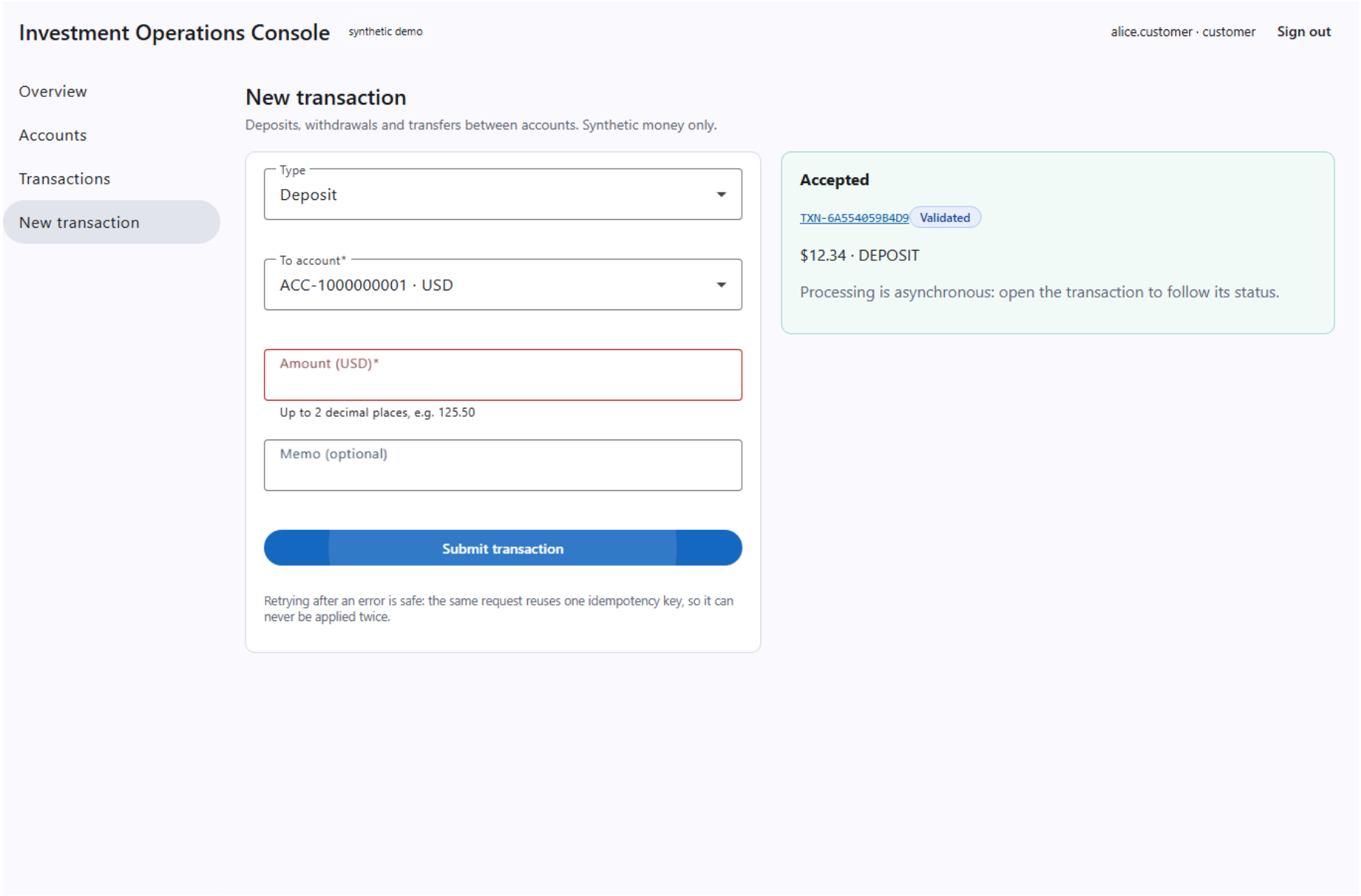


Figure 21. Transaction accepted

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

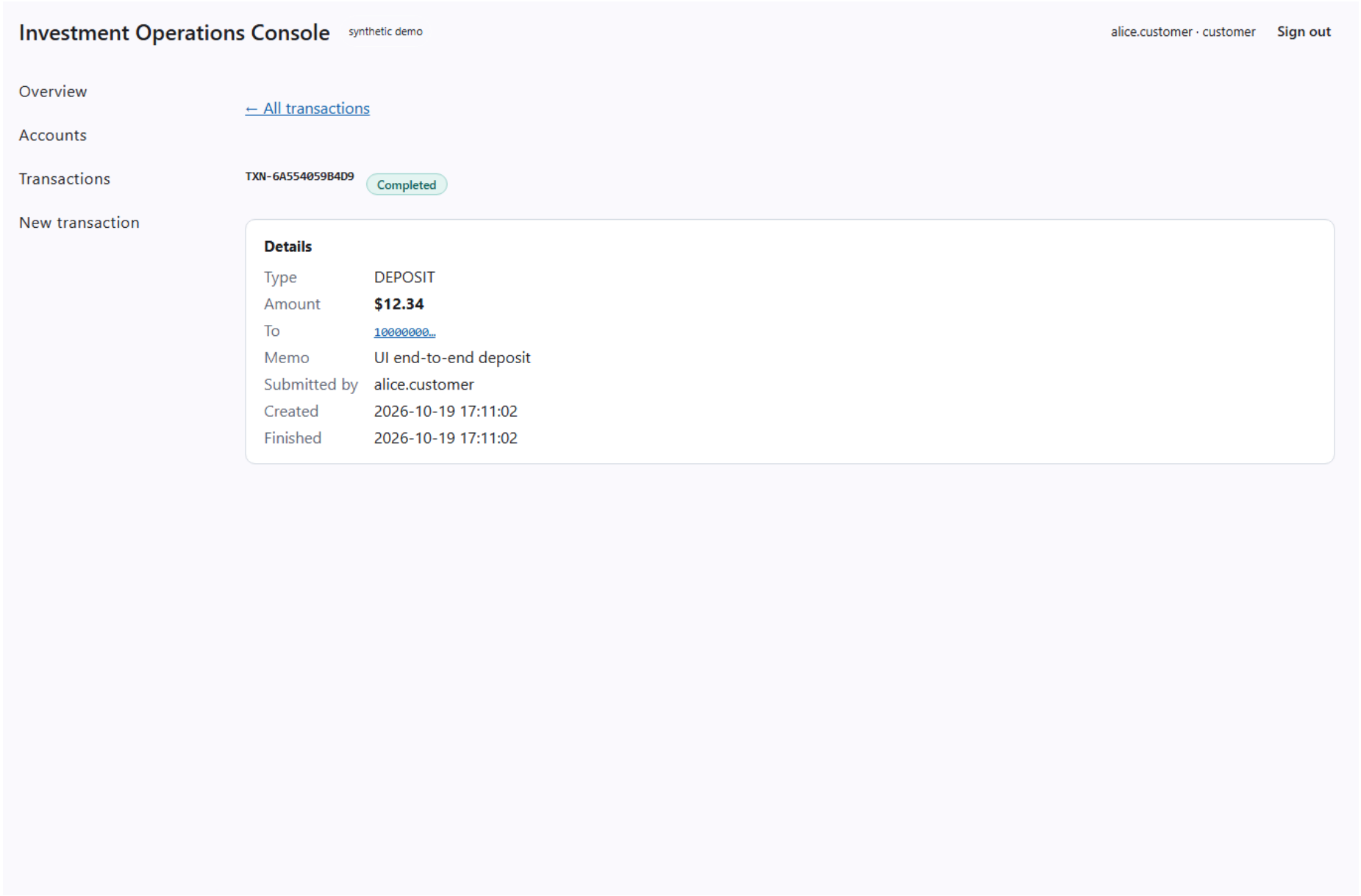


Figure 22. Transaction detail

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

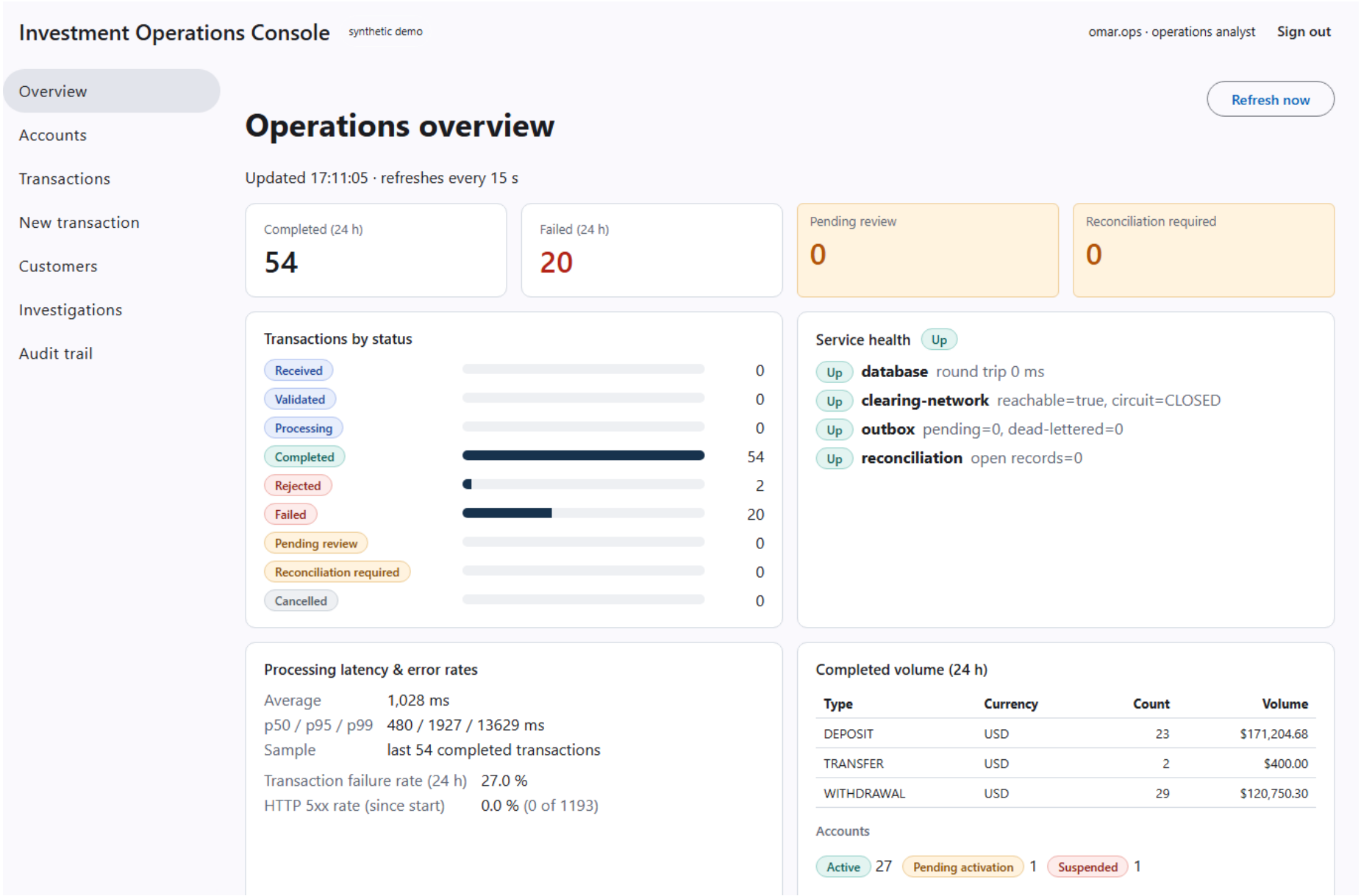


Figure 23. Operations dashboard

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

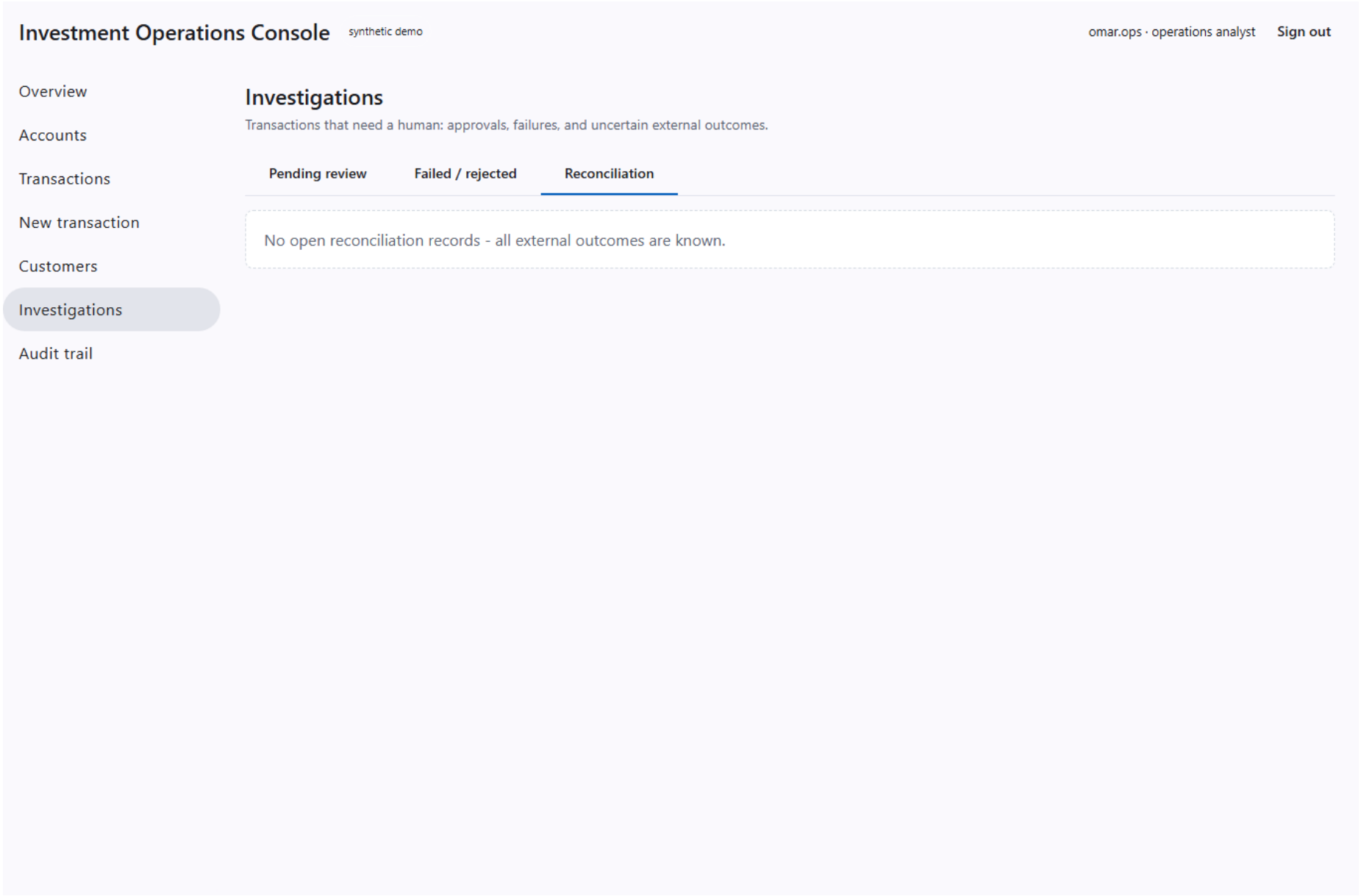


Figure 24. Investigations queue

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

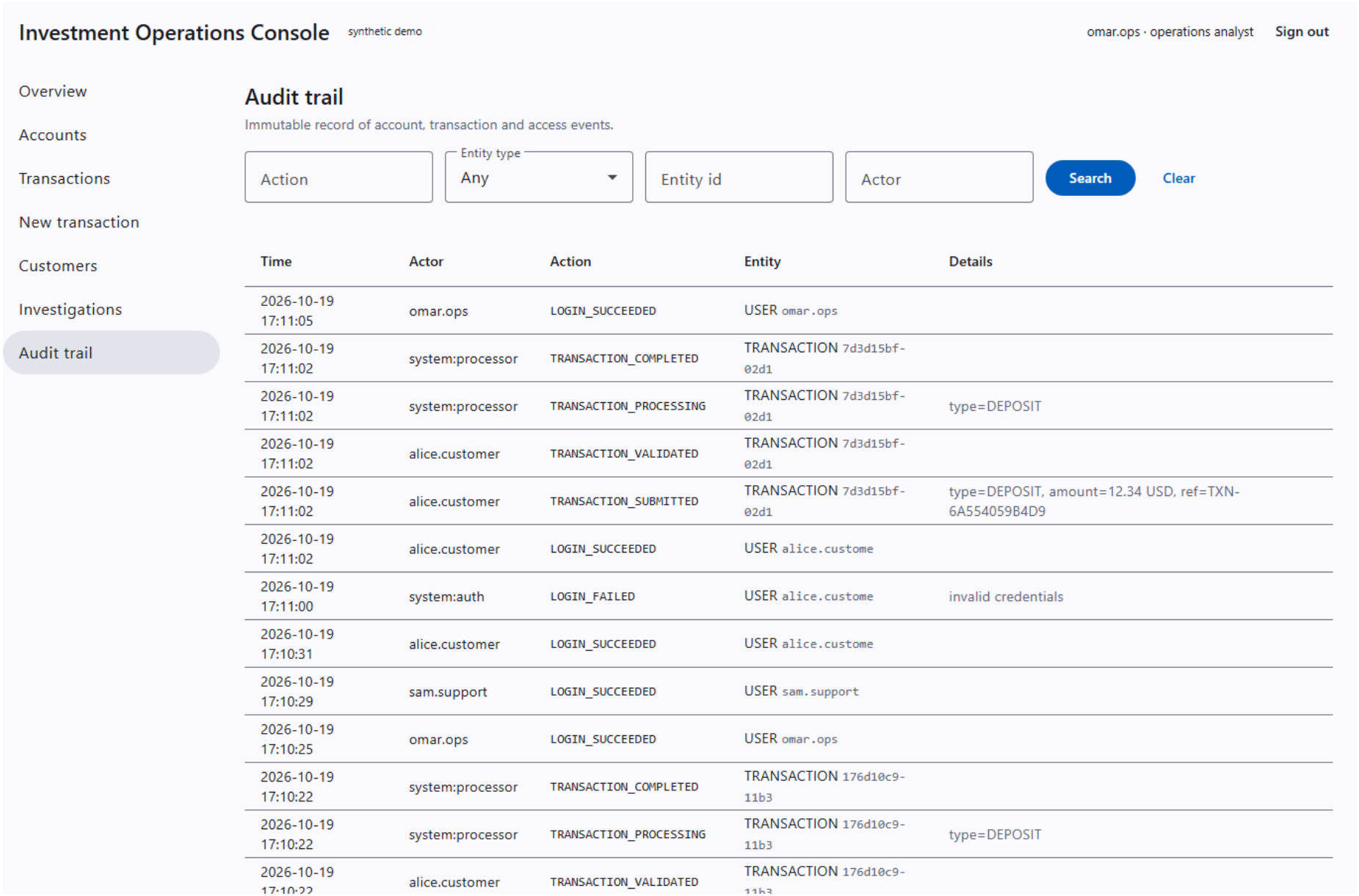


Figure 25. Audit trail

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

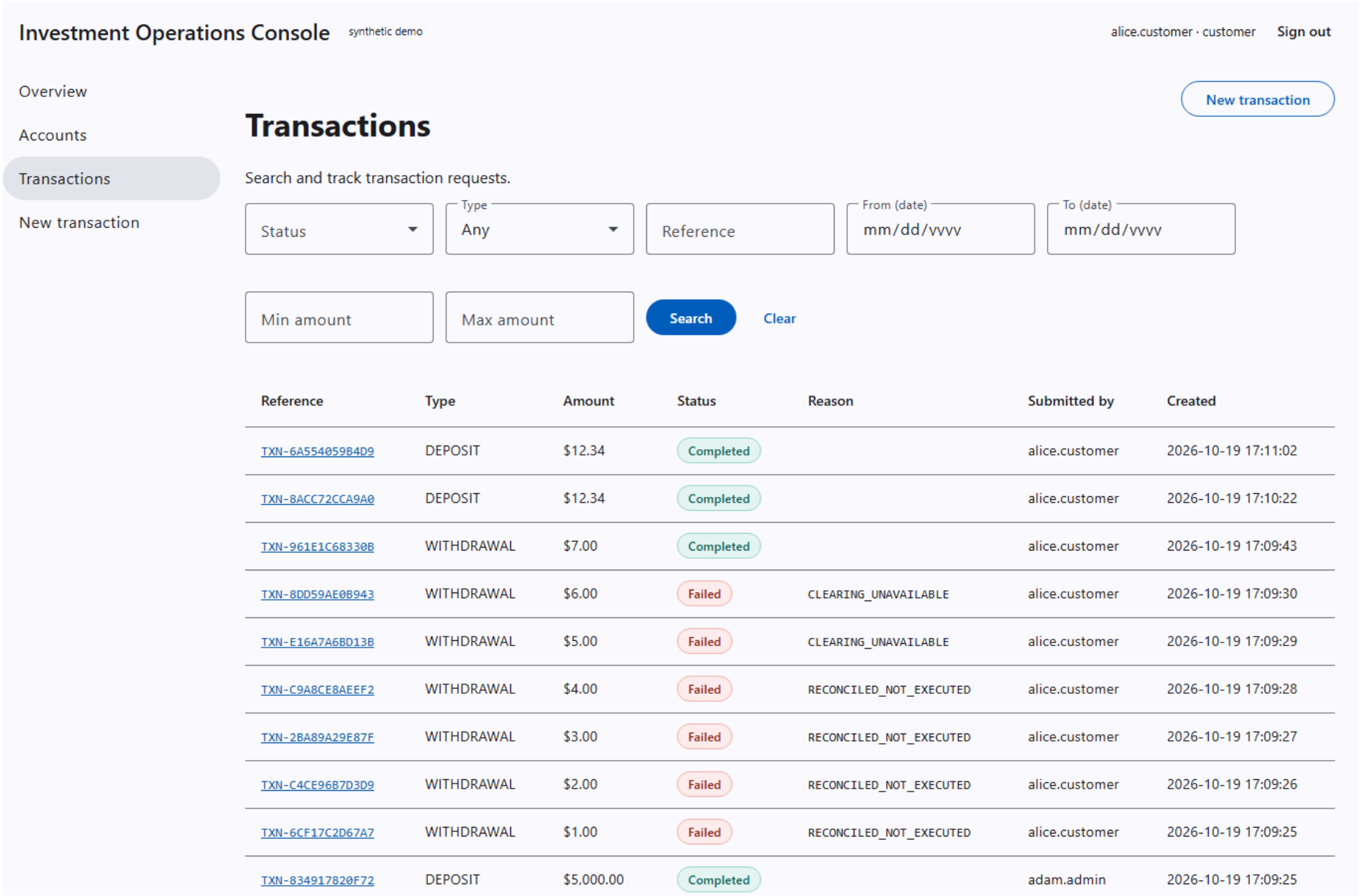


Figure 26. Transactions search

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

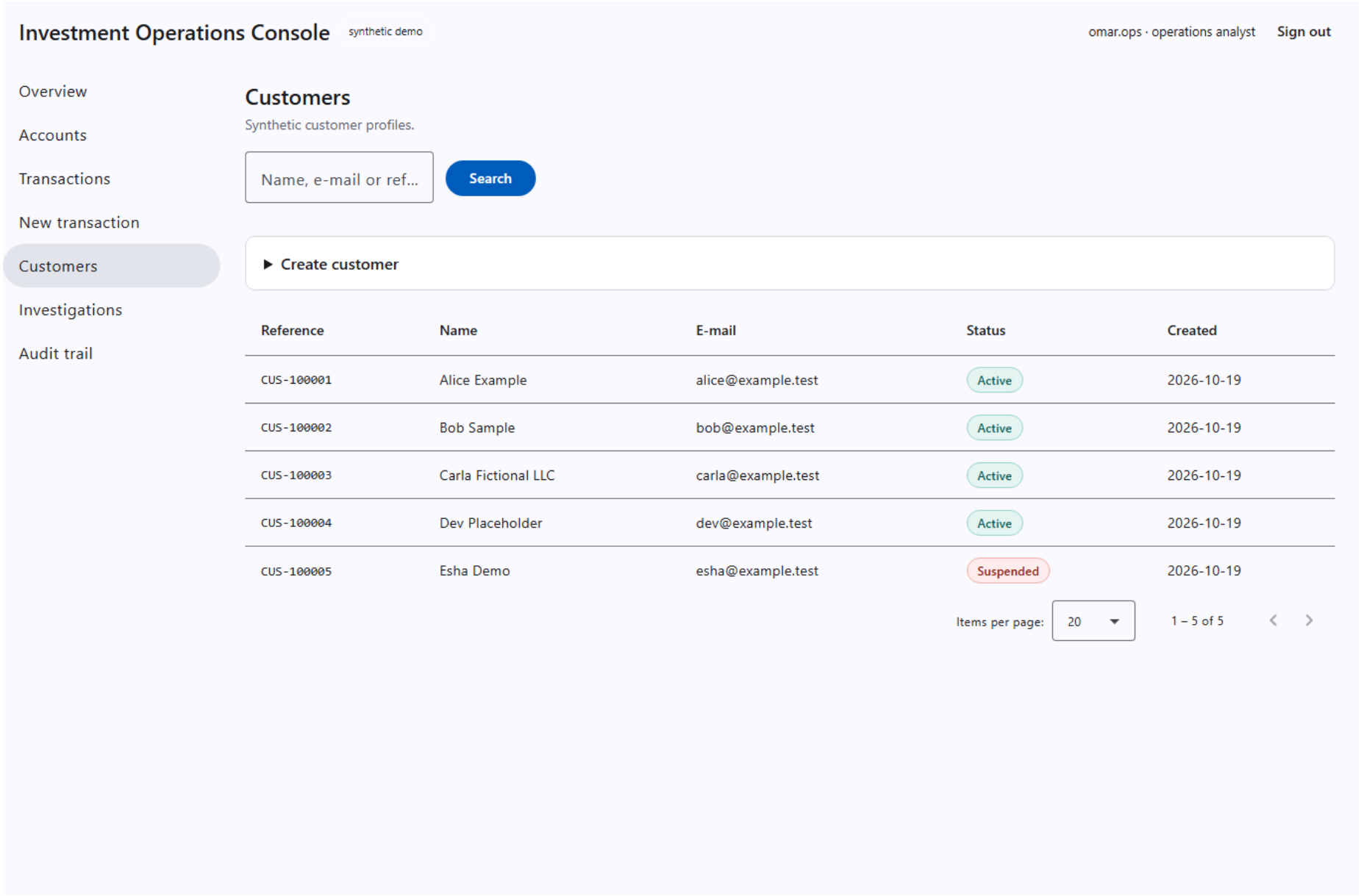


Figure 27. Customer directory

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

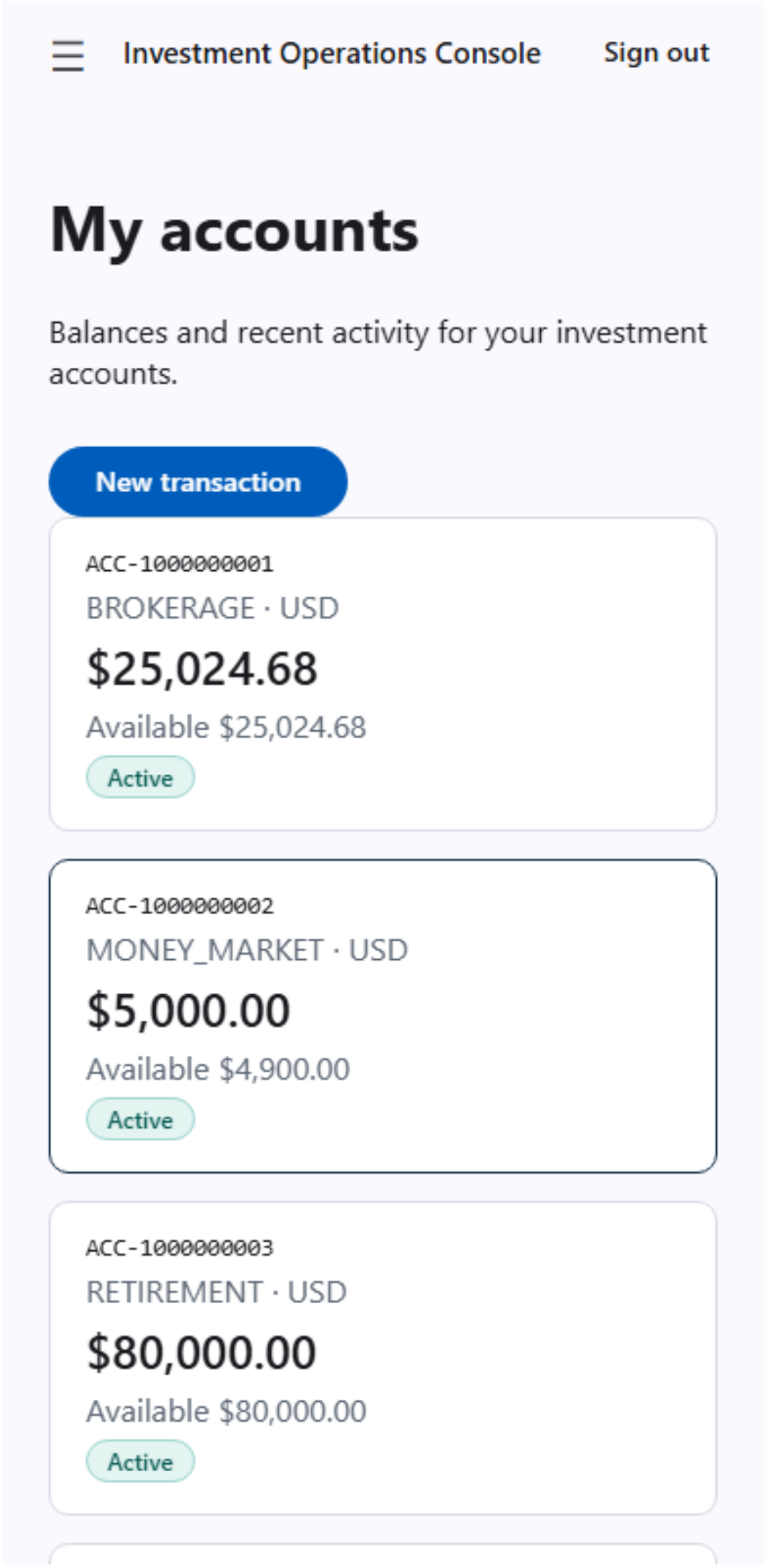


Figure 28. Responsive layout (390 px)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Appendix D: React console screenshots

The same screens in the React edition (`frontend-react/`), captured by the same browser end-to-end suite against the same backend (`--flavor react`).

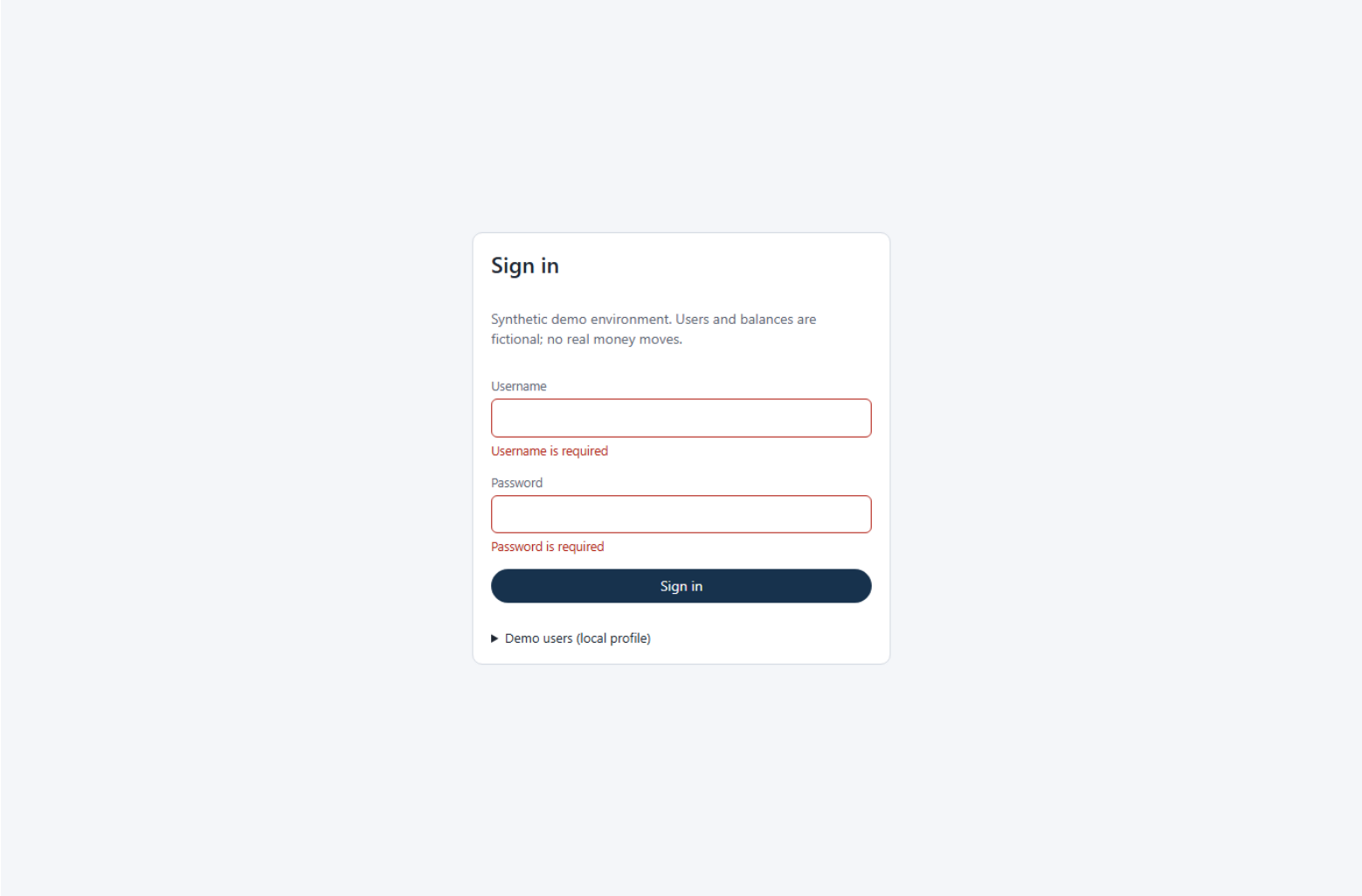


Figure 29. Sign-in screen (React)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

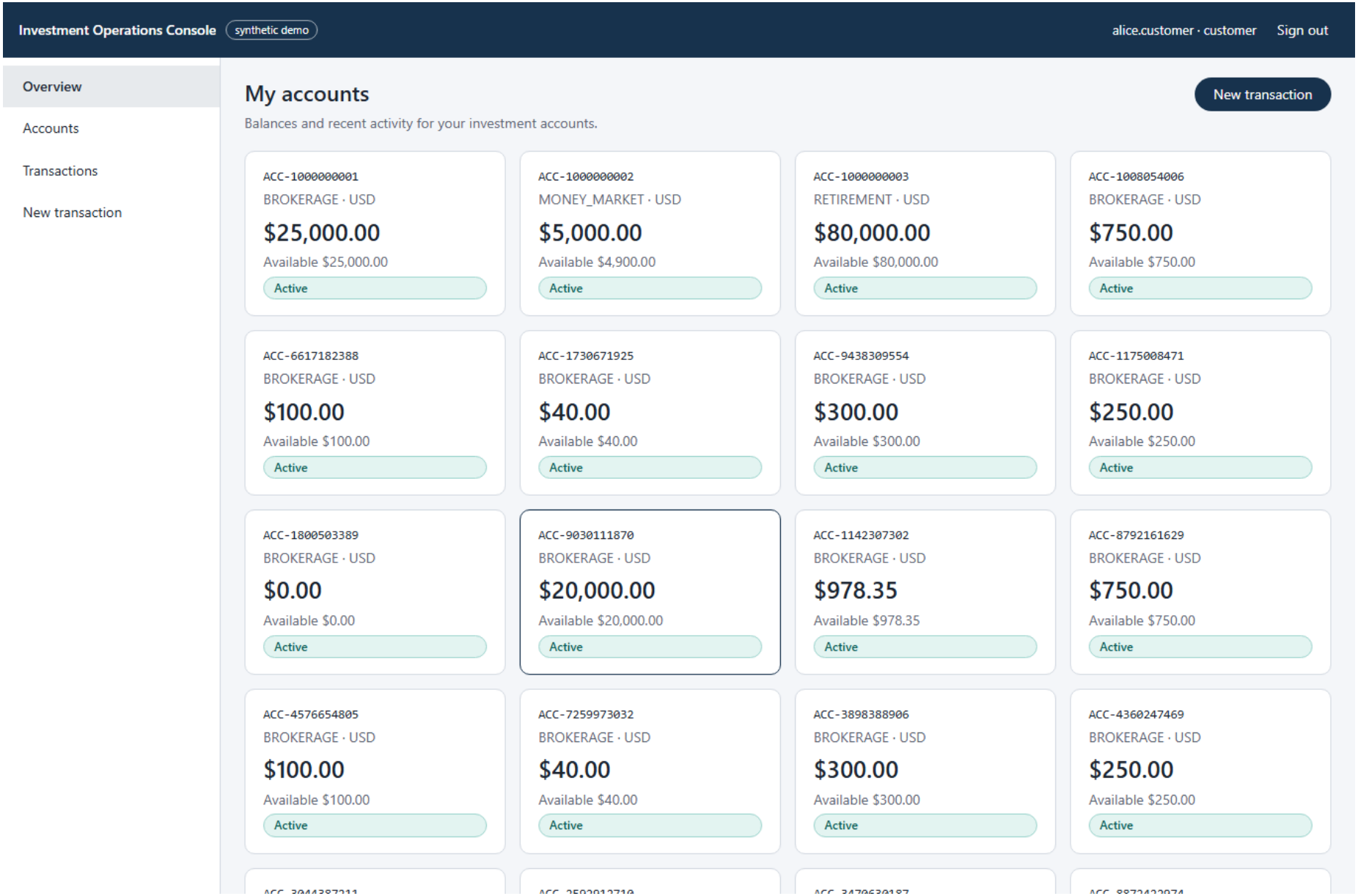


Figure 30. Customer portfolio (React)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

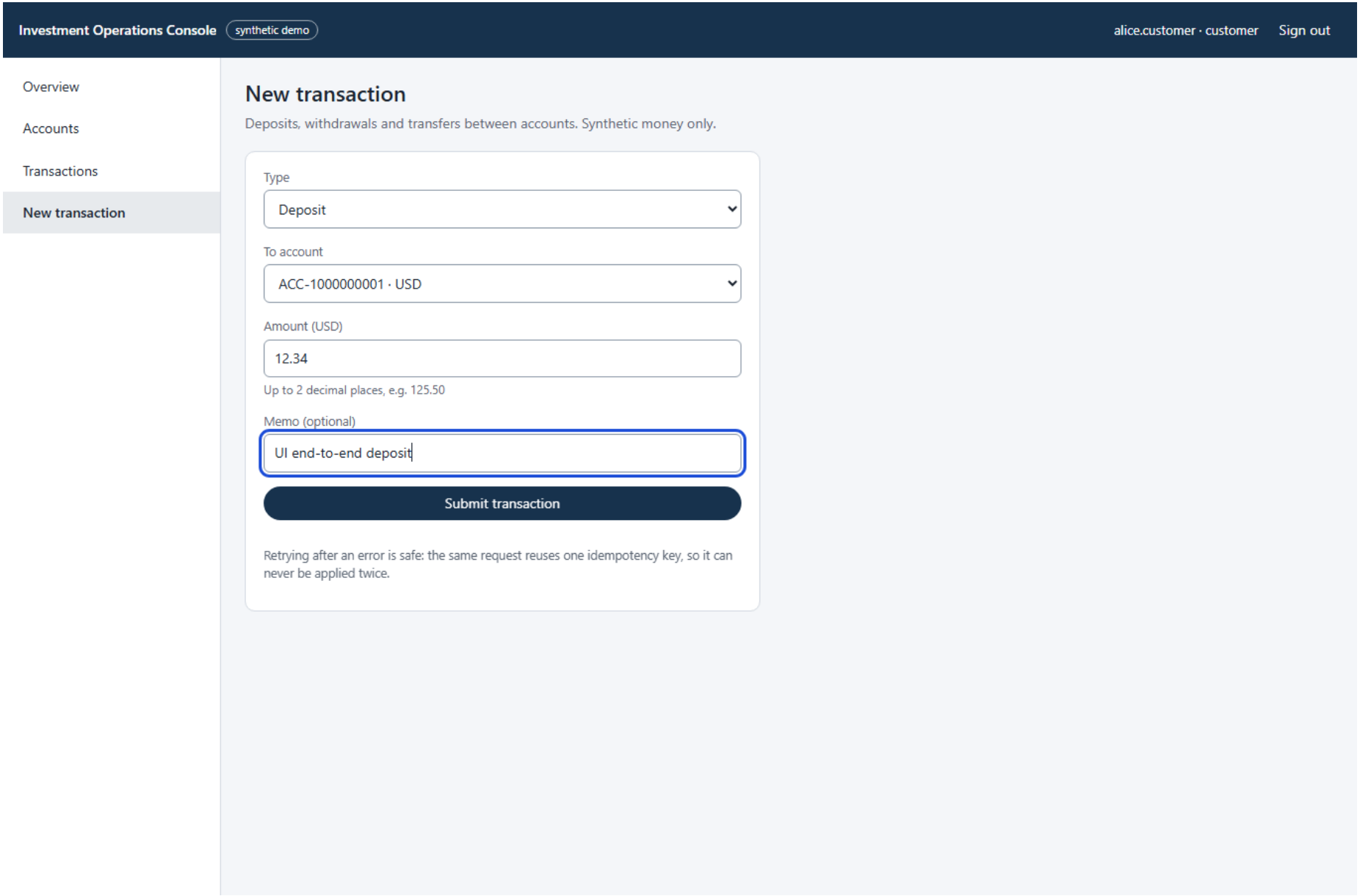


Figure 31. New transaction form (React)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

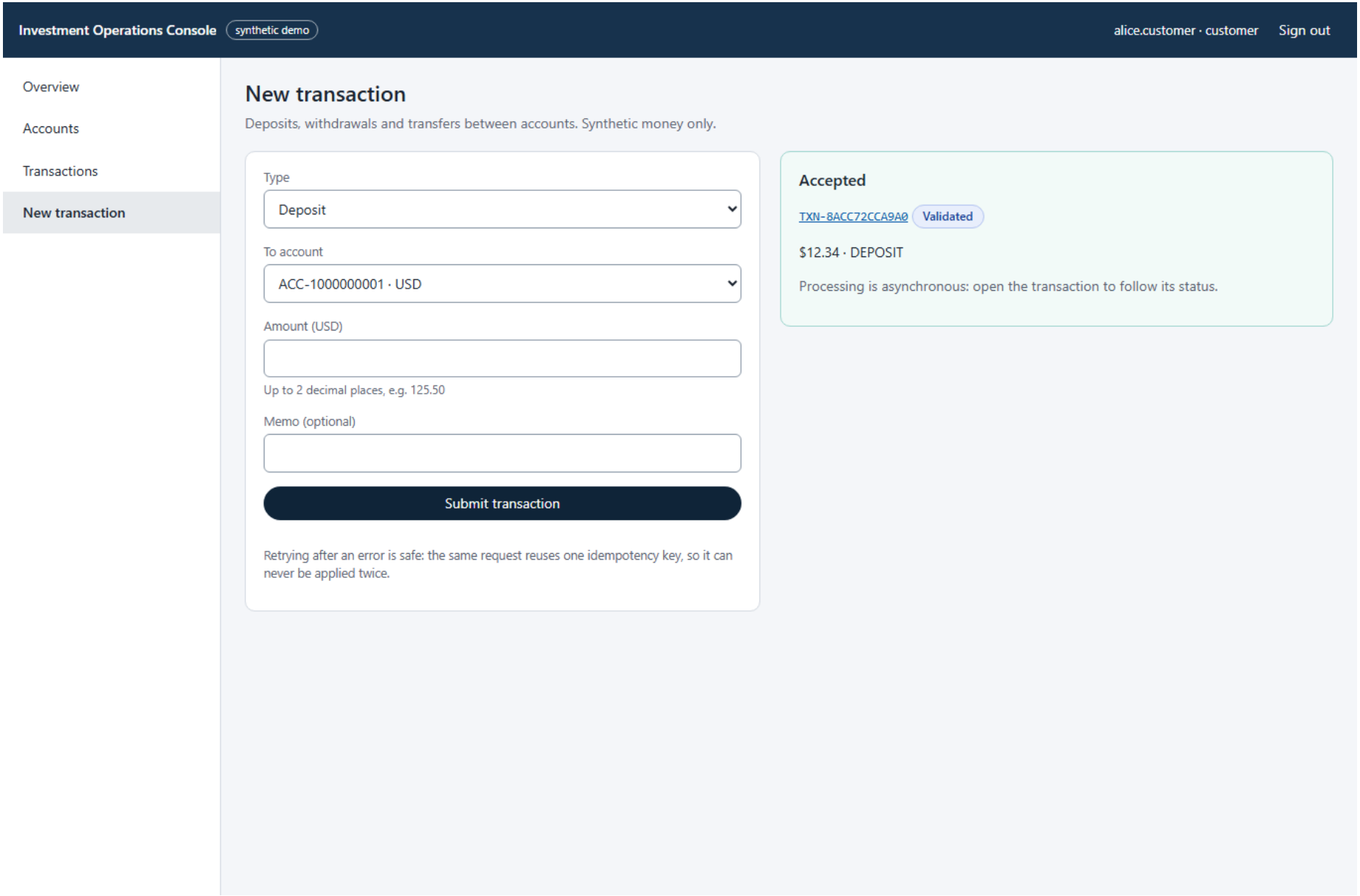


Figure 32. Transaction accepted (React)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

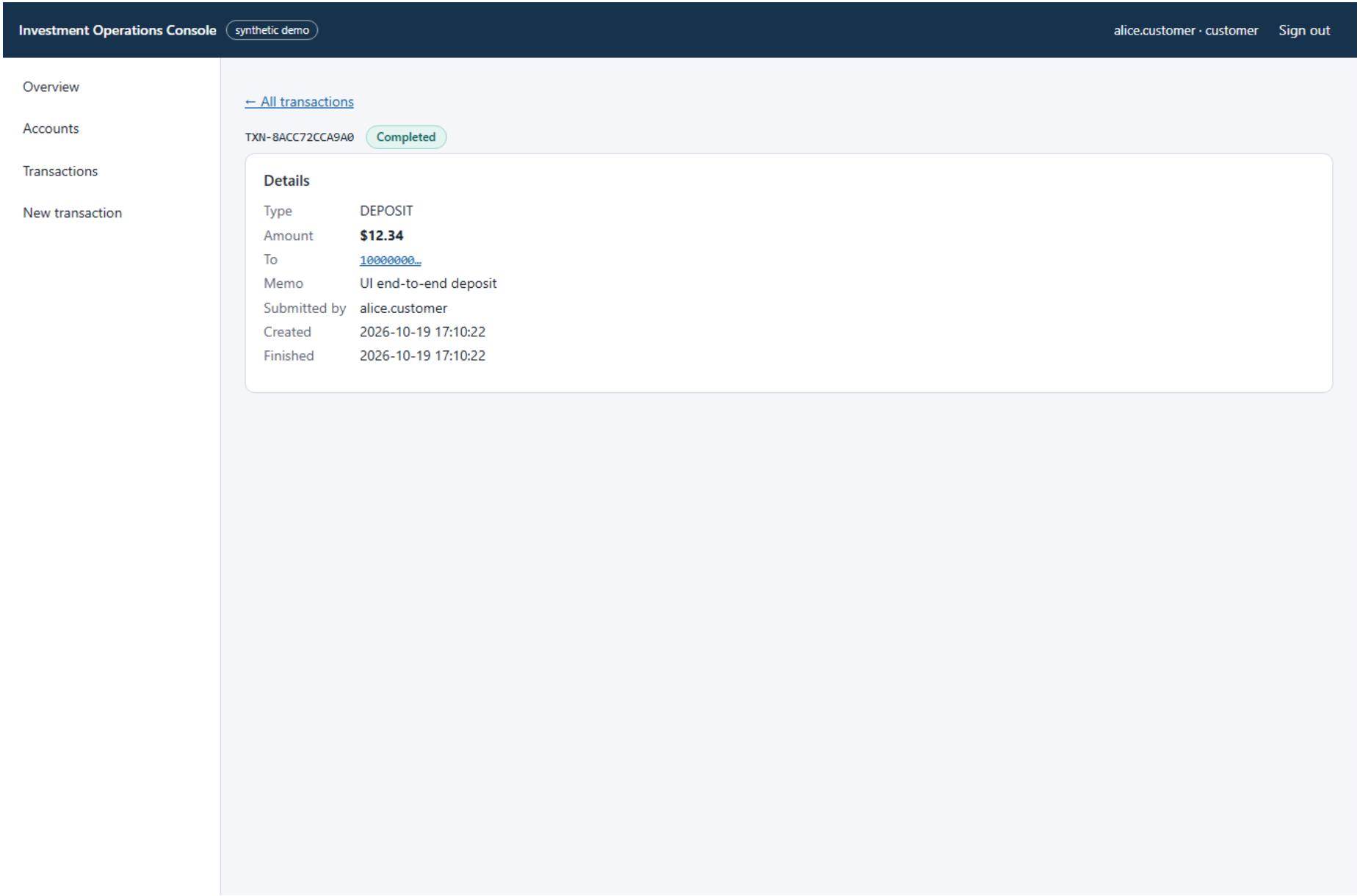


Figure 33. Transaction detail (React)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

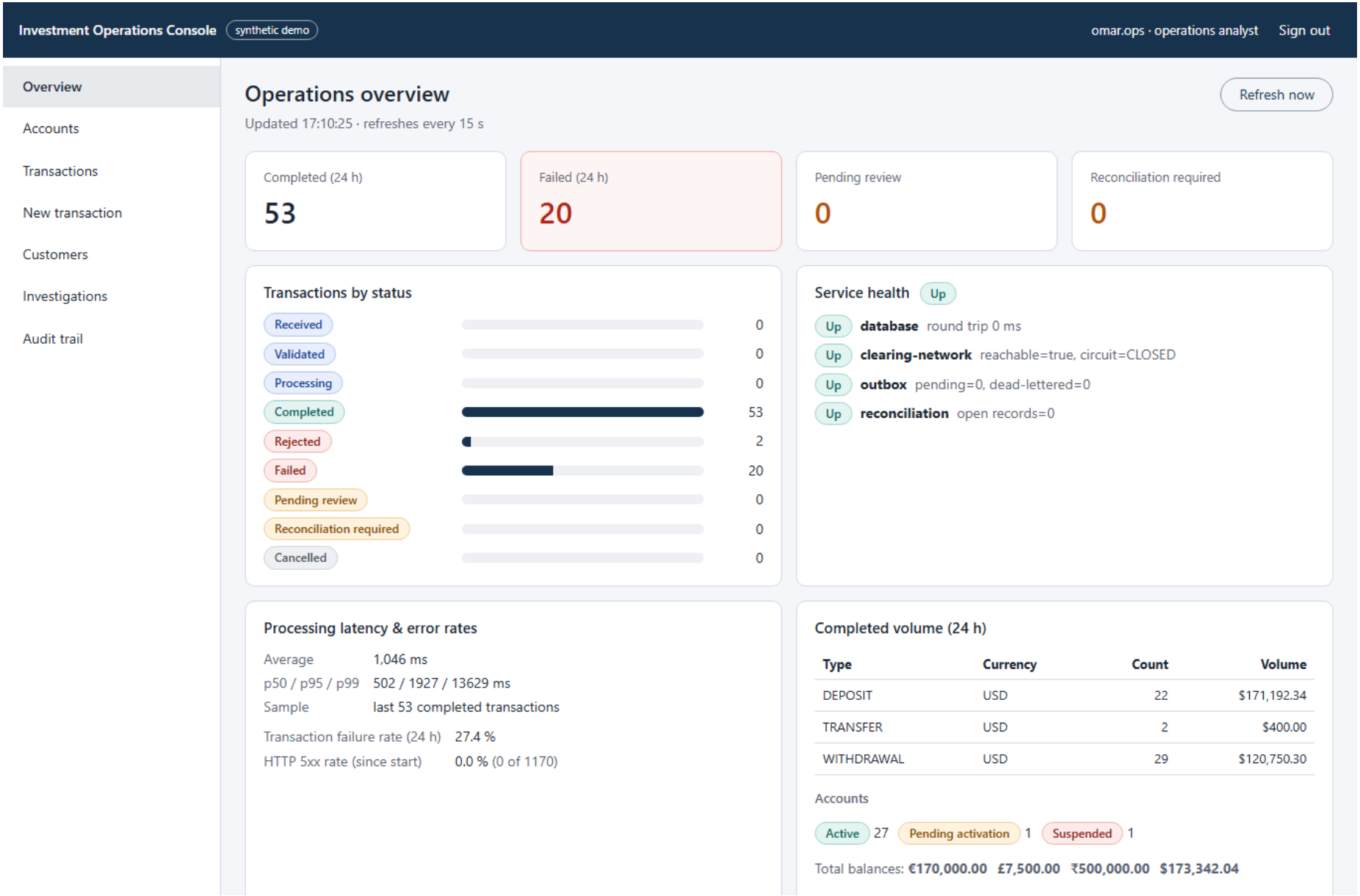


Figure 34. Operations dashboard (React)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

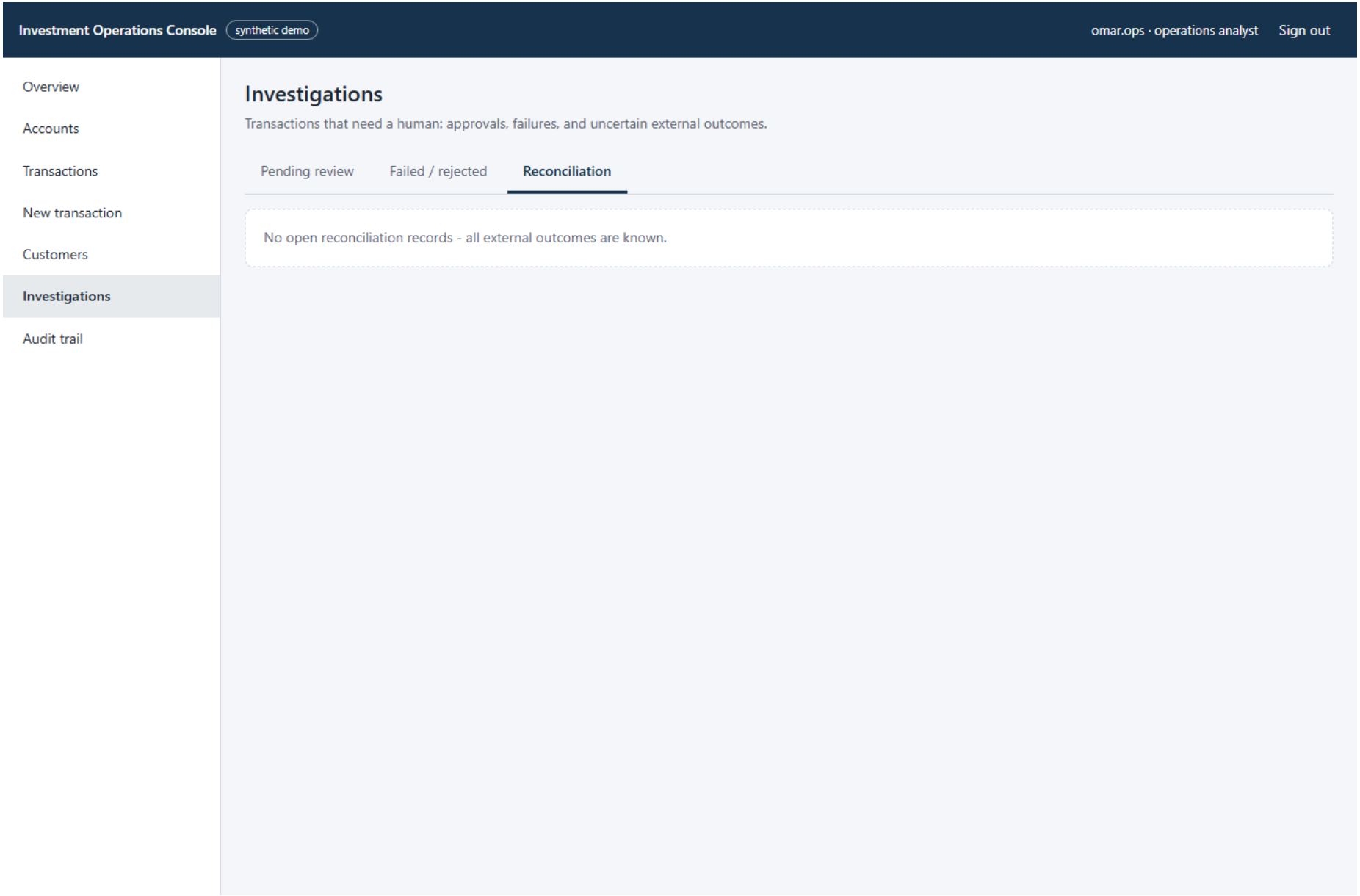


Figure 35. Investigations queue (React)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

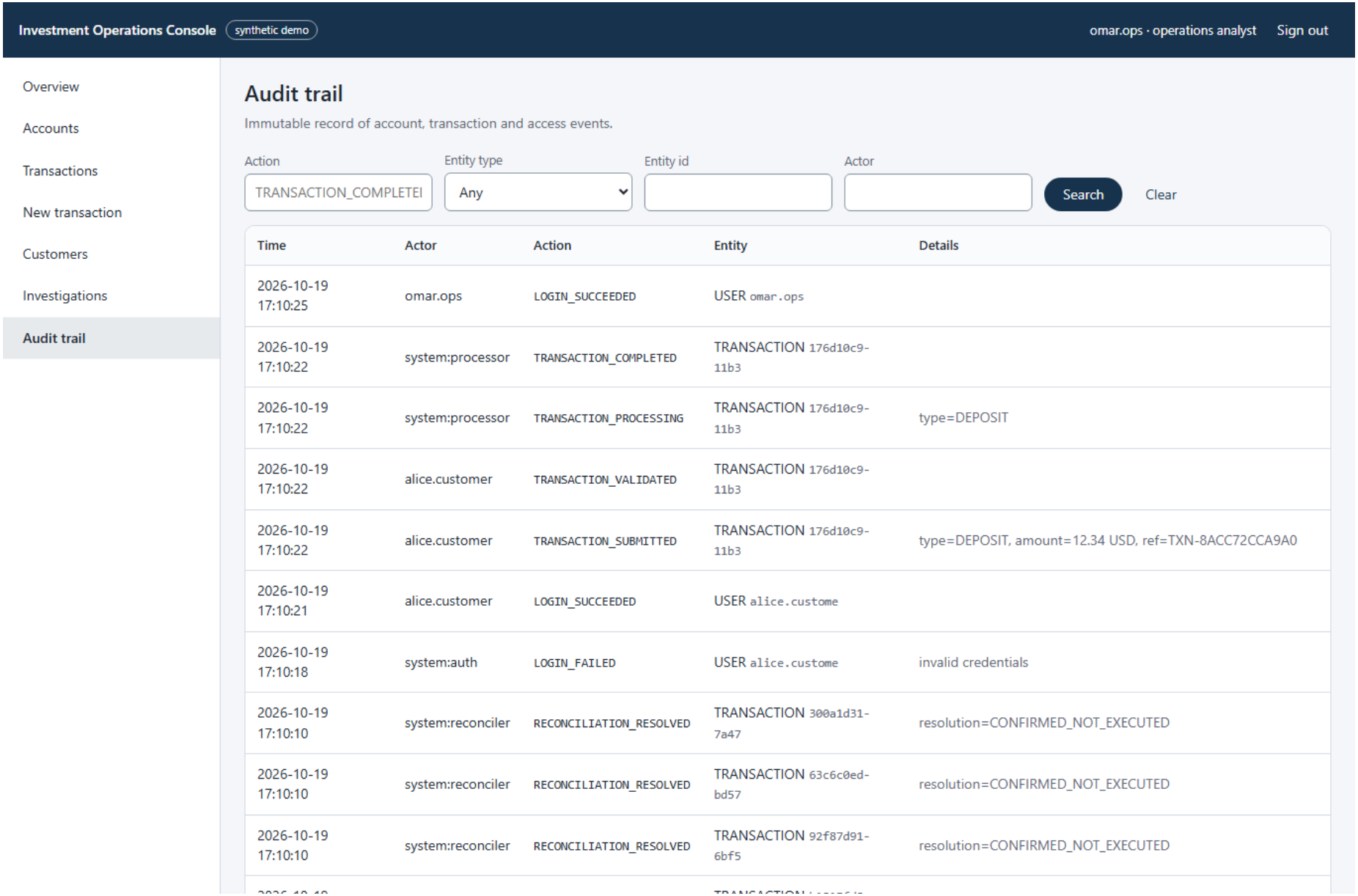


Figure 36. Audit trail (React)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

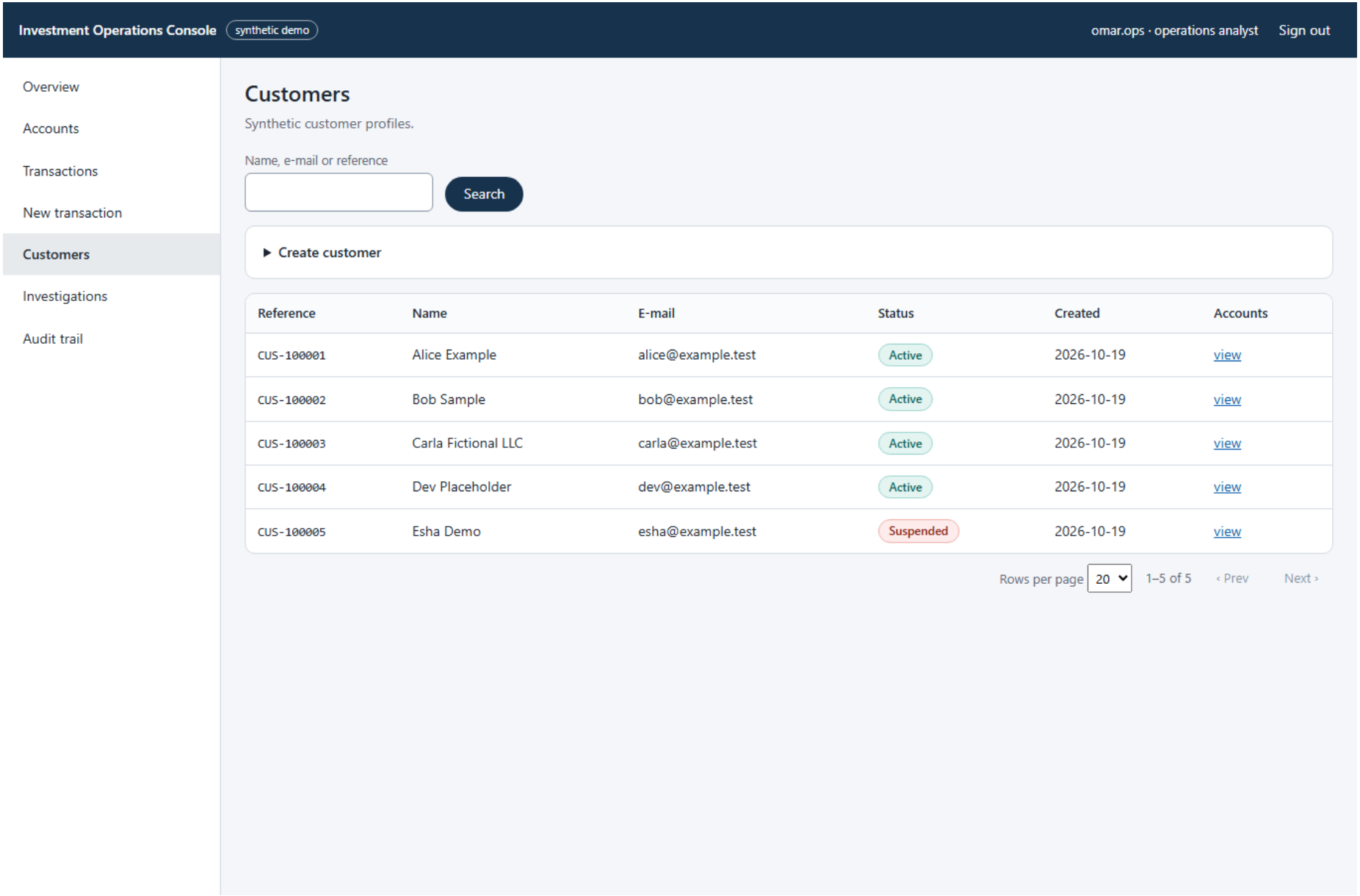


Figure 37. Customer directory (React)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

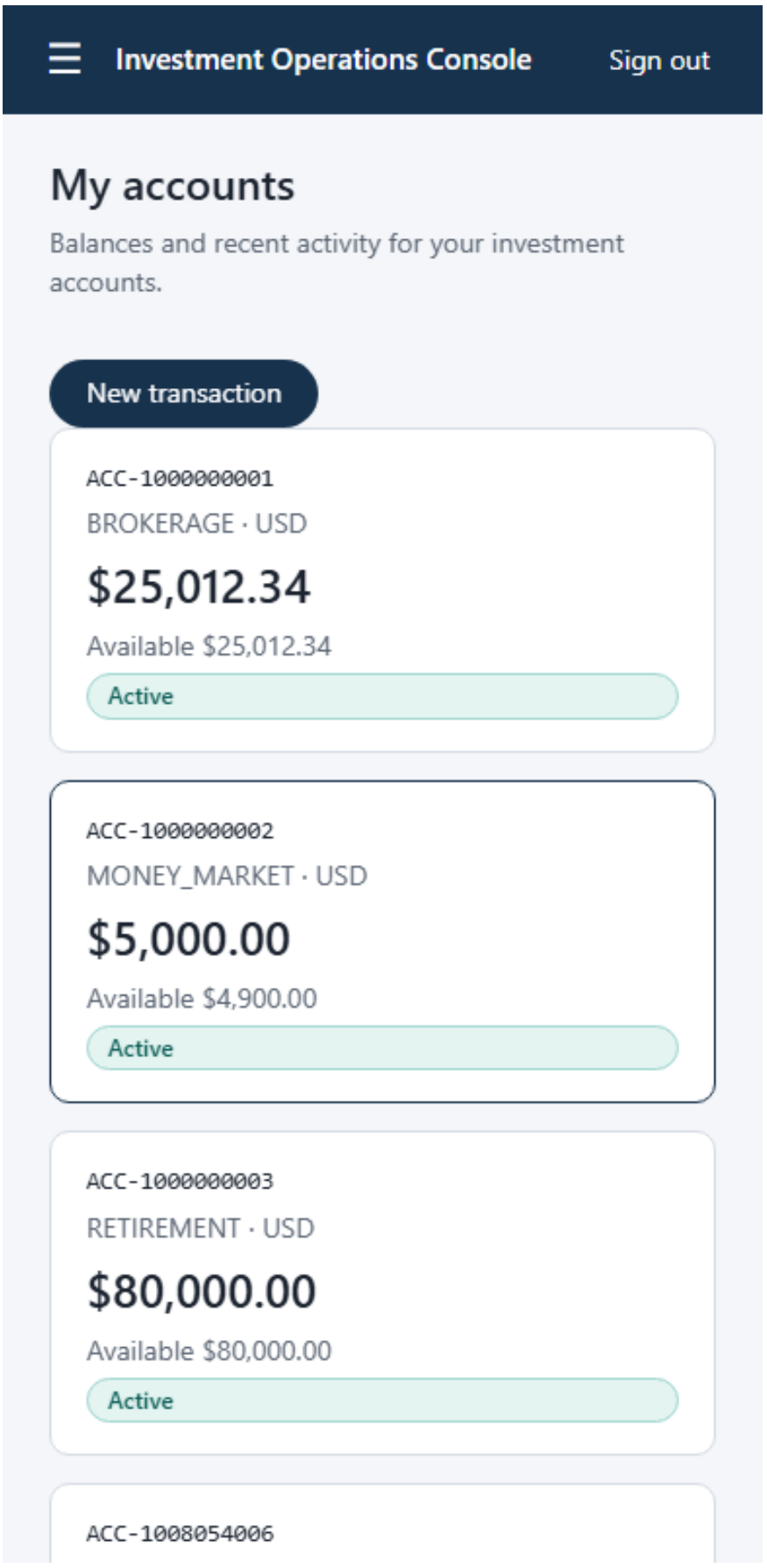


Figure 38. Responsive layout, 390 px (React)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.